PSTAT 131/231 Midterm Sheet

Fall 2026 • Midterm Wed Oct 28, 5:00–6:15 pm, CHEM 1179

Coverage (Canvas *Midterm Information Page*): Slide Set 1 (Intro to ML; bias–variance tradeoff; KNN) • Slide Set 2 (Linear models; feature engineering; EDA) • Slide Set 3 (Logistic regression; LDA/QDA; model evaluation, *up until Slide 42*) • Reading: *ROC Curves Deep Dive* • HW 1–3 • Labs 1–3. Format: ≤12 multiple choice + 4 short answer; open book/notes, paper only. Also included: practice exam + solutions, Sai's graded midterm from last quarter, and Slide Set 3 slides 43–93 (LDA/QDA; past the Slide-42 cutoff but tested by practice Q20, Lab 3 and HW 3).

Index: last pages. **Bold blue** locations = distilled notes, plain = raw transcription; 12 : C4 = page 12, grid column C (of A–H, left→right), row 4 (of 1–11, top→bottom).

Contents

PART I DISTILLED NOTES 1

1 ML foundations, bias–variance, KNN 1
1.1 What ML is; the basic model . . . 1
1.2 Training vs. test error; overfitting . . . 2
1.3 Bias–variance decomposition . . . 3
1.4 Classification: error rate and the Bayes classifier . . . 3
1.5 K-nearest neighbors (KNN) . . . 4

2 Linear models, feature engineering, EDA 5
2.1 Exploratory data analysis (EDA) . . . 5
2.2 Missing data & imputation . . . 5
2.3 Data splitting . . . 6
2.4 Linear regression: model and least squares . . . 6
2.5 Categorical predictors: dummy variables . . . 6
2.6 Interactions and nonlinear terms . . . 7
2.7 Regression diagnostics . . . 7
2.8 Feature engineering with recipes . . . 8
2.9 Linear regression vs. KNN regression . . . 8

3 Classification models: logistic regression, LDA, QDA, naive Bayes 8
3.1 Classification set-up; why not linear regression? . . . 8
3.2 The logistic regression model . . . 9
3.3 Estimating the coefficients: maximum likelihood . . . 9
3.4 Multiple logistic regression and confounding . . . 9
3.5 Predicted probabilities and the threshold . . . 9
3.6 Discriminant analysis: Bayes' theorem and the idea . . . 10
3.7 LDA with one predictor ($p = 1$) . . . 10
3.8 LDA with several predictors ($p > 1$) . . . 11
3.9 Quadratic discriminant analysis (QDA) . . . 11
3.10 Comparing classifiers . . . 11
3.11 Naive Bayes (Lab 3) . . . 11
3.12 HW3: four classifiers on the Titanic data . . . 12

4 Model evaluation: metrics, thresholds, ROC/AUC 12
4.1 Train vs. test: where performance is measured . . . 12
4.2 Regression metrics . . . 12
4.3 From probabilities to classes: the threshold . . . 13
4.4 The confusion matrix and its rates . . . 13
4.5 Class imbalance: why accuracy can mislead . . . 13
4.6 ROC curve and AUC . . . 13
4.7 Reading "ROC Curves Deep Dive": Titanic Model A vs. B . . . 14
4.8 yardstick code and output . . . 14

5 R code reference: tidyverse + tidymodels 15
5.1 The pipeline: split → recipe → spec → workflow → fit → predict → metrics 15
5.2 dplyr / tidyverse verbs . . . 15
5.3 ggplot2 for EDA . . . 15
5.4 rsample: splitting . . . 16
5.5 recipes: steps, selectors, prep/bake . . . 16
5.6 parsnip: model specs, engines, modes . . . 16
5.7 fit, predict, augment, tidy . . . 17
5.8 yardstick: metrics . . . 17
5.9 Common bugs / exam traps . . . 17

6 Exam prep: past questions, patterns, quick answers 18
6.1 Practice exam: every question, one line . . . 18
6.2 Sai's graded midterm: every question, where points were lost . . . 18
6.3 Question patterns and how to answer them . . . 19
6.4 60-second facts . . . 20
6.5 HW answers at a glance . . . 20

PART II RAW TRANSCRIBED MATERIAL (only what Part I does not cover) 22

7 Slide Set 1 — Intro to ML, bias–variance tradeoff, KNN (pp. 1–58) 22
7.1 Course overview and introduction . . . 22
7.2 The learning set-up: response, predictors, f and error . . . 22
7.3 Bias–variance tradeoff: flexibility . . . 22

8 Slide Set 1 (cont.) — pp. 59–116 22
8.1 Training MSE vs. test MSE; overfitting . . . 22
8.2 Bias-variance decomposition . . . 23
8.3 Classification: the Bayes classifier . . . 23

9 Slide Set 2 — Linear models, feature engineering, EDA 23
9.1 Linear regression . . . 23
9.2 Multiple linear regression . . . 23
9.3 Interactions, polynomial regression . . . 23

10 Slide Set 3 — Logistic regression & model evaluation (pp. 1–42, the tested part) 23
10.1 Classification: why logistic; Default data . . . 23
10.2 Confounding . . . 23
10.3 Confusion matrix, ROC . . . 23

11 Slide Set 3 (cont.) — Discriminant analysis: LDA & QDA (pp. 43–93; past the Slide-42 cutoff) 24
11.1 Why LDA? . . . 24
11.2 Geometric picture: gene-expression example . . . 24

12 Reading — ROC Curves Deep Dive 24
12.1 Recipe A . . . 24
12.2 Model A: ROC curve . . . 24
12.3 Model B: ROC curve . . . 24
12.4 Conclusions . . . 24

13 Lab 1 — R basics, dplyr, EDA (diamonds), data splitting 24
13.1 Basics of Data Processing . . . 25
13.2 Exploratory Data Analysis . . . 25
13.3 Data Splitting . . . 25

14 Lab 2 — Model fitting with tidymodels: linear regression, KNN, missing data 25
14.1 Recipe . . . 25
14.2 Linear Regression . . . 26
14.3 k-Nearest Neighbors . . . 26
14.4 Missing Data . . . 26

15 Lab 3 — Intro to binary classification (logistic, LDA, QDA, naive Bayes) 27
15.1 Logistic Regression . . . 27
15.2 LDA, QDA, Naive Bayes — training confusion matrices . . . 27
15.3 Fitting to Testing Data . . . 27

16 Homework 1–3: prompts, my submissions, solutions 27
16.1 Homework 1 – ML main ideas; EDA on mpg . . . 28
16.2 Homework 2 – linear regression and KNN (abalone) . . . 28
16.3 Homework 3 – binary classification (Titanic) . . . 29

17 Practice exam (Fall 2026) and Sai's graded midterm (last quarter) 30
17.1 Practice exam with official solutions . . . 30
17.2 Sai's midterm (graded, 20/26) . . . 31

FIGURE ATLAS (F1–F32) 32

INDEX – page : grid cell 36

Index 36

PART I DISTILLED NOTES

1 ML foundations, bias–variance, KNN

1.1 What ML is; the basic model

Machine learning vs. statistical learning (S1.7–18)

machine learning = applying learning algorithms and statistical methods to real-world data, i.e. how a machine can use data to *learn*; "all about making predictions or classifications" (S1.10).

- **ML**: subfield of AI in CS; algorithmic perspective; focus on AI and *prediction accuracy*.
- **Statistical learning**: subfield of statistics; statistical perspective; focus on interpretability and inference (S1.14).
- Classical programming: rules + data → answers. ML: data + answers → *rules* (S1.17).
- Rough workflow (S1.25): **Goal** → **Data** → **Learn** → **Evaluate**, looping back ("a different method/model?").

The model everything is built on (S1.29–33)

$$Y = f(X_1, \ldots, X_p) + \varepsilon$$

- Y = **response** (outcome, output, target, dependent variable).
- $X_1, \ldots, X_p$ = **predictors** (inputs, features, covariates, independent variables); p = # predictors, n = # observations, $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})$.
- f = fixed, unknown (non-random) function; ε = random error ("noise"), mean zero.
- **Goal: learn** f from data; the estimate is $\hat{f}$, prediction $\hat{y} = \hat{f}(\mathbf{x})$.

- Ex. (S1.29): Y=wage, X_1=age, X_2=education, X_3=year.

Supervised vs. unsupervised (S1.35, 44–47; HW1 Q1)

supervised learning	unsupervised learning
Response Y *known* for training data; the observed Y is the "supervisor" (answer key)	No response: only $X_1, \ldots, X_p$; "you never see the answer key"
Learn f to predict/explain Y	Find structure/groups/patterns in X
Linear & logistic regression, KNN, decision trees, random forests, SVM, neural nets	PCA, k-means clustering, hierarchical clustering, neural nets
Evaluated against the true Y	No direct "right answer" to score against

[Figure: S1.47: same 2-D scatter twice; supervised = points colored by known class, unsupervised = all black (response not known).]

Regression vs. classification; metrics (S1.42–43; HW1 Q2–3)

Decided by the type of the **response** Y (not the predictors):

- regression: Y **quantitative** / numeric (price, blood pressure, speed). Metrics: MSE, RMSE, MAE, R-squared.
- classification: Y **qualitative** / categorical (survived/died, spam/not spam). Metrics: accuracy, error rate, precision, area under ROC curve (AUC).

Supervised-learning goals (S1.41): **prediction** (accurately predict future response), **estimation** (how predictors affect response), **model selection** (find the "best" model), **inference** (assess quality of predictions/estimation).

Regression or classification?

- *Logistic regression* is a **classification** method (outputs class probabilities), despite the name.
- A 0/1 or 1/2/3 coded label is still categorical ⇒ classification (convert with `factor()`).
- KNN does **both**; in tidymodels you must say which with `set_mode()` (Lab 2).

Uses of models: descriptive / inferential / predictive (S1.50; HW1 Q4)

- **Descriptive**: choose a model to best *visually emphasize a trend* in data (e.g. a smooth line on a scatterplot).
- **Inferential**: "what features are significant?"; aim is to *test theories*; (possibly) causal claims; state relationship between outcome & predictor(s) (p-values, CIs, hypothesis tests).
- **Predictive**: "what combo of features fits best?"; aim is to predict Y with minimum **reducible error**; *not* focused on hypothesis tests.

HW1 Q6 (voter data). (i) "Given a voter's profile, how likely are they to vote for the candidate?" ⇒ **predictive** (we want an accurate probability for each voter; why does not matter). (ii) "How would support change if the voter had personal contact with the candidate?" ⇒ **inferential** (effect of one specific predictor on the outcome; a relationship/causal-type question).

Parametric vs. non-parametric (S1.49; HW1 Q5)

Mechanistic/ parametric model	Empirically-driven/ non-parametric model
Assume a form for f (e.g. $\beta_0 + \beta_1 X_1 + \cdots$), estimate the parameters	No/few assumptions about f; shape learned from data
Won't match the true unknown f ⇒ **higher bias**	Much more flexible by default ⇒ **low bias, higher variance**
Adding parameters = more flexibility; too many = overfitting	Requires a **larger #** of observations; can also overfit
Easier to interpret (few meaningful parameters)	Harder to interpret ("black box")
e.g. linear regression, logistic regression, LDA	e.g. KNN

Similar: both are predictive models that estimate f from data and both can overfit. **Which is easier to understand?** Mechanistic. **Bias–variance link:** mechanistic = rigid form → high bias / low variance; empirically-driven = flexible → low bias / high variance (needs more n to tame variance).

Flexibility vs. interpretability (S1.53–56)

Different methods give different $\hat{f}$, each with its own flexibility (complexity). Ranking, interpretability high → low / flexibility low → high: subset selection, lasso → least squares → GAMs, trees → bagging, boosting → SVMs (S1.55) (F1, p. 32).

- Why ever pick a *less* flexible method? Better **interpretability** and, sometimes, **smaller test error**.
- **No single method is best for all data sets**; choosing one depends on *evaluating* its performance (S1.56).

"The most flexible model is always best" — wrong (PE Q3)

(1) If the goal is interpretation/inference, flexible models are harder to interpret. (2) A very flexible model fits patterns in the training data that do not exist in the population ⇒ overfitting: low training error, high test error.

1.2 Training vs. test error; overfitting

Training MSE vs. test MSE (S1.57–60)

Training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$; test set $\{(\tilde{\mathbf{x}}_1, \tilde{y}_1), \ldots, (\tilde{\mathbf{x}}_m, \tilde{y}_m)\}$ *not seen or used to train* $\hat{f}$.

$$\text{Training MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2 \qquad \text{Test MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(\tilde{y}_i - \hat{f}(\tilde{\mathbf{x}}_i)\right)^2$$

$\hat{f}$ is obtained on the **training** set. We want the $\hat{f}$ with the **smallest test MSE**. **Lowest training MSE ≠ lowest test MSE.**

How the errors move with flexibility (S1.61–66)

(F3, p. 32)

- Higher flexibility → matches training points better, more wiggly curve → **training MSE always decreases** (monotone, can reach 0).
- Higher flexibility ↛ better fit to the true f: **test MSE is U-shaped** — falls, hits a minimum, then rises.
- Test MSE can never fall below $\mathrm{Var}(\varepsilon)$ (dashed line in the plots); training MSE can.
- **overfitting** = small training MSE, large test MSE (the method chases noise). **underfitting** = too inflexible: both errors high (misses real structure).
- Speed-vs-yams toy (S1.36–40): a green squiggle interpolates the 5 training points (training error 0), but on the 4 red test points the plain straight line has the smaller total error $\sum_{i=6}^{9}|\tilde{y}_i - \hat{f}(\tilde{x}_i)|$.

Estimating test MSE (S1.68, 94): training MSE is easy; test data may be unavailable, or using it repeatedly turns it into "another training set". Fix: **cross-validation** — estimates test MSE using only training data.

Practice Q3 — which model? (PE Q3)

Model	Train acc.	Test acc.	Reading
A	71%	70%	too simple (both low)
B	86%	82%	**prefer: highest test acc.**
C	100%	68%	**overfitting** (huge gap)

Official solution. Prefer B (highest *testing* accuracy). C is overfitting; B shows a little overfitting too, but much less.

Lab 2 (diamonds, predict price) — RMSE (Lab 2)

Model	train	test	Reading
linear regression	1126	1146	no gap, but large error: high bias
KNN, $k = 5$ (default)	386	**711**	best test RMSE
KNN, $k = 2$	152	851	lower train, higher test: overfit

Smaller k = more flexible $\Rightarrow$ lower training error but a bigger train–test gap. The linear model imposes a form on f (even predicts negative prices); KNN learns f from the data.

Training-error traps (PE Q4)

- "A model with lower training error is always preferable" — **false**: training error is a poor estimate of test error; as training error $\to 0$, test error almost always increases.
- "A predictor with zero Pearson correlation with Y can't improve prediction" — **false**: zero correlation = no *linear* relation; nonlinear/empirically-driven models (e.g. KNN) can still use it.
- "Two highly correlated predictors necessarily interact" — **false**: an interaction term means the X_1–Y relationship changes with X_2; unrelated to correlation.

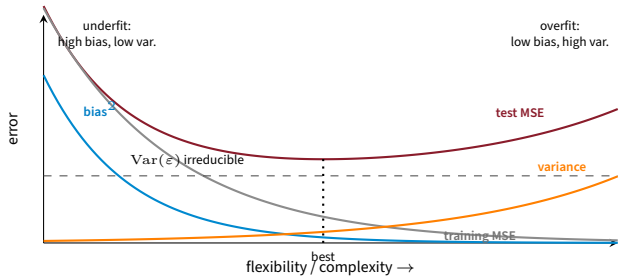
1.3 Bias–variance decomposition

Bias–variance decomposition (course form) (S1.70–79)

$$\underbrace{E\left[\left(\tilde{y}_0 - \hat{f}(\tilde{\mathbf{x}}_0)\right)^2\right]}_{\text{expected test MSE}} = \underbrace{\operatorname{Var}\left(\hat{f}(\tilde{\mathbf{x}}_0)\right)}_{\text{variance on test data}} + \underbrace{\left(\operatorname{Bias}\left(\hat{f}(\tilde{\mathbf{x}}_0)\right)\right)^2}_{\text{bias on test data}} + \underbrace{\operatorname{Var}(\varepsilon)}_{\text{irreducible error}}$$

- $\hat{f}$ learned from a training set; $(\tilde{\mathbf{x}}_0, \tilde{y}_0)$ = a test observation, *independent* of the training set, randomly sampled from $Y = f(X) + \varepsilon$ (f non-random, ε zero-mean noise).
- $\operatorname{Bias}(\hat{f}(\tilde{\mathbf{x}}_0)) = E[\hat{f}(\tilde{\mathbf{x}}_0)] - f(\tilde{\mathbf{x}}_0)$: how far the *average* fit is from the truth (error from wrong/too-simple assumptions).
- $\operatorname{Var}(\hat{f}(\mathbf{x}_0)) = E\left[(\hat{f}(\mathbf{x}_0) - E[\hat{f}(\mathbf{x}_0)])^2\right]$: how much $\hat{f}(\mathbf{x}_0)$ jumps around across different training sets.
- E averages over repeated training sets (each gives a different $\hat{f}$) and over ε.

The tradeoff picture (S1.80–84, 101) (F6, p. 32)



Sketch: test MSE = bias^2 + variance + $\operatorname{Var}(\varepsilon)$; minimum (dotted) where the falling bias^2 and rising variance balance.

- **Simple model $\to$ high bias + low variance; flexible model $\to$ low bias + high variance** (S1.81). More complexity: variance ↑, bias ↓.
- Want *simultaneously* low variance and low (squared) bias — the challenge is finding such a method.
- Where the minimum sits depends on the true f (S1.82–84) (F5, p. 32): true f moderately curved $\to$ intermediate flexibility best; f nearly linear $\to$ linear regression already near-optimal (bias≈0, extra flexibility only adds variance); f very nonlinear $\to$ bias falls steeply, flexible methods win.

Dartboards: bias = off-center, variance = spread (S1.71–76, 80)

Bull's-eye = $\tilde{y}_0$; each dart = $\hat{f}(\tilde{\mathbf{x}}_0)$ from a different training set.

1. tight on center	low bias	low var.
2. scattered around center	low bias	**high var.**
3. tight but off-center	**high bias**	low var.
4. scattered and off-center	**high bias**	**high var.**

Reducible vs. irreducible error (S1.77, 85–87)

Variance + bias^2 = **reducible error** (depends on the method). $\operatorname{Var}(\varepsilon)$ = **irreducible error**: noise no model can remove (unmeasured variables, randomness). The "best" we can do: $\hat{f}(\mathbf{x}_0) = E[Y \mid X = \mathbf{x}_0]$ (the true regression function). Then variance $= 0$ and bias $= 0$, so expected test MSE is **minimized at** $\operatorname{Var}(\varepsilon)$. But $E[Y \mid X]$ is **unknown in practice** (the population joint distribution of X and Y is unknown) $\Rightarrow$ we must estimate it (e.g. KNN).

Bias–variance traps

- No method can get expected test MSE below $\operatorname{Var}(\varepsilon)$; training MSE *can* go below it (that is overfitting).
- Bias and variance describe the *method* over many possible training sets, not one fitted curve.
- The "bias" term enters **squared**; all three terms are ≥ 0.
- "Increasing flexibility lowers bias" is (generally) true; "…lowers test error" is **not** necessarily true.

1.4 Classification: error rate and the Bayes classifier

Training / test error rate (S1.88–93)

$$\text{Training error rate} = \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i), \qquad I(y_i \neq \hat{y}_i) = \begin{cases} 1 & y_i \neq \hat{y}_i \\ 0 & y_i = \hat{y}_i \end{cases}$$

= fraction of *incorrect* classifications in the training set; $\hat{y}_i$ = class label $\hat{f}$ predicts. **Test error rate** $= \frac{1}{m}\sum_{i=1}^{m} I(\tilde{y}_i \neq \hat{\tilde{y}}_i)$ on the test set (slide writes $\frac{1}{n}$). accuracy $= 1 -$ error rate.

Worked example (S1.90)

y_i	dog	cat	bird	dog	dog
$\hat{y}_i$	cat	cat	dog	dog	dog
$I(y_i \neq \hat{y}_i)$	1	0	1	0	0

Training error rate $= 2/5 = 0.40$; training accuracy $= 1 - 2/5 = 0.60$. Same story as regression (S1.93): higher flexibility $\to$ lower training error rate, but $\not\to$ lower test error rate; **overfitting = large test error rate + small training error rate**.

Bayes classifier and Bayes error rate (S1.96–99)

(F7, p. 32) **Bayes classifier**: assign each observation to the *most likely class given its predictor values*:

$$\hat{y}_0 = \operatorname{argmax}_j \operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$$

Ex.: $P(\text{dog} \mid \mathbf{x}_0) = 0.5$, $P(\text{cat} \mid \mathbf{x}_0) = 0.3$, $P(\text{bird} \mid \mathbf{x}_0) = 0.2 \Rightarrow \hat{y}_0 =$ dog. Two classes: $\hat{y}_0 = 1$ if $\operatorname{Prob}(Y = 1 \mid X = \mathbf{x}_0) > 0.5$, else 2. The Bayes decision boundary is where $\operatorname{Prob}(Y = 1 \mid X = \mathbf{x}_0) = 0.5$ (purple dashed curve on the slides).

$$\text{Bayes error rate} = 1 - E\left[\max_j \operatorname{Prob}(Y = j \mid X)\right]$$

= lowest possible test error rate; > 0 whenever classes overlap (even the Bayes classifier misclassifies points on the "wrong" side, red squares on S1.99). It is the classification analogue of $\operatorname{Var}(\varepsilon)$.

Regression vs. classification summary (S1.100)

	Regression	Classification
Y	quantitative	qualitative
error	training/test MSE	training/test error rate
best	$E[Y \mid X = \mathbf{x}_0]$	$\operatorname{argmax}_j \operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$
floor	$\operatorname{Var}(\varepsilon)$	$1 - E[\max_j \operatorname{Prob}(Y = j \mid X)]$

Bayes classifier

It is a theoretical gold standard: it needs the true conditional distribution of Y given X, which we never know. Real classifiers (KNN, logistic regression, LDA…) *estimate* $\operatorname{Prob}(Y = j \mid X)$ and try to approach it.

1.5 K-nearest neighbors (KNN)

KNN algorithm (S1.103–109)

We never know $E[Y \mid X]$ or $\operatorname{Prob}(Y = j \mid X)$ — **estimate** them locally: look at the K training points closest to $\mathbf{x}_0$.

$$\mathcal{N}_0 = \{i : \mathbf{x}_i \text{ is among the } K \text{ nearest neighbors of } \mathbf{x}_0\}$$

Regression (average): $\hat{f}_K(\mathbf{x}_0) = \dfrac{1}{K} \sum_{i \in \mathcal{N}_0} y_i$

Classification (majority vote): $\hat{y}_K = \operatorname{argmax}_j \left\{ \dfrac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j) \right\}$

(the class proportions among the neighbors estimate $\operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$). Steps: (1) compute the distance from $\mathbf{x}_0$ to every training point; (2) keep the K closest; (3) average their y / take a vote.

Distances (S1.104)

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$ (straight line; usual default).
- Manhattan distance: $d(\mathbf{x}, \mathbf{y}) = |x_1 - y_1| + |x_2 - y_2|$ (grid / city-block).
- Minkowski distance: $d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$, $p \geq 1$; $p = 1$ Manhattan, $p = 2$ Euclidean.

Worked KNN examples

3-NN regression (S1.107): house prices of the 3 nearest (by sq. ft) are 50, 55, 51 $\Rightarrow \hat{f} = (50 + 55 + 51)/3 = 52$.

Vote with $k = 1, 3, 5, 7$ (PE Q5). Neighbors ordered closest → farthest:

neighbor	1	2	3	4	5	6	7		
class		A	B	B	A	A	B	B	
$k = 1$:	A							(A 1–0)	⇒ **A**
$k = 3$:	A	B	B					(A1, B2)	⇒ **B**
$k = 5$:	A	B	B	A	A			(A3, B2)	⇒ **A**
$k = 7$:	A	B	B	A	A	B	B	(A3, B4)	⇒ **B**

Predictions can flip as k changes. Even k can tie (here $k = 2$: 1–1, $k = 4$: 2–2) — use odd k for two classes.

ISLR picture (S1.110): $K = 3$, the circle around $\times$ holds 2 blue + 1 orange ⇒ predict blue; repeating for every point of the plane draws the KNN decision boundary.

Choosing k: the KNN bias–variance tradeoff (S1.113–115) (F8, p. 32)

For fixed $(\mathbf{x}_0, y_0)$, with $Y = f(X) + \varepsilon$:

$$E\left[(y_0 - \hat{f}(\mathbf{x}_0))^2\right] = \underbrace{\frac{\operatorname{Var}(\varepsilon)}{k}}_{\text{variance}} + \underbrace{\left[f(\mathbf{x}_0) - \frac{1}{k} \sum_{i \in \mathcal{N}_0} f(\mathbf{x}_i)\right]^2}_{\text{bias}^2} + \operatorname{Var}(\varepsilon)$$

	variance	bias^2	$\operatorname{Var}(\varepsilon)$
$k \uparrow$	decrease	increase	beyond our control
$k \downarrow$	increase	decrease	beyond our control

- **Small k = flexible**: jagged, island-filled decision boundary; $k = 1$ gives **training error 0** (each point is its own nearest neighbor) → low bias, high variance, overfits.
- **Large k = smooth / rigid**: boundary nearly linear at $K = 100$ → high bias, low variance, underfits. $k = n$: every prediction = overall mean / majority class.
- Flexibility grows with $1/K$ (S1.115) (F9, p. 33): training error falls steadily to 0 at $1/K = 1$; test error is U-shaped (min near $K \approx 10$), never below the dashed Bayes error rate (≈ 0.13).
- k is a hyperparameter (tuning parameter): not estimated by fitting; choose it by test/cross-validation error, never by training error.

k direction traps (PE Q1; Sai Q1)

- KNN great on training, worse on new data (overfitting) ⇒ **increase** k. Decreasing k makes it worse; up-weighting large-variance variables makes it worse; adding an interaction term is pointless (KNN is non-parametric).
- **Increasing k makes KNN *less* flexible** (adding polynomial/interaction terms to a regression makes it *more* flexible).

Ties (S1.111–112)

Distance ties: several points equidistant at the k-th spot (which ones count?). The slide's table keeps all tied points: L → A B; M (3 equidistant) → A A B; N (1 obvious + 2 tied) → A A B B. *Vote ties* (A B, A A B B): common fixes are an odd k, choosing at random, or going with the nearest neighbor's class / distance-weighted votes.

Why standardize predictors (PE Q6)

KNN is **distance-based**: a predictor with a big spread dominates the distance. Ex.: age (years) vs. income ($): A=(25, 50,000), B=(65, 51,000), C=(26, 58,000). Unscaled, $d(A, B) = \sqrt{40^2 + 1000^2} \approx 1001$ but $d(A, C) = \sqrt{1^2 + 8000^2} \approx 8000$, so B (40 years older!) is A's "nearest" neighbor — age barely matters. Standardizing *only* age does not fix it: income must be standardized too. Fix: `step_normalize()` (center to mean 0, scale to SD 1) on **all** numeric predictors, so each contributes on the same scale.

Curse of dimensionality; KNN profile (S1.116)

KNN works well for **small** p (about $p \leq 4$) and **large** n. It **fails when p is very large**: there are very few (almost no) training points "near" $\mathbf{x}_0$, so the "nearest" neighbors are actually far away ⇒ **curse of dimensionality**. (Classic illustration: to capture 10% of uniform data in a p-dim cube, a neighborhood needs side $0.1^{1/p}$: 0.10 for $p = 1$, 0.32 for $p = 2$, 0.79 for $p = 10$.) More sophisticated non-parametric models exist (e.g. kernel estimators).

- **Non-parametric**: no assumed form for f, no coefficients, no normality / equal-variance assumptions; can draw **nonlinear decision boundaries** because classification is decided *locally* by neighbors (Sai Q2).
- Costs: not interpretable (no coefficients to read); needs the training data at prediction time; sensitive to scale and to irrelevant/noisy predictors; categorical predictors must be dummy-coded (`step_dummy()`) so distances can be computed.

KNN in tidymodels (Lab 2; HW3 Q8)

Model: `nearest_neighbor()` with argument `neighbors` (k; default 5 — Lab 2's fit reports "Best k: 5"), engine `set_engine("kknn")` (the only engine; needs `library(kknn)`), and `set_mode()` *required* ("regression" or "classification", because KNN can do both).

```
knn_rec <- recipe(y ~ ., data = train) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

knn_mod <- nearest_neighbor(neighbors = 7) |>
  set_engine("kknn") |>
  set_mode("classification")    # or "regression"

knn_wf <- workflow() |>
  add_model(knn_mod) |>
  add_recipe(knn_rec)
knn_fit <- fit(knn_wf, data = train)

predict(knn_fit, new_data = test)        # .pred_class / .pred
predict(knn_fit, new_data = test, type = "prob")
augment(knn_fit, new_data = test) |>
```

```
  accuracy(truth = y, estimate = .pred_class)
```

Regression version (Lab 2): `set_mode("regression")`, then `predict()` + `bind_cols()` with the true `price` and a `metric_set(rmse, rsq, mae)`. `extract_fit_parsnip()` on a KNN fit shows little (no coefficients). (Engine detail: kknn weights neighbors by distance by default, "Best kernel: optimal".)

Spot-the-bug: KNN code (PE Q7)

- `y ~ .` silently includes an identifier variable (e.g. `application_id`) and post-outcome info (e.g. `default_date` = target leakage) ⇒ drop them.
- No `step_normalize()` ⇒ distances dominated by large-scale variables.
- Recipe defined but never used: `fit(defaulted ~ ., data = loans)` fits the raw data, so the `step_dummy()` never happens ⇒ put recipe + model in a `workflow()`.
- `one_hot = TRUE` is **fine** for KNN (no intercept, so no dummy-variable collinearity problem).
- Fitting/evaluating on all data (no train/test split) gives an optimistic error estimate.

2 Linear models, feature engineering, EDA

Slide map: S2.3–9 EDA • S2.10–39 linear regression • S2.40–46 categorical predictors • S2.47–56 interactions & polynomials • S2.57–75 diagnostics • S2.76–78 LR vs KNN, bias–variance.

2.1 Exploratory data analysis (EDA)

What EDA is for

Goals (S2.4): data cleaning & quality check; descriptive statistics; data visualization; ideas for feature engineering; discovering relationships among variables. **Iterative loop**: (1) generate questions about your data; (2) search for answers by visualizing, transforming and modeling; (3) use what you learn to refine questions / generate new ones. Not a fixed set of rules — "a state of curiosity about data" (HW1).

- Two questions that are always useful (S2.8): what **variation** occurs *within* my variables? what **covariation** occurs *between* my variables? You do not need to look at everything; ask questions, answer them, present the 3–5 most interesting results.
- Assess the data (S2.5, S2.9): # rows/columns; variable types (numeric, categorical, ordered); missing data (% by variable, why, impute?); unusual values — extremely large/small, impossible, unusual within context. Real data is messy.

Which plot for which question

Question	Plot	ggplot2 layer
Shape of one numeric var (skew, modes, outliers)	histogram, density plot	`geom_histogram()` (`bins=60`), `geom_density()`
Counts of one categorical var; class imbalance	bar plot	`geom_bar()`, `count()`
Numeric vs. categorical	boxplot (+ jitter)	`geom_boxplot()`, `geom_jitter(alpha=.3)`
Two numeric vars	scatterplot	`geom_point()`, `geom_smooth()`
All numeric pairs	correlation matrix	`cor()` + `corrplot()`

Boxplot: box = Q1–Q3 (IQR), line = median, whiskers to the most extreme points within 1.5 IQR, dots beyond = potential outliers. **skewness**: *right (positive) skew* = mass at low values, long right tail, mean > median: `diamonds price` (most < $\$10{,}000$) (Lab 1); abalone age (mean 11.4 > median 10.5, max 30.5) (HW2 Q1). Left skew = mirror image. Bimodal: `mpg` hwy (peaks ≈17 and ≈26 mpg: trucks/SUVs vs. small cars) (HW1 Ex1). Skewed outcome → stratify the split, consider a log transformation.

```
diamonds %>% select(where(is.numeric)) %>% cor() %>%
  corrplot(type = "lower", diag = FALSE, method = "color")
ggplot(diamonds, aes(x = price)) + geom_histogram(bins = 60)
ggplot(diamonds, aes(x = price, y = reorder(cut, price),
                     fill = color)) + geom_boxplot()
```

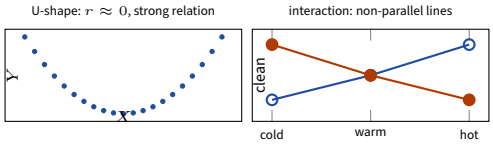
```
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot() + geom_jitter(alpha = 0.3)   # HW1 Ex4
ggplot(mpg, aes(y = fct_rev(fct_infreq(manufacturer)))) +
  geom_bar()        # HW1 Ex3: flipped bar plot, ordered by height
```

EDA findings to remember. `mpg` (HW1): hwy vs `cty` strong positive linear; median `hwy` falls with `cyl` (29/24/17 mpg for 4/6/8 cyl); `cty`–`hwy` and `displ`–`cyl` strongly +, each pair negatively correlated with the other. `diamonds` (Lab 1, S2.74) (**F17**, p. 34): $r(\text{price, carat}) = 0.92$; x, y, z correlate 0.86–0.88 with price and 0.95–0.98 with carat (collinearity). Worst color J costs *more* because J diamonds tend to be larger (carat) — a third variable drives both: confounding.

Pearson correlation

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}} \in [-1, 1]$$

Measures the strength and direction of **linear** association *only*. $|r| = 1$ iff points lie exactly on a line; r is unitless, symmetric, unchanged by centering/scaling. In simple linear regression $R^2 = r^2$ (ISLR). `cor()` returns `NA` if any `NA` is present.



Left (PE Q8): $Y = X^2$, Pearson $r \approx 0$. Right (S2.50–51) (**F11**, p. 33): cleanliness vs. water temperature, one line per detergent; the best detergent depends on temperature = interaction. Parallel lines = main effects only, no interaction.

Correlation myths (PE Q4, Q8, Q9)

- **Zero correlation ≠ useless predictor.** U-shaped scatterplot with $r = 0.03$ (PE Q8): correct answer = "little *linear* correlation despite a strong nonlinear relationship". NOT "unrelated", NOT "plot is wrong", NOT "X won't be useful" — X helps via an X^2 term, KNN or other flexible models (PE Q4a).
- **Correlated predictors ≠ interaction.** $r(X_1, X_2) = 0.91$ is evidence of collinearity, not of an interaction term. Interaction = the relationship between X_1 and Y *depends on the value of* X_2, e.g. positive for one level of X_2, negative for the other (PE Q4b, Q9). Skewness also says nothing about interaction.
- Correlation ≠ causation (confounding, e.g. color J vs. carat). Lower training error is not always better (PE Q4c).

2.2 Missing data & imputation

Handling missing values

Look first (S2.5, Lab 2): `vis_miss()` (`naniar`) on `airquality`: 4.8% of all cells missing; Ozone 24.18%, `Solar.R` 4.58%, others 0%. Fine to inspect missingness before splitting. **Why missing?** (S2.6) (dog eats homework H; H^* = observed homework; A = student attribute): **MCAR** (missing completely at random) — dog eats *any* homework, missingness unrelated to anything; **MAR** (missing at random) — dog eats *some students'* homework, missingness depends on an observed attribute A; **MNAR** (missing not at random) — dog eats *bad* homework, missingness depends on the missing value itself. Patterns: univariate, monotone, general, planned. **Options** (Lab 2): (1) drop the variable (not if it is the outcome); (2) drop rows, `drop_na()`: `airquality` 153 → 111 rows (large loss); (3) imputation = predict the missing values. **Imputation steps** (S2.7): numeric: `step_impute_bag()`, `step_impute_knn()`, `step_impute_linear()`; categorical: bag, knn, `step_impute_mode()`; also `step_impute_mean()`/`median`. mean imputation (and median/-mode) **reduces variance** (all imputed values identical); KNN, bagging or linear imputation tend to perform better. **Rules**: (a) handle *all* missingness somehow (different methods per variable is fine), (b) consider *why* it is missing, (c) *report* what you did. **Gotchas**: `mean(x)` is `NA` if any `NA` (use `na.rm = TRUE`); `geom_histogram()` silently drops `NA` rows ("Removed 37 rows containing non-finite values").

```
air_recipe <- recipe(Ozone ~ ., data = airquality_train) %>%
  step_impute_linear(Ozone,
       impute_with = imp_vars(Wind, Temp, Month, Day)) %>%
  step_impute_linear(Solar.R,
       impute_with = imp_vars(Wind, Temp, Month, Day))
prep(air_recipe) %>% bake(new_data = airquality_train) %>%
  vis_miss()                    # now 0% missing
```

2.3 Data splitting

Train/test split with rsample

```
set.seed(3435)                  # same random split every run
diamonds_split <- initial_split(diamonds, prop = 0.80,
                                strata = price)
diamonds_train <- training(diamonds_split)
diamonds_test  <- testing(diamonds_split)
dim(diamonds_train); dim(diamonds_test)     # verify sizes
```

- `prop` = fraction that goes to the **training** set (default $3/4$). (HW2 Q2): `prop = 0.85, strata = age` on 4177 abalone → 3548 train / 629 test.
- `set.seed()`: splitting is random; without a random seed every re-run gives a different split and different results (reproducibility). Any number — just remember it.
- `strata` = **stratified sampling**: sample *within* strata of the outcome so train and test have approximately the same outcome distribution (same class proportions; a numeric outcome is binned into quartiles, `breaks = 4`). **Why**: with a skewed outcome or class imbalance (e.g. `titanic`: most passengers did not survive) a plain random split can leave too few rare cases / extreme values in one set → unrepresentative fit or test estimate (Lab 3, HW3, ROC reading).
- **Never touch the test set** until the very end: EDA-driven choices, recipe estimation (`prep()`: means, SDs, imputation models), fitting and tuning all use `training` only; the test set is used once to estimate test error. Splitting *before* EDA is "arguably better practice" (Lab 1).

2.4 Linear regression: model and least squares

The model and its assumptions

(S2.12–14) The best possible prediction under squared error is $\hat{f}(\mathbf{x}_0) = E[Y \mid X = \mathbf{x}_0]$ — unknown because the joint distribution of (X, Y) is unknown. So **make an assumption**:

$$Y = \beta_0 + X_1\beta_1 + \cdots + X_p\beta_p + \varepsilon, \quad E[\varepsilon] = 0,\ \varepsilon \perp (X_1, \ldots, X_p)$$

$\Rightarrow E[Y \mid X = \mathbf{x}_0] = \beta_0 + x_{01}\beta_1 + \cdots + x_{0p}\beta_p$: the conditional expectation is **linear in the parameters**. simple linear regression: $p = 1$, data $\{(x_1, y_1), \ldots, (x_n, y_n)\}$; multiple linear regression: $p > 1$, $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})$. ε = catch-all for what the simple model misses, mean zero, independent of everything else (S2.24). A parametric model: estimate $p + 1$ numbers instead of a whole function. **Assumptions** (S2.48, S2.58–61) (**F14**, p. 33): (1) **linear** and **additive** in the predictors; (2) errors $\varepsilon_1, \ldots, \varepsilon_n$ **uncorrelated**; (3) **constant variance** $\mathrm{Var}(\varepsilon_i) = \sigma^2$; plus no influential outliers / severe collinearity (diagnostics below). **Picture** (S2.15–18): y = confirmed COVID cases, x = county population. intercept β_0 = mean y when $x = 0$; slope β_1 = change in mean y per one-unit increase in x (+10,000 people $\Rightarrow +10000\beta_1$ cases). Different (β_0, β_1) give different lines ($y = 8000 + 0.057x$ vs. $5000 + 0.061x$); residual for Riverside $= y_{RS} - \beta_0 - \beta_1 x_{RS}$ = observed − predicted.

Least squares estimates (S2.20–29)

Choose $\hat{\beta}$'s to minimize the sum of squared residuals

$$\mathrm{SSR} = \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \cdots - \hat{\beta}_p x_{ip}\right)^2 \quad \text{(ISLR: RSS)}$$

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

(the line passes through $(\bar{x}, \bar{y})$). We cannot get $\hat{\beta} = \beta$ exactly: the data are random, so $\hat{\beta}_0, \hat{\beta}_1$ are random variables (new sample → new line, S2.25). **Unbiased**: $E(\hat{\beta}_1) = \beta_1$, $E(\hat{\beta}_0) = \beta_0$, i.e. Bias $= 0$ (S2.27).**Variance** ($\sigma^2 = \mathrm{Var}(\varepsilon)$):

$$\mathrm{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_i (x_i - \bar{x})^2}, \quad \mathrm{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_i (x_i - \bar{x})^2}\right]$$

σ^2 unknown → $\hat{\sigma}^2 = \mathrm{SSR}/(n-2)$ and plug in ($\widehat{\mathrm{SE}}$). More spread in x and larger $n \Rightarrow$ smaller SE. Approx. 95% confidence interval: $\hat{\beta}_j \pm 2\,\mathrm{SE}(\hat{\beta}_j)$ (ISLR).

Model accuracy and tests (S2.30, S2.34–39)

$\mathrm{SST} = \sum_i (y_i - \bar{y})^2$, $\mathrm{SSR} = \sum_i (y_i - \hat{y}_i)^2$. **RSE** (residual standard error) $= \sqrt{\mathrm{SSR}/(n-p-1)}$ ($n-2$ in SLR): absolute lack of fit, in units of Y. $R^2 = 1 - \mathrm{SSR}/\mathrm{SST} \in [0, 1]$ = proportion of the variability in Y explained by X. R^2 **always increases** as predictors are added; RSE and R^2 favor flexible models, which may overfit. **adjusted R-squared** $= 1 - \dfrac{(1-R^2)(n-1)}{n-p-1}$ (penalizes extra predictors). **F-statistic** for $H_0 : \beta_1 = \cdots = \beta_p = 0$ vs. H_a: at least one $\beta_j \neq 0$:

$$F = \frac{(\mathrm{SST} - \mathrm{SSR})/p}{\mathrm{SSR}/(n-p-1)}$$

Under H_0 both numerator and denominator $\approx \sigma^2 \Rightarrow F \approx 1$; **large** F **favors** H_a. **t-statistic** $t = \hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$ tests $H_0 : \beta_j = 0$ *given the other predictors*; small p-value (< 0.05) $\Rightarrow$ reject.

Three questions (S2.34): Q1 is at least one predictor useful? (F-test). Q2 which subset matters? (selection). Q3 how well does the model fit? (RSE, R^2). **Variable selection** (S2.36–37): best subset selection fits all 2^p subsets ($\approx$1 billion for $p = 40$) and picks the "best" by Mallow's C_p, AIC, BIC or adjusted R^2; forward selection starts from the null model and adds the most significant variable one at a time (greedy) until a stopping rule; backward selection starts from the full model and removes the least significant one at a time (impossible when $p > n$); mixed selection combines both.

Reading lm / tidy() output

(Lab 2): `diamonds`, `price ~ .`, `step_dummy()` on all nominal predictors, `linear_reg()` + `set_engine("lm")`.

```
lm_fit %>% extract_fit_parsnip() %>% tidy()
  term         estimate std.error statistic   p.value
1 (Intercept)  5317.       496.      10.7    8.54e- 27
2 carat       11282.        54.3    208.     0
3 depth         -58.4        6.25    -9.33   1.07e- 20
5 x            -970.        49.0    -19.8    5.16e- 87
6 y               9.08      19.9      0.456  6.49e-  1
8 cut_1         596.        25.0     23.8    1.22e-124
# ... (18 more rows)
```

- `estimate` = $\hat{\beta}_j$ (coefficient); `std.error` = $\mathrm{SE}(\hat{\beta}_j)$; `statistic` = t = estimate/std.error ($11282/54.3 \approx 208$); `p.value` for $H_0 : \beta_j = 0$.
- **carat**: holding all other predictors fixed, +1 carat is *associated* with +\$11,282 in mean price; $p \approx 0 \Rightarrow$ significant. (Association, not causation.)
- **y**: $p = 0.649 \Rightarrow$ no evidence that y adds information *once carat, x, z are in the model* — not "y is unrelated to price" ($r = 0.87$). **x** is *negative* although $r(x, \text{price}) = +0.88$: coefficients are partial effects and collinearity can flip signs.
- **Intercept** = predicted price when every numeric predictor (and contrast) is 0 — an extrapolation (carat $= 0$), not meaningful here.
- `cut_1`… are **not** 0/1 dummies: `cut`, `color`, `clarity` are *ordered* factors (<ord>), so `step_dummy()` makes polynomial contrasts (`cut_1` linear, `cut_2` quadratic, …; values like 0.632, -0.267).
- Fit: train RMSE 1126 / $R^2$0.920, test RMSE 1146 / R^2 0.918 (no overfitting), but the predicted-vs-actual plot is curved and some predicted prices are negative $\Rightarrow$ a linear f is a poor approximation here.

2.5 Categorical predictors: dummy variables

Dummy (indicator) coding

(S2.41–44) categorical predictors (qualitative / factor variables) can enter a regression whose Y is quantitative via dummy variables. **2 levels** (Credit: balance vs. gender): $x_i = 1$ if female, 0 if male $\Rightarrow y_i = \beta_0 + \beta_1 + \varepsilon_i$ (female), $\beta_0 + \varepsilon_i$ (male). β_0 = mean balance of males (baseline); β_1 = average difference female $-$ male. **Coding is not unique**: 1=male/0=female flips the sign of β_1 and β_0 becomes the female mean; ± 1 coding (1 male, -1 female) gives β_0 = average of the two group means, β_1 = half the male$-$female difference. Same fitted values, different interpretation. m **levels** $\Rightarrow m-1$ **dummies.** Ethnicity {African American, Asian, Caucasian}: $x_{i1} = 1$ if Asian, $x_{i2} = 1$ if Caucasian; African American = **baseline** (no dummy).

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i = \begin{cases} \beta_0 + \beta_1 + \varepsilon_i & \text{Asian} \\ \beta_0 + \beta_2 + \varepsilon_i & \text{Caucasian} \\ \beta_0 + \varepsilon_i & \text{Afr. American} \end{cases}$$

β_0 = mean of the baseline; β_j = difference between level j and the baseline. Coding and choice of baseline are arbitrary but change the interpretation of $\beta_0, \beta_1, \beta_2$. (PE Q11): only categorical variables (`Major`, `Dorm`) need dummy encoding; numeric ones (`HS_GPA`, `CreditsCompleted`) do not.

step_dummy() vs one-hot (S2.45–46)

```
recipe(age ~ type, data = abalone_train) %>%   # type: F, I, M
  step_dummy(type)             # -> type_I, type_M  (F =
    ↪ reference)
recipe(age ~ type, data = abalone_train) %>%
  step_dummy(type, one_hot = TRUE)  # -> type_F, type_I, type_M
    ↪
```

New columns are named `var_level`. Default = $m-1$ **dummy coding**; the reference level is the *first factor level* (alphabetical for characters): a row with `type_I = 0, type_M = 0` is F. Change it with `step_relevel(type, ref_level = "M")` or `fct_relevel()`. `one_hot = TRUE` = one-hot encoding: one 0/1 column per level. Usually `step_dummy()(all_nominal_predictors())` (Lab 2).

one_hot = TRUE + intercept = rank deficient (PE Q13)

`salary ~ experience + department` (Sales, IT, HR) with `step_dummy(department, one_hot = TRUE)` and `linear_reg()`: R reports the model matrix is **rank-deficient**, a coefficient cannot be identified (`NA`, "prediction from a rank-deficient fit may be misleading"). (a) Cause: one-hot makes 3 indicators, one per level. (b) For every row they sum to 1 = the intercept column (a column of ones) $\Rightarrow$ perfect linear dependency, design matrix not full rank ("dummy-variable trap"). (c) Fix: `one_hot = FALSE` (the default). (d) Then **2** indicators are created ($m-1$). One-hot is **fine for KNN** (no intercept, no linear dependency problem) (PE Q7).

2.6 Interactions and nonlinear terms

(**F12**, p. 33)

Relaxing additivity and linearity

(S2.48, S2.54) The plain model is **additive** — the effect of changing X_j on Y does not depend on the other predictors (relax with **interaction terms**) — and **linear** — a one-unit change in X_j changes Y by the same amount whatever the value of X_j (relax with **polynomial regression** / transformations).

Interaction (S2.49)

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \varepsilon = \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \varepsilon$$

Without X_1X_2: $+1$ in $X_1 \Rightarrow +\beta_1$ in Y (holding X_2 fixed). With it: $+1$ in $X_1 \Rightarrow +(\beta_1 + \beta_3 X_2)$ — the effect of X_1 is no longer constant; changing X_2 changes the impact of X_1 (and symmetrically the effect of X_2 is $\beta_2 + \beta_3 X_1$). **Definition**: an interaction term is needed when the relationship between X_1 and Y differs depending on the value of X_2. It says nothing about whether X_1 and X_2 are correlated (PE Q4b, Q9).

Worked examples

Advertising (S2.52): sales $= \beta_0 + \beta_1$TV $+ \beta_2$radio $+ \beta_3$TV $\times$ radio $+ \varepsilon$.

	Coef.	SE	t	p
Intercept	6.7502	0.248	27.23	$<$0.0001
TV	0.0191	0.002	12.70	$<$0.0001
radio	0.0289	0.009	3.24	0.0014
TV$\times$radio	0.0011	0.000	20.73	$<$0.0001

$+\$1{,}000$ TV $\Rightarrow (\hat\beta_1 + \hat\beta_3$ radio$) \times 1000 = 19.1 + 1.1 \times$ radio increase in sales. **Practice answer**: $+\$1{,}000$ radio $\Rightarrow (\hat\beta_2 + \hat\beta_3$ TV$) \times 1000 = 28.9 + 1.1 \times$TV. **Dummy** $\times$ **numeric** (Credit, S2.53): balance $= \beta_0 + \beta_1$income$+ \beta_2$student $+ \beta_3$ income $\times$ student $\Rightarrow$ students: $(\beta_0 + \beta_2) + (\beta_1 + \beta_3)$ income; non-students: $\beta_0 + \beta_1$ income. Without the interaction the two lines are **parallel** (same slope, different intercepts); with it the slopes differ. **Interaction plots** (S2.50–51): one line per level of A over levels of B. Parallel lines = no interaction (vertical gap = A main effect, common slope = B main effect); non-parallel or crossing lines = interaction. **hierarchical principle** (ISLR): if X_1X_2 is in the model, keep X_1 and X_2 too, even if their p-values are large.

```
step_dummy(all_nominal_predictors()) %>%     # FIRST make dummies
step_interact(terms = ~ starts_with("type_"):shucked_weight +
                longest_shell:diameter + shucked_weight:shell_
    ↪ weight)
# -> type_I_x_shucked_weight, type_M_x_shucked_weight, ...   (HW2
    ↪ Q3)
```

`step_interact()` needs numeric columns: after `step_dummy()` the factor `type` no longer exists, so refer to its dummies with `starts_with()("type_")`. Slope of `shucked_weight` for infants $= \hat\beta_{\text{shucked}} + \hat\beta_{\text{type_I_x_shucked}}$; for females (reference) $= \hat\beta_{\text{shucked}}$.

Polynomials and transformations (S2.55–56)

polynomial regression of degree d (F13, p. 33): $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_d X^d + \varepsilon$. Y is no longer linear in X, **but it is still a linear model**: let $Z_k = X^k$, then $Y = \beta_0 + \beta_1 X + \beta_2 Z_2 + \cdots + \beta_d Z_d + \varepsilon$ is linear in $X, Z_2, \ldots, Z_d$ (linear in the parameters). Figure: linear vs. degree 2 vs. degree 5 fits to a curved cloud — degree 2 follows the curve, degree 5 wiggles at the edges (more flexible $\rightarrow$ overfitting risk). transformations: `step_poly()(x, degree = 2)`, `step_log()`; a log transformation tames right skew and funnel-shaped residuals. Transform the *outcome* with `mutate()` (e.g. `log(price)`) before splitting rather than in the recipe.

2.7 Regression diagnostics

Six potential problems (S2.58–65)

Problem	Detect $\rightarrow$ consequence / fix
1. Non-linearity	**residual plot**: $e_i = y_i - \hat y_i$ vs. fitted $\hat y_i$. Correct model $\Rightarrow$ *no discernible pattern*; a U-shape means missed curvature $\rightarrow$ add $X^2, \log X, \sqrt{X}$.
2. Correlated errors	e.g. time series; estimated SEs inaccurate $\rightarrow$ wrong CIs and tests.
3. Non-constant variance	heteroscedasticity: funnel shape in the residual plot (F15, p. 33) $\rightarrow$ inaccurate CIs and tests; fix by transforming Y ($\log Y, \sqrt{Y}$): S2.61 funnel vanishes for $\log(Y)$.
4. outliers	y_i far from $\hat y_i$ (e.g. recording error). Use studentized residuals (residual / its SE; $\lvert\cdot\rvert > 3$ suspicious, ISLR).
5. high leverage points	unusual x_i values. Leverage (SLR) $h_i = \frac{1}{n} + \frac{(x_i - \bar x)^2}{\sum_k (x_k - \bar x)^2}$; $h_i \in [1/n, 1]$, average $(p+1)/n$.
6. collinearity	correlation matrix, VIF (below).

influential observation: removing it changes the fit a lot. It may or may not be an outlier or have high leverage, but tends to have at least one of the two. **Example** (S2.66–72) (F16, p. 33) (fit without the red point: $y = 1.73 + 5.12x$): outlier only $(4, 40)$: $y = 2.96 + 5.04x$ — barely changes, **not** influential; high leverage on the trend $(14, 68)$: $y = 2.47 + 4.93x$ — **not** influential; outlier *and* high leverage $(13, 15)$: $y = 8.51 + 3.32x$ — **influential**. Status is model-specific (S2.65). What to do: 1. check for a data-entry error; 2. exclude the points; 3. try re-including them if the model changes.

Collinearity (S2.73–75)

collinearity: two or more predictors closely related. If they rise and fall together it is hard to separate how each is associated with $Y \Rightarrow$ **variances (SEs) of the estimates increase** $\rightarrow$ large p-values and unstable or flipped signs (diamonds x: -970). **Detect**: (1) correlation matrix of $X_1, \ldots, X_p$ (diamonds: carat–x 0.98, carat–y, z 0.95, x–y 0.97); (2) variance inflation factor (VIF) $\text{VIF}_j = 1/(1 - R^2_{X_j|X_{-j}})$, where R^2 comes from regressing X_j on the other predictors (ISLR: > 5–10 is a problem; also catches multicollinearity among 3+ variables that pairwise correlations miss). **structural collinearity** = created by us from other predictors (X and X^2, interaction terms, a ratio plus its components); **data-based** = present in the observed data (carat vs. x, y, z). **Handle** ("it depends"): 1. drop variables; 2. combine variables; 3. feature extraction (PCA); 4. regularization (ridge, lasso). Collinearity mainly hurts interpretation and inference; predictions can still be fine.

2.8 Feature engineering with recipes

What may go in the model

feature engineering = creating or reformatting predictors (features) so the model can use them: dummy coding, interactions, polynomial/log transformations, normalization, imputation, new combined variables (ratios).

- **target leakage**: a predictor that carries information about the outcome that would *not be available at prediction time*, or is derived from the outcome $\Rightarrow$ unrealistically good performance that collapses in practice. `default_date` when predicting `defaulted` (PE Q7); `GraduationDate`, `FinalGPA` when predicting `Graduated4Years` from end-of-first-year data (PE Q10); `rings` for `age = rings + 1.5` (age is a deterministic function of rings, so the model just learns the formula and the other predictors become useless) (HW2 Q3).
- **Rule**: use only information you would have at the moment the prediction is made.
- **Not leakage** (PE Q12): `CompletionRate` = CreditsCompleted/CreditsAttempted (first-year information, known at prediction time); including it together with both components is redundant (structural collinearity).
- **identifier variables** (`StudentID`, `application_id`): arbitrary labels with no meaning $\Rightarrow$ exclude (spurious patterns; they distort KNN distances). The outcome itself is obviously excluded.
- `recipe(y ~ ., data)`: "`.`" = *every* other column, including IDs and leaky variables $\rightarrow$ drop them first (`select(-id)`, `step_rm()`) or list the predictors explicitly.

Centering, scaling, normalization

centering $x - \bar{x}$ (`step_center()`); scaling x/s (`step_scale()`); both = normalization / standardization $z = (x - \bar{x}_{\text{train}})/s_{\text{train}}$ (`step_normalize()`): mean 0, SD 1.

- **Required for KNN** (distance-based): an unscaled large-variance variable (income) dominates the distance (PE Q6, Q7).
- **Linear regression**: fitted values, R^2, RMSE unchanged; coefficients become "per 1 SD". With all predictors centered the **intercept** $= \bar{y}_{\text{train}}$ (HW2: `(Intercept)` 11.5 $\approx$ mean age).
- `prep()` estimates means/SDs (and imputation models, factor levels) from the **training** data; `bake()` / `predict()` apply those same training values to new data. `bake(new_data = NULL)` returns the processed training set.
- **Step order** (recipes docs): impute $\rightarrow$ transform (log, poly) $\rightarrow$ `step_dummy` $\rightarrow$ `step_interact` $\rightarrow$ normalize.

HW2 pipeline: abalone age, LR + KNN

```
abalone <- read_csv("data/abalone.csv") %>%
  mutate(age = rings + 1.5, type = factor(type))
abalone_recipe <- recipe(age ~ type + longest_shell + diameter +
    height + whole_weight + shucked_weight + viscera_weight +
    shell_weight, data = abalone_train) %>%        # NO rings
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("type_"):shucked_weight +
      longest_shell:diameter + shucked_weight:shell_weight) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())   # = step_normalize(all_
    ↪ predictors())
lm_model  <- linear_reg() %>% set_engine("lm")
knn_model <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>% set_mode("regression")
lm_wflow <- workflow() %>% add_model(lm_model) %>%
  add_recipe(abalone_recipe)
lm_fit <- fit(lm_wflow, data = abalone_train)  # preps recipe
predict(lm_fit, new_data = new_abalone)     # one row -> .pred
abalone_metrics <- metric_set(rsq, rmse, mae)
augment(lm_fit, new_data = abalone_test) %>%   # adds .pred
  abalone_metrics(truth = age, estimate = .pred)
```

Q7: the new observation must be a tibble with *every* predictor the recipe uses and the same types (`type = factor("F", levels = levels(abalone$type))`); the workflow applies the recipe automatically $\rightarrow$ `.pred` = 21.8. **Q8 results (test)**: LR R^2 0.585, RMSE 1.95, MAE 1.44; KNN ($k = 7$) R^2 0.483, RMSE 2.21, MAE 1.62. Training: KNN R^2 0.811 vs. LR 0.552 $\Rightarrow$ KNN overfits (each training point is its own nearest neighbor); LR generalizes better. Interpretation: $R^2 = 0.585$ $\Rightarrow$ 58.5% of the variability in test-set age is explained by the model. (`yardstick` `rsq()` = squared correlation of truth and `.pred`; `rsq_trad()` $= 1 - \text{SSR}/\text{SST}$.)

2.9 Linear regression vs. KNN regression

Parametric vs non-parametric (S2.76)

Linear regression (parametric model)	**KNN regression** (non-parametric model)
assume $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$	$f(X)$ can have any functional form
no need to tune	tuning parameter K (`neighbors`)
performs well when the true f is close to linear	much more general-purpose
interpretable; statistical inference (SE, p-values)	not very interpretable
extends to very large p (ridge, lasso)	curse of dimensionality

KNN regression predicts $\hat{f}(x_0)$ = average of the y_i of the K training points closest to x_0 (ISLR); it needs normalized predictors and no interaction terms (non-parametric) (PE Q1). **Lab 2 (diamonds), RMSE train / test**: LR 1126 / 1146; KNN $k = 5$: 386 / 711; KNN $k = 2$: 152 / 851. KNN wins on test because the true f is nonlinear; $k = 2$ has the biggest train–test gap (most flexible $\rightarrow$ overfits more).

Bias–variance of least squares (S2.77)

$E\left[(y_0 - \hat{f}(\mathbf{x}_0))^2\right] = \text{Var}(\hat{f}(\mathbf{x}_0)) + [\text{Bias}(\hat{f}(\mathbf{x}_0))]^2 + \text{Var}(\varepsilon)$.If the true f is linear and $\hat{f} = \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \cdots + \hat{\beta}_p x_{0p}$ (least squares): **Property 1**: unbiased, $\text{Bias}(\hat{f}(\mathbf{x}_0)) = E[\hat{f}(\mathbf{x}_0)] - f(\mathbf{x}_0) = 0$. **Property 2** (Gauss-Markov theorem): least squares has the **smallest** expected test error among all **unbiased linear** estimators. Modern methods (ridge, lasso) can beat least squares in expected test MSE by accepting a *small bias* for a *much smaller variance*.

3 Classification models: logistic regression, LDA, QDA, naive Bayes

3.1 Classification set-up; why not linear regression?

Classification in general

- Predicting a qualitative response for an observation = classification: assign the observation to a class. Usually we first predict the *probability* of each class (a quantitative number) and then classify (S3.4).
- Classifiers in this course: KNN, logistic regression, discriminant analysis (LDA, QDA), naive Bayes.
- Running example: `Default` data (ISLR): $Y =$ `default` (Yes/No); $X_1 =$ annual `income`, $X_2 =$ monthly credit-card `balance`, $X_3 =$ `student`. Defaulters have much higher balances; income barely differs (S3.8) (F21, p. 34).

Why not linear regression for a categorical Y?

- **Binary** $Y \in \{0,1\}$: linear regression models $\Pr(Y=1 \mid X) = \beta_0+\beta_1X_1+\cdots+\beta_pX_p$, an unbounded line $\Rightarrow$ fitted "probabilities" can be <0 (Default data: **negative probability** for balance $\lesssim 500$) or >1. Logistic regression's S-curve stays in $[0,1]$ (S3.5) (**F20**, p. 34).
- >2 **classes**: coding $Y=1$ (stroke), 2 (drug overdose), 3 (seizure) imposes an **ordering** and equal spacing (stroke→overdose gap = overdose→seizure gap); a different coding (1 stroke, 2 seizure, 3 overdose) gives a different model; {dog, cat, fish} has no order at all (S3.6).
- Course focuses on **binary** classification (multi-class logistic regression exists) (S3.7).

3.2 The logistic regression model

Logistic regression: three equivalent forms

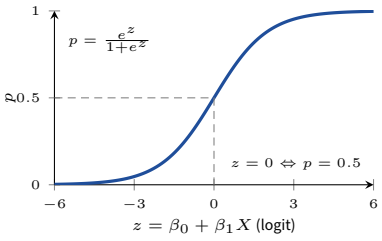
With $p(X) = \Pr(Y=1 \mid X)$ (S3.10–11, S3.24):

$$p(X) = \frac{e^{\beta_0+\beta_1X}}{1+e^{\beta_0+\beta_1X}} \in [0,1] \qquad \text{logistic function}$$

$$\frac{p(X)}{1-p(X)} = e^{\beta_0+\beta_1X} \in [0,\infty) \qquad \text{odds}$$

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0+\beta_1X \in (-\infty,\infty) \qquad \text{log odds} = \text{logit}$$

The logit is linear in X (not p!). Multiple predictors: replace $\beta_0+\beta_1X$ by $\beta_0+\beta_1X_1+\cdots+\beta_pX_p$ everywhere (S3.24). $\log = \ln$ (natural log). Equivalent: $p = \frac{1}{1+e^{-z}}$ with $z = \beta_0+\beta_1X$ (the sigmoid).
Odds $= p/(1-p)$, *not* p: rolling a 3 on a fair die has $p = 1/6$, odds $= \frac{1/6}{5/6} = 1/5$ (S3.10). $p = 0.8 \Rightarrow$ odds $= 4$ ("4 to 1").



p	odds	log odds
0	0	$-\infty$
0.269	0.368	-1
0.5	1	0
0.731	2.72	1
0.88	7.33	2
0.95	19	3
1	∞	$+\infty$

p on $[0,1] \mapsto$ log odds on $(-\infty,\infty)$ (S3.16–21)

Interpreting logistic coefficients

(S3.12–13, S3.25)

- $\beta_1 =$ the **additive** change in the logit (log odds) for a 1-unit increase in X.
- e^{β_1} = the **multiplicative** change in the odds: odds$(X+1)$/odds(X) $= e^{\beta_1}$ (the odds ratio). A c-unit increase multiplies the odds by $e^{c\beta_1}$ (HW3 Q12: $+2$ units $\Rightarrow \times e^{2\beta_1}$). % change in odds $= 100(e^{\beta_1}-1)\%$.
- β_1 is **NOT** the change in $\Pr(Y=1 \mid X)$ per unit of X. Only the **sign** carries over: $\beta_1>0 \Rightarrow X \nearrow \Rightarrow p \nearrow$; $\beta_1<0 \Rightarrow X \nearrow \Rightarrow p \searrow$; $\beta_1=0 \Rightarrow$ no association.
- The rate of change of p per unit X **depends on the current value of** X — not linear (calculus: $dp/dX = \beta_1p(1-p)$, steepest, $= \beta_1/4$, at $p=0.5$).
- Multiple logistic regression: $\beta_j =$ change in logit per unit X_j **conditional on (holding fixed) the other predictors** (S3.25).
- $\beta_0 =$ log odds when all X's $= 0$. Regression on observational data shows **association, not causation**.

PE Q16: coefficient 0.04 is NOT "+4% probability"

Logistic regression for canceling a subscription; coefficient of `MonthlyFee` $= 0.04$.
Wrong: "A \$1 increase in the monthly fee *causes* a customer's probability of canceling to increase by 4%." Problems: (1) it treats the probability as linear in the fee — the relationship is linear in the log odds, and the change in p depends on the current fee; (2) "causes" — only **association** can be claimed.
Right: "A \$1 increase in the monthly fee is associated with a 0.04 increase in the log odds of canceling" (equivalently, odds $\times e^{0.04} \approx 1.041$, about $+4.1\%$ in the *odds*), or "a higher fee is associated with a higher probability of canceling" (sign is positive) (PE Q16).

HW3 Q11–Q12 (231): logit is the inverse of the logistic

$p = \frac{e^z}{1+e^z} \iff z = \ln\frac{p}{1-p}$ (logit and logistic are inverses) (HW3 Q11).
$z = \beta_0+\beta_1x_1$: increasing x_1 by 2 multiplies the odds by $e^{2\beta_1}$. If $\beta_1<0$: $x_1 \to \infty \Rightarrow p \to 0$; $x_1 \to -\infty \Rightarrow p \to 1$ (HW3 Q12).

3.3 Estimating the coefficients: maximum likelihood

Fitting with glm()

- Coefficients are estimated by maximum likelihood (MLE), *not* least squares (S3.26): choose $\hat\beta$'s so $\hat p(x_i)$ is near 1 when $y_i=1$ and near 0 when $y_i=0$, i.e. maximize $\ell(\beta_0,\beta_1) = \prod_{i:y_i=1}p(x_i)\prod_{i':y_{i'}=0}(1-p(x_{i'}))$ (ISLR 4.3.2).
- Base R: `glm()` with `family = binomial` ("generalized linear model"):

```
glm.fit <- glm(default ~ student + balance + income,
               data = Default, family = binomial)
summary(glm.fit)
```

```
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***
studentYes  -6.468e-01  2.363e-01  -2.738  0.00619 **
balance      5.737e-03  2.319e-04  24.738  < 2e-16 ***
income       3.033e-06  8.203e-06   0.370  0.71152
    Null deviance: 2920.6  on 9999  degrees of freedom
Residual deviance: 1571.5  on 9996  degrees of freedom
AIC: 1579.5
```

- `z value` = Estimate/Std. Error (a z-statistic, not t); small p-value $\Rightarrow$ reject H_0: $\beta_j=0$ given the other predictors. Here income is not significant ($p=0.71$) once balance and student are in the model.
- `studentYes` is the dummy variable for student = Yes (reference level = No).
- Units: `income` coefficient is 3.033×10^{-6} per \$1; the slide tables show 0.0030 because there income is in \$1000s.
- deviance: residual deviance < null deviance $\Rightarrow$ predictors help; lower AIC is better.

Perfect separation makes logistic regression unstable

If $Y=0$ for $X \le 50$ and $Y=1$ for $X>50$ (**perfectly separable**), `glm(y~x, family = "binomial")` warns "`algorithm did not converge`" and "`fitted probabilities numerically 0 or 1 occurred`": the MLE $\hat\beta_1 \to \infty$, so the coefficient estimates are unstable. **LDA does not suffer from this problem** (S3.45).

3.4 Multiple logistic regression and confounding

The student paradox on the Default data

(S3.27–29)

	`default~student`	`~balance+income+student`
Intercept	-3.5041	-10.8690
balance	—	0.0057 $(p<.0001)$
income (\$1000s)	—	0.0030 $(p=.7115)$
student[Yes]	$\mathbf{0.4049}$ $(p=.0004)$	$\mathbf{-0.6468}$ $(p=.0062)$

- **Contradicting signs!** Simple model: a student is **riskier** than a non-student *if no other information* about the student is available. Multiple model: a student is **less risky** than a non-student *with the same credit-card balance* (and income).
- Why: **student and balance are correlated** — students are more likely to carry large balances, which are associated with high default rates. Balance confounding the student effect.
- Lesson: pay extra attention when regressing on a **single predictor** while other relevant predictors exist (confounding; Simpson's paradox: S3.30 shows two groups each with a positive y–x trend, but the pooled data trend is negative) (S3.29–31) (**F22**, p. 34).

3.5 Predicted probabilities and the threshold

Worked prediction (Default data)

$$\widehat{\Pr}(\text{default} = \text{Yes} \mid \text{balance} = 1500, \text{income} = \$40\text{k}, \text{student} = \text{No})$$

$$= \frac{e^{-10.869+0.00574(1500)+0.0030(40)-0.6468(0)}}{1+e^{-10.869+0.00574(1500)+0.0030(40)-0.6468(0)}}$$

$$= \frac{e^{-2.14}}{1+e^{-2.14}} = 0.105$$

Same person but student = Yes: $z = -2.14 - 0.6468 = -2.79 \Rightarrow \hat{p} = 0.058$ (S3.32). Recipe: (1) $z = \hat{\beta}_0 + \sum \hat{\beta}_j x_j$; (2) $\hat{p} = e^z/(1+e^z)$. The slide writes "0.0030×40000", but with income in \$1000s you must plug in 40 (40000 would give $\hat{p} \approx 1$); fully rounded $\hat{\beta}$'s give ≈ 0.10.

From predicted probability to predicted class

(S3.33)

- Classify by **thresholding** the predicted probability: predict default = Yes if $\hat{p} > 0.5$ (the usual threshold; $\hat{p} > 0.5 \iff \text{logit} > 0$).
- A **conservative** credit company might predict default = Yes whenever $\hat{p} > 0.1 \Rightarrow$ more individuals flagged: more true positives *and* more false positives.
- Raising the threshold $\Rightarrow$ fewer positive classifications (PE Q15): $\hat{p} = .31, .48, .63, .81$ for borrowers A–D; threshold $0.5 \Rightarrow$ C, D; threshold $0.7 \Rightarrow$ only D.
- Changing the threshold changes the confusion matrix, sensitivity, specificity — but **not** the ROC curve or AUC (PE Q18). Details: see the model-evaluation section.

PE Q14: what a predicted probability means

Pneumonia model gives one patient $\hat{p} = 0.78$. Correct: **(c)** "the fitted model assigns this patient a probability of 0.78 of belonging to the pneumonia class." NOT "the model is 78% accurate for this patient", NOT "definitely has pneumonia", NOT "78% of the symptoms indicate pneumonia" (PE Q14).

Logistic regression in tidymodels (Lab 3)

(Lab 3) Data: `Smarket` (outcome `Direction`: Down/Up; predictors Lag1–Lag5, `Volume`) and `OJ` (outcome `Purchase`: CH/MM).

```
library(tidymodels); library(discrim); library(klaR)
set.seed(3435)
oj_split <- initial_split(OJ, prop = 0.70, strata = Purchase)
oj_train <- training(oj_split); oj_test <- testing(oj_split)
oj_recipe <- recipe(Purchase ~ PriceCH + PriceMM + DiscCH +
    ↪ DiscMM +
                      LoyalCH + PctDiscMM + PctDiscCH, data = oj
    ↪ _train)
log_reg <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")
ojlog_wkflow <- workflow() %>%
  add_model(log_reg) %>% add_recipe(oj_recipe)
ojlog_fit <- fit(ojlog_wkflow, oj_train)
ojlog_fit %>% tidy()
predict(ojlog_fit, new_data = oj_train, type = "prob")
augment(ojlog_fit, new_data = oj_train) %>%
  conf_mat(truth = Purchase, estimate = .pred_class)
```

```
   term         estimate std.error statistic  p.value
1 (Intercept)     4.47      2.21       2.02   4.35e- 2
6 LoyalCH        -6.07      0.450    -13.5    1.48e-41
...
# predict(type = "prob"):    .pred_CH .pred_MM
                          1     0.606    0.394
                          2     0.898    0.102
```

- `predict()` default (`type = "class"`) returns `.pred_class` (threshold 0.5); `type = "prob"` returns one column per level, `.pred_<level>` (rows sum to 1). `augment()` attaches *both* to the data. `tidy()` gives the coefficient table.
- Smarket: training probabilities all ≈ 0.4–$0.6 \Rightarrow$ the model cannot discriminate Up/Down (training accuracy 0.525); OJ: probabilities near 0/1, training accuracy 0.833.

Which level is "Y = 1"? (factor levels)

`glm` (and `logistic_reg()` with `"glm"`) models the probability of the **second** factor level; the first is the baseline. OJ levels are (CH, MM), so the coefficients are log odds of buying **MM**: `LoyalCH` $= -6.07$ means more Citrus-Hill loyalty $\Rightarrow$ lower odds of MM. **yardstick** metrics, in contrast, treat the **first** level as the event (`event_level = "first"`). HW3 therefore reorders `survived` so "Yes" is first (`factor(survived, levels = c("Yes","No"))`) and uses `.pred_Yes` in `roc_auc()` — but then the glm coefficients describe the log odds of *No* (flip the sign to talk about survival).

3.6 Discriminant analysis: Bayes' theorem and the idea

Why bother with LDA?

(S3.45–46) **(1)** logistic regression coefficient estimates can be **unstable** (e.g. well/perfectly separated classes); **(2)** n is **small** and the distribution of X is close to **normal** (in each class); **(3)** multi-class classification ($K > 2$) problems.

Bayes' theorem for classification

The Bayes classifier $\operatorname{argmax}_{k=1,\dots,K} \Pr(Y = k \mid X_1, \dots, X_p)$ is the "best" we can do (S3.48). Logistic regression models $\Pr(Y = 1 \mid X)$ directly; LDA instead uses Bayes' theorem and makes assumptions on $X_1, \dots, X_p \mid Y$ (S3.48–49):

- prior probability $\pi_k = \Pr(Y = k)$; density function of X in class k: $f_k(x) = \Pr(X = x \mid Y = k)$.

$$\underbrace{\Pr(Y = k \mid X = x)}_{\text{posterior probability}} = \frac{\pi_k f_k(x)}{\sum_{j=1}^{K} \pi_j f_j(x)}$$

The denominator is the same for every $k \Rightarrow$ classify to the k with the largest $\pi_k f_k(x)$.

Discriminant analysis in one sentence: assume f_k is the **normal** density, i.e. $(X_1, \dots, X_p) \mid Y$ is normally distributed (S3.68).

Example (mine): $\pi_1 = 0.3$, $\pi_2 = 0.7$, $f_1(x_0) = 0.4$, $f_2(x_0) = 0.1 \Rightarrow \Pr(Y = 1 \mid x_0) = \frac{0.12}{0.12+0.07} = 0.63 \Rightarrow$ class 1, despite the smaller prior.

3.7 LDA with one predictor ($p = 1$)

LDA, p = 1: model, discriminant function, estimates

Assume $X \mid Y = k \sim N(\mu_k, \sigma^2)$ — **different mean, same variance** (S3.69):

$$f_k(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(-\frac{1}{2\sigma^2}(x - \mu_k)^2\Big).$$

Assign x to the class whose discriminant function

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \quad \text{is largest}$$

(S3.70–76 print the middle term as μ_k^2/σ^2; the correct form, as in ISLR, has $2\sigma^2$.)

$\mu_1, \dots, \mu_K, \pi_1, \dots, \pi_K, \sigma^2$ are unknown $\Rightarrow$ **estimate them** (S3.71, S3.76):

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i, \quad \hat{\sigma}^2 = \frac{1}{n-K} \sum_{k=1}^{K} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2$$

(n_k = # training obs. in class k; $\hat{\sigma}^2$ is the pooled variance.) The LDA classifier plugs these into $\hat{\delta}_k(x) = x\,\hat{\mu}_k/\hat{\sigma}^2 - \hat{\mu}_k^2/(2\hat{\sigma}^2) + \log \hat{\pi}_k$, which is **linear in** x — hence "linear" discriminant analysis.

Decision boundary ($p = 1, K = 2$)

(S3.77) (**F30**, p. 35)

- decision boundary: $\delta_1(x) = \delta_2(x)$. With equal priors $\pi_1 = \pi_2$: $x^* = \frac{\mu_1+\mu_2}{2}$ (midpoint of the means). Unequal priors (ISLR): $x^* = \frac{\mu_1+\mu_2}{2} + \frac{\sigma^2 \log(\pi_2/\pi_1)}{\mu_1-\mu_2}$ — the boundary moves toward the *less* likely class, enlarging the region of the more likely one.
- S3.77: left, densities $N(\mu_1, \sigma^2)$ (green) and $N(\mu_2, \sigma^2)$ (purple) cross at 0 = Bayes decision boundary (dashed); right, histograms of training samples with the **LDA decision boundary** (solid, slightly left of 0) — LDA estimates the Bayes boundary from data.
- Intuition (S3.52–75, gene-transcription/drug example): LDA builds a **new axis** that **maximizes** $\frac{d^2}{s_1^2 + s_2^2}$ — squared distance between class means over the within-class "scatter" — rather than ignoring a

predictor (projecting on Gene X alone loses separation). For 3 classes: $\frac{d_1^2+d_2^2+d_3^2}{s_1^2+s_2^2+s_3^2}$ with d_k = distance from class centroid to the overall centroid; data are projected onto LD1, LD2 (S3.82–85).

3.8 LDA with several predictors ($p > 1$)

LDA, p > 1

Assume $(X_1, \dots, X_p) \mid Y = k \sim N(\mu_k, \Sigma)$ — **different mean vector, same covariance matrix** Σ ($p \times p$, shared by all classes) (S3.78) (multivariate normal distribution):

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\mu_k)^T\Sigma^{-1}(\mathbf{x}-\mu_k)\Big)$$

The Bayes classifier assigns $\mathbf{x}$ to the class with the largest

$$\delta_k(\mathbf{x}) = \mathbf{x}^T\Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log\pi_k.$$

LDA plugs in $\hat{\mu}_k$, $\hat{\Sigma}$ (pooled), $\hat{\pi}_k$: $\hat{\delta}_k(\mathbf{x})$ is **linear in** $\mathbf{x}$ (only the term $\mathbf{x}^T\hat{\Sigma}^{-1}\hat{\mu}_k$ involves $\mathbf{x}$) (S3.79). Posterior probabilities: $\widehat{\Pr}(Y = k \mid \mathbf{x}) = e^{\hat{\delta}_k(\mathbf{x})} / \sum_j e^{\hat{\delta}_j(\mathbf{x})}$.

Boundary (S3.81) (**F31**, p. 35): Bayes $= \{\mathbf{x} : \delta_k(\mathbf{x}) = \delta_j(\mathbf{x})\}$, LDA $= \{\mathbf{x} : \hat{\delta}_k(\mathbf{x}) = \hat{\delta}_j(\mathbf{x})\}$, $k \neq j \Rightarrow$ **linear boundary, i.e. a hyperplane** (a line when $p = 2$; with $K = 3$, three line pieces meeting at a point). In S3.81 the LDA (solid) and Bayes (dashed) boundaries nearly coincide.

3.9 Quadratic discriminant analysis (QDA)

(**F32**, p. 35)

QDA

Key in discriminant analysis: $(X_1, \dots, X_p) \mid Y$ is normal (S3.86). **LDA:** $N(\mu_k, \Sigma)$ — different mean, **same** covariance. **QDA:** $N(\mu_k, \Sigma_k)$ — different mean, **different** covariance (each class its own Σ_k). Assign $\mathbf{x}$ to the class with the largest (S3.87)

$$\begin{aligned}\delta_k(\mathbf{x}) = &-\tfrac{1}{2}\mathbf{x}^T\Sigma_k^{-1}\mathbf{x} + \mathbf{x}^T\Sigma_k^{-1}\mu_k \\ &-\tfrac{1}{2}\mu_k^T\Sigma_k^{-1}\mu_k - \tfrac{1}{2}\log|\Sigma_k| + \log\pi_k\end{aligned}$$

(QDA: hats on μ_k, Σ_k, π_k.) The class-specific term $-\frac{1}{2}\mathbf{x}^T\hat{\Sigma}_k^{-1}\mathbf{x}$ makes it **quadratic in** $\mathbf{x} \Rightarrow$ curved (quadratic) decision boundary.

Parameter count (covariances): LDA one Σ: $p(p+1)/2$; QDA: $K \cdot p(p+1)/2$ (e.g. $p = 50$: 1275 vs. $1275K$) (S3.91).

S3.88: LDA vs QDA boundaries (purple dashed = Bayes, black dotted = LDA, green = QDA)

- $\Sigma_1 = \Sigma_2$: the Bayes boundary is **linear**; LDA is close to it, QDA's curve deviates (extra variance, no bias gain).
- $\Sigma_1 \neq \Sigma_2$: the Bayes boundary is **curved**; QDA tracks it, LDA's straight line cannot (bias).

3.10 Comparing classifiers

KNN vs logistic vs LDA vs QDA vs naive Bayes

Method	Param.?	Boundary	Models / assumes
KNN	no	nonlinear (any shape)	majority vote of the k nearest training points; no assumption on f
Logistic	yes	linear	$\Pr(Y = 1 \mid X)$ directly via the logit (discriminative model); fit by MLE
LDA	yes	linear	$X \mid Y = k \sim N(\mu_k, \Sigma)$, common Σ (generative model)
QDA	yes	quadratic	$X \mid Y = k \sim N(\mu_k, \Sigma_k)$, class-specific Σ_k
Naive Bayes	yes	quadratic, no cross terms	predictors independent within each class

Model flexibility (S3.89): KNN > QDA > logistic regression = LDA (choose via the bias-variance tradeoff); interpretability runs the other way. KNN is non-parametric model; logistic, LDA, QDA are parametric models.

Logistic regression vs LDA

(S3.90)

- **Both produce linear decision boundaries**; the coefficient-estimation procedures differ (logistic: maximum likelihood; LDA: plug in estimated normal means/variance/priors).
- LDA assumes a **normal** distribution of the predictors given the response class.
- Logistic regression learns $\Pr(Y = k \mid X)$ **directly**: discriminative model (discriminative learning). LDA learns $\Pr(Y = k \mid X)$ by modeling $\Pr(Y = k, X) = \pi_k f_k(x)$: generative model (generative learning).
- Rule of thumb: normality roughly holds / small n / well-separated classes / $K > 2 \Rightarrow$ LDA; normality doubtful $\Rightarrow$ logistic regression is more robust.

LDA vs QDA (bias–variance)

(S3.91)

- LDA is a **special case** of QDA (all $\Sigma_k = \Sigma$). QDA is **more flexible**, but has **more parameters** to estimate (the Σ_k's) $\Rightarrow$ lower bias, higher variance.
- Prefer **LDA**: few training observations (small n) relative to p, or class covariances similar — reducing variance is crucial. Prefer **QDA**: large n (variance less of a concern) or the common-covariance assumption is clearly untenable.
- In practice we often do not know which assumptions are reasonable — let the data decide via cross-validation (S3.93).

PE Q20 (LDA vs QDA)

50,000 observations, two predictors, clear evidence that the covariance structures differ substantially across outcome levels. Which has greater potential to capture the true decision boundary? (a) LDA (b) QDA (c) equal (d) neither (both parametric).

Answer. (b) QDA. LDA assumes a common covariance, so it would not do well; QDA estimates more parameters, but $p = 2$ is small and n is huge, so the extra variance is negligible (parametric is fine) (PE Q20).

LDA/QDA traps

- **QDA only helps if the class covariances really differ**; if $\Sigma_1 \approx \Sigma_2$ it just adds variance (more parameters, no bias reduction) and LDA usually wins on test data.
- "Linear" in LDA refers to the **decision boundary** (linear $\hat{\delta}_k$), not to linear regression. LDA and logistic regression share the boundary shape but are estimated differently.
- LDA/QDA assume **normal** predictors within each class — categorical/-dummy predictors violate this (they still run, but the assumption is wrong).
- Higher training error rate is not the criterion: the most flexible model (QDA/KNN) tends to win on *training* data and can lose on testing data (overfitting).

3.11 Naive Bayes (Lab 3)

Naive Bayes

- Same Bayes'-theorem idea, $\Pr(Y = k \mid x) \propto \pi_k f_k(x)$, but instead of a multivariate normal, assume the p predictors are **independent within each class**: $f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \cdots \times f_{kp}(x_p)$.
- With `usekernel = FALSE` each f_{kj} is a **normal (Gaussian)** density (`TRUE` $\Rightarrow$ a kernel density estimate) (Lab 3). Gaussian naive Bayes $\approx$ QDA with *diagonal* Σ_k (no within-class correlations).
- "Naive" = the independence assumption (usually false); it slashes the number of parameters $\Rightarrow$ higher bias, much lower variance; useful when p is large relative to n.

LDA / QDA / naive Bayes in tidymodels (Lab 3)

Needs `library(discrim)` (and `klaR` for naive Bayes). The recipe/workflow is reused; only the model spec changes:

```
lda_mod <- discrim_linear() %>%
  set_mode("classification") %>% set_engine("MASS")
qda_mod <- discrim_quad() %>%
  set_mode("classification") %>% set_engine("MASS")
nb_mod <- naive_Bayes() %>%
  set_mode("classification") %>% set_engine("klaR") %>%
  set_args(usekernel = FALSE)
```

```
smarketqda_wkflow <- workflow() %>% add_model(qda_mod) %>%
  add_recipe(smarket_recipe)
smarketqda_fit <- fit(smarketqda_wkflow, smarket_train)
augment(smarketqda_fit, new_data = smarket_train) %>%
  accuracy(truth = Direction, estimate = .pred_class)
```

Spelling: `naive_Bayes()` (capital B), `discrim_linear()`, `discrim_quad()`; engines `"MASS"` (LDA, QDA), `"klaR"` (naive Bayes).

Lab 3 training accuracies (picked by training accuracy only because CV had not been taught):

Model	Smarket	OJ
Logistic regression	0.525	**0.833**
LDA	0.524	0.828
QDA	**0.548**	0.797
Naive Bayes	0.529	0.778

Test set: Smarket QDA accuracy **0.487** (sensitivity 0.254, specificity 0.703) — worse than a coin flip despite the best training accuracy; OJ logistic accuracy 0.832 (sens. 0.908, spec. 0.714). Lesson: choosing a model by training error is **not** a good idea in general; use cross-validation. Test metrics via `metric_set()(accuracy, sensitivity, specificity)` and `roc_curve()(Direction, .pred_Down)` (see model evaluation).

3.12 HW3: four classifiers on the Titanic data

HW3 pipeline (Q1, Q4–Q10)

(HW3) `titanic` (891 passengers; `survived` Yes $\approx$ 38%; `age` has 177 NAs). Make `survived` and `pclass` factors ("Yes" first), split with `strata = survived` (stratified sampling keeps the ≈38/62 class ratio in both sets — class imbalance).

```
titanic <- read_csv("data/titanic.csv") %>%
  mutate(survived = factor(survived, levels = c("Yes", "No")),
         pclass = factor(pclass))
set.seed(131)
t_split <- initial_split(titanic, prop = 0.7, strata = survived)
t_train <- training(t_split); t_test <- testing(t_split)
t_recipe <- recipe(survived ~ pclass + sex + age + sib_sp +
    ↪ parch + fare,
                   data = t_train) %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp, parch,
    ↪ fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
log_spec <- logistic_reg() %>% set_engine("glm") %>% set_mode("
    ↪ classification")
lda_spec <- discrim_linear() %>% set_engine("MASS") %>% set_mode
    ↪ ("classification")
qda_spec <- discrim_quad() %>% set_engine("MASS") %>% set_mode("
    ↪ classification")
knn_spec <- nearest_neighbor(neighbors = 10) %>%
  set_engine("kknn") %>% set_mode("classification")
log_fit <- workflow() %>% add_model(log_spec) %>% add_recipe(t_
    ↪ recipe) %>%
  fit(t_train)                                  # repeat for lda
    ↪ /qda/knn
predict(log_fit, new_data = t_train, type = "prob") %>%
  bind_cols(t_train %>% select(survived)) %>%
  roc_auc(truth = survived, .pred_Yes)
augment(log_fit, new_data = t_test) %>%
  conf_mat(truth = survived, estimate = .pred_class) %>%
    ↪ autoplot(type = "heatmap")
augment(log_fit, new_data = t_test) %>% roc_curve(survived, .
    ↪ pred_Yes) %>% autoplot()
```

- Order of steps: **impute** → **dummy** → **interact**. After `step_dummy()` `sex` becomes `sex_male`, so the interaction term must use `starts_with()("sex")`. (`imp_vars()` lists the predictors of the linear imputation model; use complete columns.)
- AUC needs **probabilities** (`type = "prob"`, column `.pred_Yes`), not `.pred_class`. Compare **training vs testing** AUC: a big drop (typical for KNN with small k, and for the more flexible QDA) signals overfitting.
- HW3 Q11–12 (231): see the logit-inverse box above.

4 Model evaluation: metrics, thresholds, ROC/AUC

4.1 Train vs. test: where performance is measured

Training vs. test error (S1.57–60, S1.89–91)

- Regression. Training MSE$= \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2$ on the n training points; test MSE $= \frac{1}{m}\sum_{i=1}^{m}\left(\tilde{y}_i - \hat{f}(\tilde{\mathbf{x}}_i)\right)^2$ on m points *not seen or used* to fit. In both, $\hat{f}$ is obtained on the training set.
- Classification. error rate $= \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i)$ = fraction misclassified; accuracy $= 1-$ error rate. Toy example (S1.90): 2 of 5 wrong ⇒ error rate 0.40, accuracy 0.60.
- Goal: the $\hat{f}$ with the smallest *test* MSE / error rate. "Can we just select $\hat{f}$ that minimizes training MSE? NO!" Lowest training MSE $\neq$ lowest test MSE (S1.60).

Why training error is optimistic. $\hat{f}$ was chosen to fit exactly those training points (noise included), so it looks better on them than on new data. As flexibility (complexity) grows, training MSE/error rate keeps falling, but test error is U-shaped (S1.62, S1.101) (**F2**, p. 32): higher complexity → lower training error, but higher complexity $\not\to$ lower test error (see bias-variance tradeoff).

Overfitting / underfitting signatures

- overfitting: training performance $\gg$ test performance (train accuracy $\gg$ test accuracy; train RMSE $\ll$ test RMSE). Extreme case: 100 % training accuracy.
- underfitting (too rigid, high bias): train ≈ test, but both worse than a more flexible model.
- Compare models on **test** performance (or, later, cross-validation). Lab 3 picks a model by *training* accuracy only because CV had not been covered: "not a good idea in general" (Lab 3).
- A model with lower training error is *not* always preferable (PE Q4c).

Worked: PE Q3 — which model?

Model	Train acc.	Test acc.	Verdict
A	71%	70%	no overfit, but weaker
B	86%	**82%**	**choose** (best test)
C	100%	68%	overfitting (gap 32 pts)

Official solution. Prefer B (highest *testing* accuracy). C overfits; B shows some overfitting too, but less. (PE Q3)

Worked: Lab 2 `diamonds` (predict `price`)

Model	train RMSE	test RMSE	test R^2
linear reg. (`lm`)	1126	1146	0.918
KNN, $k = 5$	386	**711**	**0.968**
KNN, $k = 2$	152	851	0.955

Best on *test*: KNN $k = 5$. $k = 2$ has the best training RMSE ($R^2_{\text{train}} = 0.999$) but the largest train–test gap ⇒ overfitting. The linear model has train ≈ test but both high: a linear form is too rigid for f here (Lab 2).

4.2 Regression metrics

Formulas and direction (S1.57, S2.30, Lab 2)

Metric	Definition	Better
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$ (units of Y^2)	lower
RMSE	$\sqrt{\text{MSE}}$ (units of Y)	lower
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$ (units of Y)	lower
R^2	$1 - \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	higher
RSE	$\sqrt{\frac{1}{n-2}\text{SSR}}$ (simple LR)	lower

- RMSE ≈ size of a typical prediction error in Y's units (e.g. \$711 for `price`). Squaring makes RMSE punish large errors more than MAE; always MAE $\leq$ RMSE.
- R-squared $\in [0, 1]$ = proportion of the variability in Y that can be explained using X (S2.30); RSE = an absolute measure of lack of fit.
- `rsq()` in `yardstick` is the squared correlation of `truth` and `.pred`, so it stays in $[0, 1]$ on test data too.

RSE and R^2 favor flexible models (S3.30)

"Both RSE and R^2 favor **flexible** methods, which may **overfit** the data!" Training R^2 never decreases when you add predictors ⇒ judge models on *test* RMSE/R^2.

```
lm_test <- predict(lm_fit, new_data = diamonds_test %>% select(-
    ↪ price))
lm_test <- bind_cols(lm_test, diamonds_test %>% select(price)) # .
    ↪ pred + truth
rmse(lm_test, truth = price, estimate = .pred)
diamond_metrics <- metric_set(rmse, rsq, mae)
diamond_metrics(lm_test, truth = price, estimate = .pred)
```

```
# A tibble: 3 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard    1146.
2 rsq     standard       0.918
3 mae     standard     751.
```

Output is a tibble: the number is in `.estimate`; `.estimator` is `standard` for regression, `binary` for a 2-class outcome. Extract with `acc$.estimate` (Lab 3).

4.3 From probabilities to classes: the threshold

Thresholding. logistic regression gives $\widehat{\Pr}(Y = 1 \mid X)$, e.g. $\widehat{\Pr}(\text{default} = \text{TRUE} \mid \text{balance} = 1500, \text{income} = 40000, \text{student} = \text{FALSE}) = 0.105$ (S3.32). Predict the class by *thresholding the predicted probability*: default = TRUE if $\hat{p} > 0.5$; a "super conservative" credit-card company might use $\hat{p} > 0.1$ (S3.33). A predicted probability of 0.78 means "the model assigns probability 0.78 of belonging to that class", *not* "78% accurate" (PE Q14).

Moving the threshold t (predict positive if $\hat{p} \geq t$)

	Lower t	Raise t
# predicted positives	more	fewer
TP, FP	↑ (or same)	↓ (or same)
FN, TN	↓ (or same)	↑ (or same)
TPR (sensitivity)	↑	↓
FPR ($1-$specificity)	↑	↓
specificity	↓	↑
ROC point moves	up/right	down/left

TPR and FPR move *together*: you trade false negatives for false positives. $t = 0$ (or $-\infty$): everything positive, TPR = FPR = 1. $t = 1$ (or ∞): nothing positive, TPR = FPR = 0.

Worked: PE Q15 — four borrowers, $\hat{p}$ = A .31, B .48, C .63, D .81

(a) $t = 0.50$: predicted default = **C, D**. **(b)** $t = 0.70$: only **D**. **(c)** Raising the threshold ⇒ the number of positive classifications tends to *decrease*. (PE Q15)

4.4 The confusion matrix and its rates

For a binary classifier, use a confusion matrix. Course layout (S3.34) (rows = *true* class, columns = *predicted* class, class 0 first):

		Predicted class $\hat{Y} = 0$	Predicted class $\hat{Y} = 1$	row total
True class	$Y = 0$	TN	FP (Type I)	N = TN+FP
True class	$Y = 1$	FN (Type II)	TP	P = FN+TP

true positive (TP): truly 1, predicted 1. false positive (FP): truly 0, predicted 1 = Type I error. false negative (FN): truly 1, predicted 0 = Type II error. true negative (TN): truly 0, predicted 0. Column totals = number *predicted* 0 / 1; row totals = number *actually* 0 / 1.

Rates (S3.34, S3.36–37)

- Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$; error rate = $\frac{FP+FN}{\text{total}}$ = $1-$ accuracy.
- TPR = $\frac{TP}{TP+FN} = \frac{TP}{P}$ = sensitivity = recall = power = $1-$ Type II error ("probability of detection").
- FPR = $\frac{FP}{FP+TN} = \frac{FP}{N}$ = Type I error = $1-$ specificity ("fall-out", "false alarm").
- specificity = TNR = $\frac{TN}{TN+FP}$ = $1-$ FPR. FNR (miss rate) = $\frac{FN}{TP+FN}$ = $1-$ TPR.
- precision = PPV = $\frac{TP}{TP+FP}$ (of those *predicted* positive, fraction truly positive); negative predictive value (NPV) = $\frac{TN}{TN+FN}$.
- F1 score = $\frac{2}{1/\text{recall} + 1/\text{precision}}$; prevalence = P/total (S3.36).

Memory hook: TPR, FNR divide by the *actual-positive* total; FPR, specificity divide by the *actual-negative* total; precision, NPV divide by a *predicted* total.

Venn picture (S3.38): sensitivity = "how many relevant items are selected?" (sick people correctly identified as having the condition); specificity = how many truly negative elements are identified as negative (healthy people identified as not having the condition).

Multiclass (S3.35): a $K \times K$ table; correct predictions on the diagonal. Slide example (rows = predicted, cols = true; Apple/Orange/Mango): diagonal $7 + 2 + 1 = 10$ of 36 ⇒ accuracy = $10/36 \approx 0.28$.

Worked: PE Q17

	Actually Pos.	Actually Neg.
Predicted Pos.	TP = 72	FP = 18
Predicted Neg.	FN = 28	TN = 182

(Rows are *predicted* here — the opposite of S3.34. Find cells by meaning, not position.)
Accuracy = $\frac{72+182}{300} = \frac{254}{300} = 0.8467$. TPR = $\frac{72}{72+28} = \frac{72}{100} = 0.72$. FPR = $\frac{18}{18+182} = \frac{18}{200} = 0.09$.
Extra: specificity = $182/200 = 0.91$; precision = $72/90 = 0.80$; error rate = $46/300 = 0.153$. (PE Q17)

Worked: reading yardstick output (Lab 3, `Smarket` QDA, test set)

```
          Truth
Prediction Down  Up
      Down   46  58
      Up    135 137
```

`Direction` levels are `Down`, `Up` ⇒ **Down is the event**: TP = 46, FP = 58, FN = 135, TN = 137. Sensitivity = $46/181 = 0.254$, specificity = $137/195 = 0.703$, accuracy = $183/376 = 0.487$ — exactly the printed `metric_set(accuracy, sensitivity, specificity)` output. Training accuracy was 0.548, so the test accuracy (below 0.5) is the honest number (Lab 3).

4.5 Class imbalance: why accuracy can mislead

- With class imbalance, the trivial "always predict the majority class" rule already gets accuracy = majority proportion. Compare accuracy to that baseline.
- **Titanic Model B** (`survived` ~ `sib_sp` only) *never predicts Yes*: test matrix TP = 0, FP = 0, FN = 103, TN = 165 ⇒ accuracy $165/268 \approx 0.61$ "relatively high", yet TPR = 0 and AUC = 0.496 (a coin flip) (ROC).
- Check sensitivity and specificity (or AUC), not just accuracy. Lab 3: `Smarket` is fairly balanced (602 Down / 648 Up); `OJ` is unbalanced (653 CH / 417 MM) (Lab 3).
- Split with stratified sampling on the outcome, `initial_split(data, prop = 0.7, strata = survived)`, so training and test sets keep the same class proportions (ROC, Lab 3). (Up-/down-sampling: a later lab.)

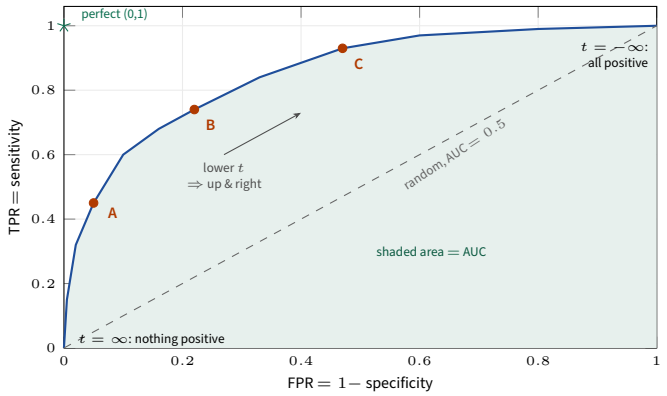
4.6 ROC curve and AUC

ROC curve (S3.39–40, ROC)

(**F23**, p. 34)

- A popular graphic for displaying the **2 types of error** for **all threshold values**: TPR (sensitivity, y-axis) vs. FPR ($1-$specificity, x-axis).
- **Construction:** for every threshold t (in practice each distinct $\hat{p}$, plus $\pm\infty$): classify "Yes" if $\hat{p} \geq t$, build the confusion matrix, compute (FPR, TPR), plot the point; connect the points. Each point on the curve = one threshold.

- Ends: $t = \infty \to (0,0)$; $t = -\infty \to (1,1)$. Lowering t moves up and to the right.
- **Perfect classifier** = point (0, 1), top-left corner. **Random classifier** = the diagonal TPR = FPR. Curves toward the top-left are better (S3.40) (F24, p. 34).
- Useful for *comparing* classifiers (e.g. logistic regression vs. KNN); methods perform differently in different parts of the curve, i.e. at different thresholds, so curves can cross (S3.39–40).



Sketch modelled on PE Q19: A ≈ (0.05, 0.45), B ≈ (0.22, 0.74), C ≈ (0.47, 0.93). A = high threshold, C = low threshold.

AUC (S3.39–40, ROC)

- AUC = area under the ROC curve; measures the *overall* performance of a classifier, **summarized over all possible thresholds**. Higher AUC, the better.
- 1 = perfect; 0.5 = random guessing / coin flip (the diagonal); < 0.5 = worse than random. Slide example: Model A 0.923 > B 0.794 > C 0.588 (S3.40).
- It measures the model's **ability to discriminate between classes**: high when positives get higher $\hat{p}$ than negatives (ROC).
- **Threshold-free:** the curve is built from *all* thresholds, so changing the cutoff used for `.pred_class` (0.5 → 0.3) does *not* change AUC; it only picks a different point on the same curve (PE Q18).
- AUC $= 1$ is suspicious: usually the outcome (or a linear combination of it) slipped in as a predictor (target leakage) (ROC).

Two uses of ROC curves (ROC): (1) evaluating and comparing models (prefer AUC closer to 1; overlay curves); (2) selecting the optimal threshold for a chosen model.

Worked: PE Q18 and Q19

Q18. "I changed my threshold from 0.50 to 0.30, so the ROC AUC will change." **False**: the ROC curve plots TPR and FPR *across* thresholds, so the chosen threshold has no impact on AUC.

Q19. Screening for a very dangerous disease; a false negative is much worse than a false positive. Choose **Point C**: maximize TPR (minimize FNR) regardless of FPR — even though FPR ≈ 50%. In general: FN costly ⇒ *lower* the threshold (point up/right); FP costly ⇒ raise it (down/left). (PE Q18–19)

4.7 Reading "ROC Curves Deep Dive": Titanic Model A vs. B

- Data `titanic` (Kaggle; 891 rows): `survived = factor(survived, levels = c("Yes", "No"))` ⇒ **"Yes" is the first level = the event**; also adds a dummy `coin_flip` column (445 zeros then 446 ones). Split 70/30 with `strata = survived` because the outcome is imbalanced (more died than survived).
- **Recipe A**: `pclass + sex + age + sib_sp + parch + fare`, `step_impute_linear(age)`, `step_dummy()`, `step_interact()`. **Recipe B**: `survived ~ sib_sp` only (intentionally bad). Both: `logistic_reg()` with engine `"glm"`.
- Training accuracy: A 0.807, B 0.616.

Model A (test set: 103 Yes, 165 No).

- Default $t = 0.5$: TP 80, FP 20, FN 23, TN 145 ⇒ TPR $= 80/103 = 0.777$, FPR $= 20/165 = 0.121$: one point near the top-left of the curve.
- $t \approx 0.0013$ (smallest $\hat{p}$): essentially everything predicted Yes: TP 103, FP 163, FN 0, TN 2 ⇒ TPR $= 1$, FPR $= 163/165 \approx 0.99$: the (1, 1) corner.
- `roc_curve()` table has 254 rows (one per threshold); predicted-probability histograms are well separated (No near 0, Yes mostly > 0.5) ⇒ good discrimination.

Model B.

- ROC curve hugs the diagonal; `roc_auc()` $= 0.496$: "just about as bad as …a coin flip".
- At $t = 0.5$ it never predicts Yes ⇒ point (0, 0).
- Only 9 thresholds: `sib_sp` takes few values, so there are few distinct $\hat{p}$; max $\hat{p} = 0.408$. At $t \approx 0.408$: TP 71, FP 118, FN 32, TN 47 ⇒ TPR $= 71/103 = 0.689$, FPR $= 118/165 = 0.715$ (a point just under the diagonal).
- $\hat{p}$ distributions of Yes and No overlap almost completely: the model cannot discriminate.

4.8 yardstick code and output

```
log_fit_a <- fit(log_wkflow_a, titanic_train)      # fit on
    ↪ TRAINING
res <- augment(log_fit_a, new_data = titanic_test)
# adds .pred_class, .pred_Yes, .pred_No to the test data
predict(log_fit_a, new_data = titanic_test, type = "prob") # .pred
    ↪ _Yes .pred_No
predict(log_fit_a, new_data = titanic_test)        # .pred_class
    ↪ only
res %>% conf_mat(truth = survived, estimate = .pred_class)
res %>% conf_mat(truth = survived, estimate = .pred_class) %>%
  autoplot(type = "heatmap")
res %>% accuracy(truth = survived, estimate = .pred_class)
m <- metric_set(accuracy, sensitivity, specificity)
res %>% m(truth = survived, estimate = .pred_class)
res %>% roc_auc(survived, .pred_Yes)    # prob. column of FIRST
    ↪ level
res %>% roc_curve(survived, .pred_Yes) %>% autoplot()
```

Real outputs (Lab 3 OJ logistic regression on test, event = CH; Titanic Model B):

```
          Truth                              # conf_mat: rows =
    ↪ Prediction,
Prediction  CH  MM                           #            cols =
    ↪ Truth
        CH 178  36      <- TP  FP
        MM  18  90      <- FN  TN
# A tibble: 3 x 3                            # metric_set(
    ↪ accuracy,
  .metric     .estimator .estimate           #   sensitivity,
    ↪ specificity)
1 accuracy    binary         0.832           # 268/322
2 sensitivity binary         0.908           # 178/196
3 specificity binary         0.714           # 90/126
# A tibble: 1 x 3                            # roc_auc(survived,
    ↪ .pred_Yes)
  .metric .estimator .estimate
1 roc_auc binary          0.496
# A tibble: 9 x 3                            # roc_curve(survived
    ↪ , .pred_Yes)
  .threshold specificity sensitivity
1   -Inf           0           1             # everything "Yes":
    ↪ (1,1)
2      0.113       0           1
...
8      0.408       0.285       0.689         # FPR = 1 - 0.285 =
    ↪ 0.715
9    Inf           1           0             # nothing "Yes":
    ↪ (0,0)
```

`autoplot()` on a `roc_curve()` tibble plots `sensitivity` vs. `1 - specificity` with a dotted diagonal.

yardstick traps / spot-the-bug

- **event level = FIRST factor level.** `sensitivity()`, `specificity()`, `roc_auc()`, `roc_curve()` treat `levels(y)[1]` as "positive". `factor()` orders levels alphabetically by default ("No" before "Yes"), hence Titanic's `levels = c("Yes", "No")`. `Smarket`: first level Down ⇒ `roc_curve(Direction, .pred_Down)`; OJ: `.pred_CH` (Lab 3).
- Pass the probability column *of the first level*. Using the other class's column (e.g. `.pred_No` when Yes is first) mirrors the curve: AUC becomes 1−AUC.

- `roc_curve()` / `roc_auc()` need **probabilities** (`.pred_Yes`), not `.pred_class`; given classes they throw an error (ROC). `accuracy()` and `conf_mat()` use `truth` = the factor outcome and `estimate` = `.pred_class`.
- yardstick's `conf_mat()` prints **Prediction as rows, Truth as columns**, event level first — the transpose (and reorder) of the slide layout S3.34. Always read the labels.
- Evaluate on the test set with the model fit on the training set: `augment(fit_on_train, new_data = test)`. (The reading's code re-fits the workflow on `titanic_test` before `augment()`; do not copy that.) Training-set metrics are optimistic.
- `augment()` adds `.pred_class` and one `.pred_<level>` per class (classification) or `.pred` (regression); `predict()` returns only predictions, so you `bind_cols()` the truth.

5 R code reference: tidyverse + tidymodels

Packages and set-up (Lab 1–3, S1.27–28)

`tidyverse` = data handling + plots (`dplyr`, `ggplot2`, `readr`, `tibble`, `forcats`, `tidyr`). `tidymodels` = modeling (`rsample` split, `recipes` preprocessing, `parsnip` model specs, `workflows`, `yardstick` metrics, `broom` `tidy()`, `tune`). Workflow picture (S1.28): Import → Tidy → Transform ↔ Visualize ↔ Model (tidymodels) → Share.

```
library(tidyverse); library(tidymodels)
library(corrplot); library(ggthemes)     # EDA extras
library(kknn)                  # engine for nearest_neighbor()
library(discrim); library(klaR) # LDA/QDA/naive Bayes specs
library(naniar); library(corrr) # vis_miss(); correlate()
tidymodels_prefer()  # tidymodels wins name conflicts
```

- `tidymodels_prefer()`: when two loaded packages export the same name (e.g. `MASS::select()` vs `dplyr::select()`, `MASS` is loaded with `klaR`), use the tidymodels/tidyverse version.
- Install once *outside* the .Rmd (`install.packages("kknn")`); an `install.packages()` line inside the .Rmd stops it knitting (Lab 1). `?fun` / `help(fun)` = documentation.
- Pipes: `%>%` (magrittr, Ctrl/Cmd+Shift+M) and native `|>` do the same here: `x %>% f(y)` = `f(x, y)`. `<-` = assignment ("gets").

5.1 The pipeline: split → recipe → spec → workflow → fit → predict → metrics

The 7 steps (memorise the order)

1. **Seed + split**: `set.seed()`, `initial_split()(data, prop, strata)`, `training()`, `testing()`.
2. **Recipe** (on *training* data): `recipe()(y ~ ., data = train)` + `step_*()`. Only a *plan*; nothing is computed.
3. **Model spec**: model function + `set_engine()` + `set_mode()`. No data yet.
4. **Workflow**: `workflow()` %>% `add_model()` %>% `add_recipe()`.
5. **Fit**: `fit()(wflow, train)` — preps the recipe on train, bakes train, fits the engine.
6. **Predict**: `predict()(fit, new_data = test)` (only predictions) or `augment()(fit, new_data = test)` (data + predictions).
7. **Evaluate**: `yardstick` `metric(data, truth = y, estimate = .pred_class / .pred)` or `metric_set()`.

Regression template (`diamonds` Lab 2, `abalone` HW2):

```
set.seed(3435)
d_split <- initial_split(diamonds, prop = 0.80, strata = price)
d_train <- training(d_split); d_test <- testing(d_split)

d_rec <- recipe(price ~ ., data = d_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

lm_mod <- linear_reg() %>% set_engine("lm")
knn_mod <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>% set_mode("regression")

lm_wf <- workflow() %>% add_model(lm_mod) %>%
  add_recipe(d_rec)

lm_fit <- fit(lm_wf, data = d_train)

lm_fit %>% extract_fit_parsnip() %>% tidy()  # coef table
res <- augment(lm_fit, new_data = d_test)  # test + .pred
reg_metrics <- metric_set(rmse, rsq, mae)
reg_metrics(res, truth = price, estimate = .pred)
```

Classification template (`titanic` HW3, `OJ`/`Smarket` Lab 3):

```
titanic <- read_csv("data/titanic.csv") %>%
  mutate(survived = factor(survived, levels = c("Yes", "No")),
         pclass = factor(pclass))   # outcome MUST be a factor
set.seed(131)
t_split <- initial_split(titanic, prop = 0.7, strata = survived)
t_train <- training(t_split); t_test <- testing(t_split)
t_rec <- recipe(survived ~ pclass + sex + age + sib_sp +
                parch + fare, data = t_train) %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp, parch,
    ↪ fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
log_mod <- logistic_reg() %>% set_engine("glm") %>%
  set_mode("classification")
log_fit <- workflow() %>% add_model(log_mod) %>%
  add_recipe(t_rec) %>% fit(t_train)
res <- augment(log_fit, new_data = t_test)
# res = t_test + .pred_class + .pred_Yes + .pred_No
res %>% conf_mat(truth = survived, estimate = .pred_class)
res %>% accuracy(truth = survived, estimate = .pred_class)
res %>% roc_auc(truth = survived, .pred_Yes)
res %>% roc_curve(truth = survived, .pred_Yes) %>% autoplot()
```

To swap models, change only the spec (step 3) and `add_model()`; the recipe is reused (Lab 3). Without a recipe use `add_formula(y ~ x1 + x2)`.

5.2 dplyr / tidyverse verbs

Every verb takes a data frame first and returns a new data frame (tibble); nothing changes unless you assign with `<-` (Lab 1).

Function	Does / example
`filter()`	keep *rows*: `filter(class == "compact" & manufacturer == "audi")`; `==`, `!=`, `>`, `&` and, `|` or, `!` not, `%in% c("IAH","HOU")`
`select()`	keep *columns*: `select(year, hwy, class)`; ranges `year:class`; drop `-(year:class)`, `-price`; helpers `starts_with()`, `everything()`, `where(is.numeric)`
`mutate()`	add/modify columns (added at the end): `mutate(age = rings + 1.5)`, `mutate(after_2k = if_else(year <= 2000, 0, 1))`
`transmute()`	like `mutate()` but keeps only the new columns
`summarise()`	collapse to one row (per group): `summarise(avg_hwy = mean(hwy), count = n())`
`group_by()`	following `summarise`/`mutate`/`filter` work per group; `ungroup()` undoes
`arrange()`	sort rows ascending; `arrange(desc(x))` or `arrange(-x)` descending
`count()`	`count(manufacturer, sort = TRUE)` = `group_by` + `summarise(n = n())`
`drop_na()`	delete rows with any NA (`airquality`: 153 → 111 rows)
`head()`, `glimpse()`, `dim()`, `summary()`	first 6 rows; column types; (rows, cols); summary stats
`factor()`, `levels()`	make categorical; `factor(x, levels = c("Yes","No"))` sets level order
`bind_cols()` / `bind_rows()`	glue side by side (predictions + truth) / stack tables
`table()`	class counts: `Smarket %>% select(Direction) %>% table()` → Down 602, Up 648

```
mpg %>%
  group_by(manufacturer) %>%
  summarise(avg_hwy = mean(hwy), count = n()) %>%
  filter(count >= 9) %>%
  arrange(avg_hwy)          # ascending; top 10 makers
airquality %>% summarise(mean(Ozone))              # NA!
airquality %>% summarise(mean(Ozone, na.rm = TRUE)) # number
abalone %>% summarise(n = n(), mean = mean(age), median = median(
    ↪ age),
  sd = sd(age), q1 = quantile(age, 0.25))   # HW2 Q1
```

5.3 ggplot2 for EDA

Grammar: `ggplot()(data, aes()(x, y, fill, color))` + `geom_*()` + `labs()` + `theme_bw()`. Layers are joined with `+`, *not* `%>%` (but you may pipe the data *into* `ggplot()`). Map a variable inside `aes(fill = sex)`; a constant goes outside (`geom_point(alpha = 0.2)`). Which plot answers

which question: see the EDA table in *Linear models*.

```
ggplot(mpg, aes(x = hwy)) + geom_histogram(binwidth = 2)
ggplot(mpg, aes(x = hwy, y = cty)) + geom_point() +
  geom_smooth(method = "lm")          # fitted line
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot() + geom_jitter(alpha = 0.3)
ggplot(Smarket, aes(x = Direction)) + geom_bar() # class balance
# HW3 Q2: percent stacked bar (each bar sums to 100%)
ggplot(t_train, aes(x = survived, fill = sex)) +
  geom_bar(position = "fill") + labs(y = "proportion")
# correlation matrix of numeric columns
t_train %>% select(where(is.numeric)) %>%
  cor(use = "complete.obs") %>%   # NAs (age) else NA
  corrplot(type = "lower", diag = FALSE, method = "number")
Smarket %>% select(-Direction) %>% correlate() %>% rplot()
# predicted vs observed (Lab 2, HW2): points on dashed 45-degree
    ↪ line = perfect
ggplot(res, aes(x = .pred, y = price)) + geom_point(alpha = 0.2) +
  geom_abline(lty = 2) + coord_obs_pred()
ggplot(res, aes(.pred, price)) + geom_point() + facet_wrap(~ model
    ↪ )
```

Other helpers seen: `reorder(cut, price)` orders categories by mean price; `fct_infreq()` by frequency, `fct_rev()` reverses; `geom_tile()` heatmap; `xlab()`; `theme_bw()`. `vis_miss()(airquality)` plots missingness (`naniar`).

5.4 rsample: splitting

`set.seed()(3435)`	before every random step (split; later CV) ⇒ reproducibility; any number
`initial_split()(df, prop, strata)`	returns an `rsplit` object (prints `<Training/Testing/Total>`), not data
`prop`	fraction to **training** (default 0.75); 0.80 Lab 1–2, 0.70 Lab 3, 0.85 HW2 (3548/629)
`strata`	stratified sampling on the outcome (numeric: binned into quartiles) so train/test have the same outcome distribution
`training()`, `testing()`	extract the two data frames from the split

Details and why stratify: see *Data splitting* in Linear models.

5.5 recipes: steps, selectors, prep/bake

`recipe(outcome ~ predictors, data = train)`: the formula lists roles (" . " = all other columns, *including* IDs and leaky columns); `data` supplies column names/types and is what `prep()` estimates from. Steps run **in the order written**.

Step	What it does
`step_dummy()`	usually on `all_nominal_predictors()`: factor/character with K levels → $K-1$ 0/1 columns `var_level`; first level = reference level (no column) (S2.45)
`step_dummy(type, one_hot = TRUE)`	K columns (one-hot encoding) (S2.46): rank-deficient with `lm` (has intercept), fine for KNN (PE Q7, Q13)
`step_normalize()`	$(x-\bar{x})/s$ with train mean/SD = `step_center()` + `step_scale()`
`step_interact()(terms = ~ a:b)`	adds product column `a_x_b`; for factors use the dummy names: `starts_with("type_")`: `shucked_weight`
`step_impute_linear()`	`(x, impute_with = imp_vars()(a, b))`: fill NAs of x by a linear regression on the listed (complete) columns
`step_impute_mean()` / `_median` / `step_impute_mode()` / `step_impute_knn()` / `step_impute_bag()`	other imputations (S2.7); mean/median/mode shrink variance
`step_poly()(x, degree = 2)`	orthogonal polynomial columns `x_poly_1`, `x_poly_2`
`step_log()(x)`	$\log(x)$ (natural; `base` = to change)
`step_mutate(new = expr)`	any new column, e.g. `square_meters = square_feet * 0.092903` (Sai Q9)
`step_rm(id)`	remove a column (IDs, leaky vars)

Selectors (used inside steps)

`all_predictors()` (every predictor) · `all_numeric_predictors()` · `all_nominal_predictors()` (factor/character) · `all_outcomes()` · by name (type, age) · `starts_with()("type_")` · `contains()`, `ends_with()` · exclude with `-`: `all_predictors(), -age`. `all_predictors()` never touches the outcome, so the outcome is not normalized.

Step order that works

impute → transform (log, poly, mutate) → step_dummy → step_interact → step_normalize/center/scale.

- Impute first: dummies/products/means cannot be computed from NAs.
- Dummy before interact: after `step_dummy()` the factor `sex` no longer exists; it is `sex_male`, hence `starts_with("sex")` (HW3 Q4).
- Dummy before normalize when normalizing `all_predictors()`: centering/scaling needs numeric columns (a factor would error). With `all_numeric_predictors()` written *after* `step_dummy`, the 0/1 dummies get normalized too (HW2 centers/scales every predictor, dummies and interactions included).

```
abalone_recipe <- recipe(age ~ type, data = abalone_train) %>%
  step_dummy(type)                                  # S2.45
abalone_recipe %>% prep() %>%
  bake(new_data = abalone_train)  # or new_data = NULL
```

```
# A tibble: 3,131 x 3
    age type_I type_M      # type has levels F, I, M
  <dbl>  <dbl>  <dbl>      # F = reference (both 0)
1   8.5      1      0      # an infant
3   8.5      0      1      # a male
5   9.5      0      0      # a female
```

- `prep()` = *estimate* everything from the training data (means/SDs, imputation models, factor levels); `bake()(new_data = X)` = *apply* those fixed values to `X`; `new_data = NULL` returns the processed training set. Used only to *look* at the result — `fit()` on a workflow does prep+bake itself, and `predict()`/`augment()` bake new data with the **training** estimates.
- Baked columns: original numerics, then outcome, then new dummies/interactions at the end (HW2: `type_I`, `type_M`, `type_I_x_shucked_weight`, `longest_shell_x_diameter`, …).
- **Ordered factors** (`diamonds`: `cut`, `color`, `clarity`) get *polynomial* contrasts, not 0/1: columns `cut_1`…`cut_4` with values like 0.632, -0.267 (Lab 2).

Printed recipe (HW2):

```
-- Recipe ------------------------------------
-- Inputs
Number of variables by role
outcome:   1
predictor: 8
-- Operations
* Dummy variables from: all_nominal_predictors()
* Interactions with: ...
* Centering for: all_predictors()
* Scaling for: all_predictors()
```

5.6 parsnip: model specs, engines, modes

Model	Spec function	Engine	Mode / package / main args
linear regression	`linear_reg()`	`"lm"`	regression only (default); base R `lm`
logistic regression	`logistic_reg()`	`"glm"`	classification only; base R `glm` (binomial)
KNN	`nearest_neighbor()`	`"kknn"`	**must** set mode `"regression"` or `"classification"`; `library(kknn)`; `neighbors` = k (default 5)
LDA	`discrim_linear()`	`"MASS"`	classification; `library(discrim)`, engine pkg `MASS`
QDA	`discrim_quad()`	`"MASS"`	classification; `library(discrim)`
naive Bayes	`naive_Bayes()`	`"klaR"`	classification; `library(discrim)` + `library(klaR)`; `set_args(usekernel = FALSE)` = Gaussian

- Spec = model type + `engine` (which R package computes it) + `mode` (regression/classification). Same thing as arguments: `nearest_neighbor(mode = "classification", engine = "kknn", neighbors = 7)`.
- Hyperparameters go *in the model function* (`nearest_neighbor(neighbors = 2)`) or via `set_args()`; never in the recipe. Spelling: `naive_Bayes` (capital B), `nearest_neighbor` (no s).
- The spec function for LDA/QDA/naive Bayes only exists after `library(discrim)`; "could not find function" = package not loaded.

```
> knn_model             # printing a spec (HW2)
K-Nearest Neighbor Model Specification (regression)
Main Arguments:
```

```
  neighbors = 7
Computational engine: kknn
> knn_fit %>% extract_fit_parsnip()   # Lab 2
parsnip model object
Call: kknn::train.kknn(formula = ..y ~ ., data = data,
                       ks = min_rows(5, data, 5))
Type of response variable: continuous
minimal mean absolute error: 373.8765
Minimal mean squared error: 515091.1
Best kernel: optimal
Best k: 5
```

("Best k: 5" only because 5 was the only (default) value tried (Lab 2).)

5.7 fit, predict, augment, tidy

predict() / augment() output columns

Call	Returns (a tibble, same rows as new_data)
`predict(fit, new_data = d)` regression	`.pred` (type `"numeric"`)
`predict(fit, new_data = d)` classification	`.pred_class` (type `"class"`: most probable class, i.e. $\hat{p} > 0.5$ for 2 classes)
`predict(fit, d, type = "prob")`	one column per level `.pred_<level>`: `.pred_Yes` `.pred_No`; `.pred_Down` `.pred_Up`; `.pred_CH` `.pred_MM` (rows sum to 1)
`predict(lm_fit, d, type = "conf_int")` / `"pred_int"`	`.pred_lower`, `.pred_upper` (CI / PI, linear reg.)
`augment()(fit, new_data = d)`	all columns of d **plus** `.pred` (regr.) or `.pred_class` + all `.pred_<level>` (classif.)

```
pr <- predict(lm_fit, new_data = d_train %>% select(-price))
pr <- bind_cols(pr, d_train %>% select(price))  # add truth
predict(smarketlog_fit, new_data = smarket_train, type = "prob")
```

```
# A tibble: 874 x 2
   .pred_Down .pred_Up          # Lab 3: all near 0.5 =>
1       0.521    0.479          # model can't discriminate
```

- **Predicting one new case** (HW2 Q7): build a `tibble()` with *every* predictor the recipe uses, same names and types (factor with the training levels): `tibble(type = factor("F", levels = levels(abalone$type)), longest_shell = 0.5, ...)`, then `predict(lm_fit, new_data = new_abalone)` (recipe applied automatically; → `.pred` 21.8).
- `predict()` gives *only* the prediction columns ⇒ `bind_cols()` the truth before metrics; `augment()` already keeps it.

Reading tidy() output

`tidy()` on a fitted workflow (directly, or after `extract_fit_parsnip()`) = coefficient table:

```
# A tibble: 24 x 5                      # Lab 2 diamonds, lm
  term        estimate std.error statistic   p.value
1 (Intercept)  5317.       496.      10.7   8.54e- 27
2 carat       11282.        54.3    208.    0
6 y               9.08      19.9      0.456 6.49e-  1
8 cut_1         596.        25.0     23.8   1.22e-124
```

- `term` = predictor column after the recipe (dummy names); `estimate` = $\hat{\beta}_j$; `std.error` = SE($\hat{\beta}_j$); `statistic` = estimate/std.error (t for `lm`, z for `glm`); `p.value` tests H_0: $\beta_j = 0$ given the other predictors (here y is not significant, $p = 0.65$).
- Interpretation: `carat` $+1 \Rightarrow$ price $+\$11{,}282$ holding the others fixed. If the recipe normalized predictors, an estimate is the change per 1 SD and `(Intercept)` $= \bar{y}_{\text{train}}$ (HW2: 11.5).
- Logistic (`glm`): estimates are changes in log odds of the **second** factor level; $e^{\hat{\beta}}$ = odds ratio (see *Classification models*).
- KNN has no coefficients: `tidy()` is not informative; print `extract_fit_parsnip()` instead.

Base R equivalents (slides, ISLR)

```
fit <- lm(price ~ carat + cut, data = d_train); summary(fit)
glm.fit <- glm(default ~ student + balance + income,
               data = Default, family = binomial)   # S3.26
predict(glm.fit, newdata = new, type = "response")  #
    ↪ probabilities
predict(glm.fit, newdata = new)  # default type = "link": LOG
    ↪ ODDS
```

Base R uses `newdata=` and `summary()`; tidymodels uses `new_data=` and `tidy()`. Perfectly separable data ⇒ `glm` warns "algorithm did not converge" / "fitted probabilities numerically 0 or 1 occurred" (S3.45).

5.8 yardstick: metrics

Every metric: `metric(data, truth = <true col>, estimate = <prediction col>)`, `truth` **first**; returns a tibble `.metric` / `.estimator` (`standard` = regression, `binary`) / `.estimate`; pull the number with `$.estimate`. Full outputs and interpretation: see *Model evaluation*.

Call	Needs
`rmse()`, `rsq()`, `mae()` (`rsq_trad()`)	numeric `truth`, `estimate = .pred`. No `mse()` in yardstick: MSE = RMSE^2
`accuracy()`, `sensitivity()`, `specificity()`, `precision()`	factor `truth`, `estimate = .pred_class` (same levels)
`conf_mat()(truth =, estimate = .pred_class)`	rows = Prediction, cols = Truth; `%>% autoplot(type = "heatmap")` (or `"mosaic"`)
`roc_auc()(truth, .pred_Yes)`	factor truth + **probability** column of the **event** (first level); 3rd argument is unnamed
`roc_curve()(truth, .pred_Yes)`	returns `.threshold`, `specificity`, `sensitivity`; `%>% autoplot()` draws the ROC curve
`metric_set()(accuracy, sensitivity, specificity)`	makes a new function; list metric names *without* `()`; call it like any metric

- **Event level** = **first** factor level (`event_level = "first"` default). Smarket: `Down` ⇒ `roc_curve(Direction, .pred_Down)`; OJ: `.pred_CH`; Titanic: relevel so "Yes" is first. Alternative: keep levels and pass `event_level = "second"`.
- Lab 3 Smarket QDA test: sensitivity 0.254 $= 46/(46+135)$ is the rate for `Down`, the first level.

5.9 Common bugs / exam traps

Spot-the-bug checklist

- **No `set.seed()`** before `initial_split()` ⇒ a new random split (and new results) every run (Lab 1, HW2).
- **Recipe/model on the wrong data**: `recipe(..., data = test)`, `fit(wf, test)`, or recipe built on the full data before splitting ⇒ test information leaks into training (means/SDs/imputation from test). Recipe and `fit()` use **training**; test is only for the final `predict()`/`augment()`.
- **Metrics on training data** reported as performance: optimistic (KNN: each training point is its own neighbor; Lab 2 $k = 2$ train R^2 0.999). Use `augment(fit, new_data = test)`.
- **Recipe defined but never used** (PE Q7): `loan_model |> fit(defaulted ~ ., data = loans)` fits raw data; the `step_dummy` never runs. Fix: `workflow() %>% add_model() %>% add_recipe()` then `fit(wf, train)`.
- **`y ~ .`** pulls in identifier variables (`application_id`) and target leakage (`default_date`, `rings` for age) (PE Q7, HW2 Q3).
- **KNN without `step_normalize()`** ⇒ large-scale variables dominate distances (PE Q6–7).
- **`one_hot = TRUE` with `lm`** ⇒ "rank-deficient" fit (dummies sum to the intercept column); use default `one_hot = FALSE` → $K-1$ dummies (3 departments → 2) (PE Q13). Fine for KNN.
- **Redundant predictors**: `step_mutate(square_meters = square_feet * 0.092903) + step_normalize()` still leaves two perfectly collinear columns; normalizing does not fix collinearity — drop one (Sai Q9).
- **Step order**: `step_interact(~ sex:fare)` after `step_dummy` must use `starts_with("sex")`; `step_normalize(all_predictors())` before `step_dummy` hits factor columns; impute before anything that uses the column.
- **Outcome not a factor** for classification (0/1 numbers, character): `logistic_reg()` / `conf_mat()` need a factor ⇒ `mutate(y = factor(y))`.
- **Wrong event level**: `factor()` sorts levels alphabetically (`"No"` before `"Yes"`), so sensitivity/AUC would be for "No". Relevel (`levels = c("Yes","No")`) or `event_level = "second"`. Passing `.pred_No` when Yes is the event gives AUC $= 1-$AUC.
- **glm vs yardstick**: glm coefficients are for the **second** level; yardstick's event is the **first**.
- **Missing `type = "prob"`**: `predict()` default gives only `.pred_class`;

`roc_auc()`/`roc_curve()` need `.pred_<event>` (`augment()` has both).

- **`roc_auc(truth, .pred_class)`** = error (needs numeric probabilities); `accuracy(truth, .pred_Yes)` = error (needs classes).
- **Argument names**: `new_data =` (tidymodels) vs `newdata =` (base R); `truth` before `estimate`.
- **Missing `set_mode()`** for `nearest_neighbor()` (mode unknown) ⇒ error at fit.
- **Package not loaded**: `discrim_linear()`/`naive_Bayes()` need `library(discrim)`; `"kknn"` needs kknn installed.
- **`filter(x = 5)`** instead of ==; `%>%` between ggplot layers instead of +; `mean(x)` with NAs returns NA (`na.rm = TRUE`); forgetting <- so the result is not saved.
- **Picking a model by training accuracy** (Lab 3 did, only because CV was not yet taught): Smarket QDA best on train (0.548) but 0.487 on test (Lab 3).
- Lab 2 text says "metric set of RMSE, MSE, and R^2" but the code `metric_set(rmse, rsq, mae)` computes **MAE**.

6 Exam prep: past questions, patterns, quick answers

Format (Canvas): ≤12 multiple choice + 4 short answer, paper notes only; R code / R output may appear. Sai's midterm (Summer 2026, 26 pts) = 7 MC × 2 pts + 2 free-response (8 + 4 pts). MC rule from the cover sheet: *more than one bubble = zero*; "Both X and Y" options are traps unless *both* parts are right. Round to 3–4 decimals.

6.1 Practice exam: every question, one line

Practice exam (Fall 2026) answer key, condensed (PE, official solutions)

Q1 KNN great on train, worse on new data ⇒ **(b) increase** k. Small k = overfitting (high variance); decreasing k makes it worse; weighting high-variance variables hurts (standardization); interactions irrelevant for non-parametric KNN. (PE Q1)

Q2 "Most flexible is always best" — wrong: flexible models lose interpretability (bad if goal is inference) and can fit noise ⇒ low training error, high testing error. (PE Q2)

Q3 Train/test acc. A 71/70, B 86/82, C 100/68 ⇒ **B** (highest *test* accuracy); C overfits (B a little). (PE Q3)

Q4 All three claims **false**: (a) zero Pearson correlation = no *linear* relation, can still help nonlinear/empirical models; (b) correlated ≠ interacting (interaction term = effect of X_1 depends on X_2); (c) lower training error not better. (PE Q4)

Q5 Neighbors A B B A A B B: $k = 1$ **A**, $k = 3$ **B** (2B:1A), $k = 5$ **A** (3A:2B), $k = 7$ **B** (4B:3A). (PE Q5)

Q6 Age standardized, income not: KNN is distance-based; income's huge variance dominates the Euclidean distance. Normalize *all* numeric predictors. (PE Q6)

Q7 Loan KNN code, ≥4 bugs: (1) `defaulted ~ .` pulls in `application_id` (identifier variable) and `default_date` (target leakage) — drop them; (2) no `step_normalize()`; (3) recipe never used (`fit()` on raw formula, no `workflow()`/`add_recipe()`); (4) so `employment_type` never dummy-coded. `one_hot = TRUE` is *fine* for KNN (no intercept). (PE Q7)

Q8 Strong U-shape, $r = 0.03$ ⇒ **(c)** little linear correlation despite strong nonlinear relationship; X can still predict Y. (PE Q8)

Q9 Evidence of interaction = **(b)** X_1–Y slope positive for one level of X_2, negative for the other. Correlation/skew are not evidence. (PE Q9)

Q10 Graduation model (info at end of year 1): exclude **StudentID** (ID), **GraduationDate**, **FinalGPA** (unknown at prediction time = leakage), **Graduated4Years** (the outcome). Keep HS_GPA, FirstYearGPA, Major, Credits*. (PE Q10)

Q11 Need dummies for logistic regression: **Major, Dorm** (categorical). (PE Q11)

Q12 CompletionRate = Completed/Attempted (year-1 info) is **not** leakage; but with both components it is redundant (collinearity). (PE Q12)

Q13 Rank-deficient `lm`: `step_dummy()` with `one_hot = TRUE` gives 3 indicators that sum to the intercept column ⇒ design matrix not full rank. Fix `one_hot = FALSE` ⇒ **2** indicators ($K-1$, one reference level). (PE Q13)

Q14 $\hat{p} = 0.78$ ⇒ **(c)** model assigns probability 0.78 of the pneumonia class (not "78% accurate", not certain). (PE Q14)

Q15 $\hat{p}$ = .31,.48,.63,.81: threshold 0.5 ⇒ **C, D**; 0.7 ⇒ **D**; raising t ⇒ fewer positives. (PE Q15)

Q16 $\beta = 0.04$ for MonthlyFee: not "causes +4% probability". Linear in log odds, and only association: "$1 more is *associated* with +0.04 log odds" (odds $\times e^{0.04} = 1.041$). (PE Q16)

Q17 TP 72, FP 18, FN 28, TN 182: accuracy $254/300 = 0.8467$, TPR $72/100 = 0.72$, FPR $18/200 = 0.09$. (PE Q17)

Q18 Threshold 0.5 → 0.3 does **not** change AUC (ROC built over all thresholds). (PE Q18)

Q19 Dangerous disease, FN ≫ FP cost ⇒ **Point C** (highest TPR, accept FPR ≈ 0.5) on the ROC curve. (PE Q19)

Q20 $n = 50{,}000$, $p = 2$, covariances clearly differ ⇒ **(b)** QDA (LDA assumes a common covariance matrix; extra variance negligible at this n). (PE Q20)

6.2 Sai's graded midterm: every question, where points were lost

Q	Topic (correct answer, why)	Sai	Key	Pts
1	More flexible? Add polynomial/interaction terms. (↑ k, fewer predictors, LDA vs QDA = *less* flexible)	C	C	2/2
2	Why KNN makes nonlinear boundaries: classification decided *locally* by neighbors	C	C	2/2
3	ROC point A→C = *decreasing* the threshold (TPR and FPR both rise)	B	B	2/2
4	Readmission predictor: `HospitalID` (15 hospitals, populations differ = real group info); others are unique IDs	E	B	**1/2**
5	Covariances truly equal: QDA gains nothing, only adds estimation variance	B	B	2/2
6	Interaction: dosage effect $+10$ under A vs. $+25$ under B	F	B	**0/2**
7	$\beta_{\text{age}} > 0$ but class unchanged: $\hat{p}$ rose without crossing the threshold	A	B	**0/2**
8a	Threshold $0.40 \to 0.70$: sensitivity decreases	✓		2/2
8b	specificity increases (= 1−FPR, FPR falls)	✓		2/2
8c	ROC curve unchanged (model-level; only the operating point moves)	✓		2/2
8d	Number flagged decreases (fewer $\hat{p}$ above the bar)	✓		2/2
9	Keep `square_feet` & `square_meters` after normalizing? **No**: exact collinearity	ok		**3/4**

Total 20/26. Verified against the scanned booklet and Gradescope rubric ((Sai pp. 1–8)).

Sai's lost points and the correct reasoning

- **Q4 (−1).** Sai: (E) "None of the above". Key: (B) `HospitalID`. A grouping ID with few levels that *carries information* (patient populations differ) is a legitimate categorical predictor (dummy-encode it); `PatientID`, `AdmissionID`, `RecordID` are unique per row (identifier variable: no pattern can generalize). *Unclear grading:* the rubric item applied reads "partial credit for answering all variables", which does not literally match (E); it appears the grader gave half credit for rejecting the three unique IDs. Rule: unique-per-row ID ⇒ drop; shared group label ⇒ keep. (Sai Q4)
- **Q6 (−2).** Sai: (F) "Both B and D". D ("dosage increases the fitted value under both treatments") is a main effect, not evidence of interaction; so F is wrong even though B is right. Interaction = the *size* (or sign) of one variable's effect differs across levels of the other: $30 - 20 = 10$ vs. $50 - 25 = 25$. (Sai Q6)
- **Q7 (−2).** Sai: (A) "a positive coefficient does not affect predicted probabilities" — false; a positive β raises the log odds and hence $\hat{p}$. Key (B): $\hat{p}$ can rise (e.g. 0.31→0.32) without crossing $t = 0.5$, so the class is unchanged. (Sai Q7)
- **Q9 (−1).** Sai said "not appropriate" (right) but argued via normal curves. Required word: **collinearity**. `square_meters` = $0.092903 \times$ `square_feet` exactly, so after `step_normalize()` the two columns are *identical* ($r = 1$); OLS cannot separate their coefficients (one is NA/huge SEs). Normalizing never removes collinearity — drop one. (Sai Q9)

6.3 Question patterns and how to answer them

P1. Confusion matrix → metrics (PE Q17; HW3 Q10; S3.34–37)

Recipe. (1) Find the orientation: are *rows* predicted or true? (PE Q17: rows = predicted; S3.34: rows = true; `conf_mat()` prints `Prediction` rows × `Truth` columns.) (2) Which class is "positive"? (yardstick: the *first* factor level.) (3) Label TP, FP, FN, TN by *meaning*. (4) Divide by the right total: actual positives (TPR, FNR), actual negatives (FPR, spec.), predicted positives (precision).

Example. Truth Yes: 40 predicted Yes, 10 predicted No; truth No: 20 predicted Yes, 130 No. So TP 40, FN 10, FP 20, TN 130, $n = 200$. accuracy $= 170/200 = 0.85$; sensitivity $= 40/50 = 0.80$; specificity $= 130/150 = 0.867$; FPR $= 20/150 = 0.133$; precision $= 40/60 = 0.667$; error rate $= 0.15$.

P2. Change the threshold: what happens? (Sai Q7–8; PE Q15)

Recipe. Predict positive if $\hat{p} \geq t$. Raise $t \Rightarrow$ fewer positives $\Rightarrow$ TP, FP ↓; sensitivity/TPR ↓, FPR ↓, specificity ↑; the ROC curve and AUC are **unchanged** (only the operating point slides down-left). Lower t: the reverse. Coefficients and $\hat{p}$ never change with t.

Example. $\hat{p}$ / truth: 0.15 No, 0.40 Yes, 0.62 No, 0.85 Yes.

t	TP	FP	FN	TN	sens	spec
0.3	2	1	0	1	1.0	0.5
0.5	1	1	1	1	0.5	0.5
0.7	1	0	1	2	0.5	1.0

P3. Read an ROC curve / compare AUCs (PE Q18–19; Sai Q3, Q8c; ROC)

Recipe. x = FPR $(1-$spec.$)$, y = TPR (sens.); each point = one threshold. Up/right = lower t. Pick the point by costs: FN costly (disease, fraud) $\Rightarrow$ high-TPR point (up/right); FP costly $\Rightarrow$ low-FPR point (down/left). Compare models by AUC (higher better; 0.5 = coin flip, 1 = perfect, AUC $= 1 \Rightarrow$ suspect target leakage). AUC does not depend on the chosen threshold.

Example. Same 4 cases as P2: positives score 0.40, 0.85; negatives 0.15, 0.62. Of the 2×2 positive–negative pairs, the positive scores higher in 3 $\Rightarrow$ AUC $= 3/4 = 0.75$ (standard "probability a random positive outranks a random negative" reading). Model A AUC 0.88 vs. B 0.79 $\Rightarrow$ A discriminates better at almost every threshold.

P4. Flexibility, bias–variance, k in KNN (PE Q1–2; Sai Q1–2, Q5; S1.113–115)

Recipe. More flexible $\Rightarrow$ lower bias, higher variance, training error ↓, test error U-shaped. *More* flexible: smaller k, add predictors / polynomial / interaction terms, QDA instead of LDA, KNN instead of a linear model. Overfit symptom: train $\ll$ test $\Rightarrow$ make it *less* flexible (increase k, drop terms).

Example. $k = 1$: each training point is its own neighbor, training error $= 0$ (maximal variance). $k = n$: always predicts the overall majority class (maximal bias). With 400 training points, $k = 1$ train/test error 0.00/0.30 vs. $k = 15$ 0.18/0.21 $\Rightarrow$ choose $k = 15$.

P5. Pick a model from a train/test table (PE Q3; HW2 Q9; Lab 2)

Recipe. Choose by the **test** column (highest accuracy/AUC/R^2, lowest RMSE/MAE). Then diagnose: big train–test gap = overfitting; train ≈ test but both poor = underfitting.

Example. RMSE train/test: linear 2.18/1.95, KNN 1.46/2.21 (HW2 abalone) $\Rightarrow$ linear model (lower test RMSE); KNN overfits.

P6. KNN by hand (PE Q5–6; S1.103–109)

Recipe. Standardize first; compute distances $d = \sqrt{\sum_j (x_j - x_{0j})^2}$; sort; classification = majority vote of the k closest; regression = *average* y of the k closest.

Example. $x_0 = (0, 0)$; $(1, 1)$ A: $d = 1.41$; $(2, 0)$ B: 2; $(0, 3)$ B: 3; $(3, 4)$ A: 5. $k = 1 \Rightarrow$ A; $k = 3 \Rightarrow$ B (A,B,B). Regression: nearest 3 have $y = 10, 12, 17$ $\Rightarrow \hat{y} = 13$.

P7. Interpret a logistic coefficient / compute $\hat{p}$ (PE Q14, Q16; Sai Q7; S3.12–25)

Recipe. β_j = change in log odds per unit X_j, others held fixed; e^{β_j} = odds ratio (multiplicative); only the *sign* transfers to p; say "associated with", never "causes". To predict: $z = \beta_0 + \sum \beta_j x_j$, $\hat{p} = e^z/(1+e^z)$, then compare with t.

Example. $\beta_0 = -3$, $\beta_1 = 0.05$ (age). Age 40: $z = -1$, $\hat{p} = e^{-1}/(1+e^{-1}) = 0.269 \Rightarrow$ "No" at $t = 0.5$. Age $+10$: odds $\times e^{0.5} = 1.65$. $\beta = 0.7$: odds $\times e^{0.7} = 2.01$ (about double). Template: "A one-unit increase in X is associated with a β increase in the log odds of Y=Yes (odds multiplied by e^{β}), holding other predictors fixed."

P8. Read regression output; dummy and interaction coefficients (HW2 Q6; PE Q13; S2)

Recipe. `tidy()` columns: `term`, `estimate` ($\hat{\beta}$), `std.error`, `statistic` = estimate/SE (t-statistic), `p.value` ($< 0.05 \Rightarrow$ significant). Dummy coefficient = difference from the reference level (the level with no column), others fixed. Interaction $X_1{:}X_2$ coefficient = how much the X_1 slope changes per unit of X_2.

Example. `diameter` 2.20, SE 0.308 $\Rightarrow t = 7.14$, $p \approx 10^{-12}$. Salary $= 40 + 2$ exp $+ 5$ IT $- 3$ HR (Sales = reference): IT with 10 yrs $\Rightarrow 40 + 20 + 5 = 65$; HR earns 8 less than IT at equal experience.

P9. Spot an interaction in a table / plot (PE Q9; Sai Q6)

Recipe. Compute the effect of X_1 at each level of X_2 (differences, or slopes). Different $\Rightarrow$ interaction; equal (parallel lines) $\Rightarrow$ additive. Correlation between predictors is irrelevant.

Example. Low/High dose: A 20/30, B 25/50 $\Rightarrow$ +10 vs +25: interaction. A 20/30, B 25/35 $\Rightarrow$ +10 both: none (B just shifted up 5).

P10. Which variables may be predictors? (PE Q7, Q10, Q12; Sai Q4; HW2 Q3)

Recipe — ask four questions of each variable: (1) Is it the outcome or computed from it? (2) Is it known *at prediction time*? (no $\Rightarrow$ target leakage). (3) Is it unique per row (identifier variable)? (4) Is it an exact function of another predictor (collinearity)? Group labels with a few shared levels (hospital, dorm, major) are fine $\Rightarrow$ dummy-encode.

Example. Predict abalone age: `rings` is out (age = rings $+1.5$, leakage). Loan default: `default_date` out (only exists after default), `application_id` out (ID).

P11. Choose recipe steps (and their order) (S2; Lab 2–3; HW2 Q3; HW3 Q4)

Situation	Step
missing numeric values	`step_impute_linear()` (`imp_vars()`) or `step_impute_mean()`
categorical predictor, lm/glm/LDA	`step_dummy()` (default $K - 1$ columns)
categorical, KNN	`step_dummy(..., one_hot = TRUE)` OK (no intercept)
KNN (distance)	`step_normalize()` = `step_center()` + `step_scale()`
effect of X_1 depends on X_2	`step_interact()`, after dummying: `~ starts_with("sex"):fare`
curved relationship	`step_poly()`; skewed X `step_log()`

Order: impute → dummy → interact → normalize. Recipe is *trained on the training data* (`recipe(y ~ ., data = train)`).

P12. Spot the bug in tidymodels code (PE Q7, Q13; Sai Q9; Lab 3)

Checklist: **(1)** recipe built but model fit with `fit(y ~ ., data)` $\Rightarrow$ recipe ignored; use `workflow()` %>% `add_recipe()` %>% `add_model()`. **(2)** `y ~ .` includes IDs / leakage. **(3)** KNN without `step_normalize()`. **(4)** `one_hot = TRUE` with an intercept model (rank deficient). **(5)** `nearest_neighbor()` without `set_mode()` (mode unknown). **(6)** metrics on *training* data / `predict()` on training instead of `testing()`. **(7)** `roc_auc()` given `.pred_class` instead of the probability column (`.pred_Yes`, from `type = "prob"`). **(8)** wrong event level (first factor level is the event). **(9)** no `set.seed()` before `initial_split()`; no `strata` for an imbalanced outcome. **(10)** `step_interact()` naming a variable that `step_dummy()` already renamed (`sex` → `sex_male`).

P13. LDA vs. QDA vs. logistic vs. KNN (PE Q20; Sai Q2, Q5; S3.89–93)

Recipe. Boundary linear? → logistic or LDA (LDA if predictors roughly normal within classes / small n; logistic if normality doubtful). Class covariances clearly differ *and* n large relative to p → QDA. Covariances equal → LDA (QDA only adds variance). Very irregular boundary, large n, small p → KNN. Flexibility: KNN > QDA > LDA = logistic.

Example. $n = 150, p = 8$, similar spreads ⇒ LDA. $n = 50{,}000, p = 2$, different covariances ⇒ QDA. Binary outcome, skewed/dummy predictors ⇒ logistic regression.

P14. Correlation, collinearity, class imbalance (PE Q4, Q8; Sai Q9; HW3 Q1–2)

Recipe. r measures *linear* association only ($r \approx 0$ with a U-shape is possible). Two predictors with $|r| \approx 1$ (or one an exact rescaling of the other) ⇒ collinearity: keep one. Imbalanced outcome ⇒ stratify the split (`strata`) and judge accuracy against the majority-class baseline.
Example. 90% negatives: "always negative" scores accuracy 0.90 with sensitivity 0 and AUC 0.5. Titanic: 61.6% died ⇒ any useful model must beat 0.616.

P15. Read other R output (Lab 2–3; HW2–3)

`predict()` default `type = "class"` → `.pred_class`; `type = "prob"` → `.pred_Yes`/`.pred_No`; regression → `.pred`. `augment()` = data + all prediction columns. Metric tibbles: `.metric`, `.estimator`, `.estimate`. LDA print: *Prior probabilities* = training class proportions ($\hat{\pi}_k$), *Group means* = $\hat{\mu}_k$, *LD1* = discriminant direction. kknn print: "Minimal misclassification" = leave-one-out training error. `initial_split()` print `<Training/Testing/Total>`.

6.4 60-second facts

High-yield one-liners

- Expected test MSE $= \mathrm{Var}(\hat{f}) + \mathrm{Bias}^2 + \mathrm{Var}(\epsilon)$; last term = irreducible error, always $\geq$ it.
- supervised learning has a Y; unsupervised learning does not. regression = numeric Y, classification = categorical Y.
- Parametric (linear, logistic, LDA, QDA) = assume a form for f; nonparametric model (KNN) = no form, needs more data.
- KNN: small k = flexible = low bias / high variance; $k = 1$ training error 0. Always normalize.
- curse of dimensionality: with many predictors, the "nearest" neighbors are far away ⇒ KNN degrades.
- Lower training error is never by itself a reason to prefer a model; compare on testing data.
- Bayes classifier: assign the class with the largest $\Pr(Y = k \mid X = x_0)$; its error = Bayes error rate (lowest possible).
- R^2 = fraction of variance in Y explained; RMSE/MAE in units of Y, lower is better.
- K levels ⇒ $K - 1$ dummies with an intercept; K (one-hot) only without an intercept / for KNN.
- Interaction = effect of one predictor depends on another; hierarchical principle: keep main effects if the interaction is in.
- Correlation $\neq$ interaction; correlation $\approx 0 \neq$ useless; correlation $\neq$ causation.
- logistic regression: logit linear in X; e^{β} = odds ratio; $p = 0.5 \Leftrightarrow$ log odds $= 0$; odds $= p/(1-p)$.
- Logistic fit by maximum likelihood; LDA by plugging in $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$.
- LDA: common Σ, linear decision boundary; QDA: Σ_k per class, quadratic boundary, more parameters.
- confounding: a coefficient's sign can flip once another predictor is added (Default `student`).
- sensitivity = TPR = recall = $TP/(TP + FN)$; specificity = $TN/(TN + FP) = 1 -$FPR; precision $= TP/(TP + FP)$.
- Raise threshold: sens ↓, spec ↑, positives ↓; ROC curve and AUC unchanged.
- ROC curve: x = FPR, y = TPR; diagonal = random (AUC 0.5); top-left corner = perfect.
- AUC is threshold-free and summarizes all thresholds; AUC = 1 on real data ⇒ check for target leakage.
- `yardstick`: first factor level = event ⇒ `factor(survived, levels = c("Yes","No"))`.
- `glm` (logistic) models the *second* level: with Yes first, coefficients are for "No".
- Stratify on the outcome so train/test keep the class proportions; `set.seed()` for reproducibility.
- Recipe + model → `workflow()` → `fit()` on training → `augment()`/`predict()` on testing → metrics.
- `step_normalize()` cannot fix collinearity; it only puts predictors on a common scale.
- A predicted probability is *not* an accuracy; a model has accuracy, one patient has a probability.

6.5 HW answers at a glance

HW1 — ML ideas; EDA on `mpg` (HW1; submission)

Q1 supervised = labeled Y to learn f; unsupervised = no Y, find structure (clusters). **Q2** regression: quantitative Y; classification: qualitative Y (class or class probabilities). **Q3** regression: RMSE, R^2 (MAE); classification: accuracy, sensitivity/recall (AUC). **Q4** descriptive model: visually emphasize a trend; inferential model: which features are significant, test theories/relationships; predictive model: best prediction of Y, minimal reducible error (S1.50). **Q5** mechanistic model = parametric model (assumed form; easier to understand; higher bias, lower variance); empirically-driven model = non-parametric model (few assumptions, more data, more flexible: lower bias, higher variance); both estimate f and both can overfit. (Correction to my submission: empirical models are *lower*-bias, not "unbiased".) **Q6** (a) chance a voter supports the candidate = **predictive**; (b) effect of personal contact = **inferential**. **EDA:** `hwy` histogram bimodal (peaks ≈16 and ≈26), slightly right-skewed; `hwy–cty` scatterplot strong positive linear ($r = 0.96$); bar plot: most cars dodge (37), fewest lincoln (3); boxplot: `hwy` falls with `cyl` (medians 29/24/17 for 4/6/8); `corrplot()` correlation matrix: `displ–cyl` 0.93, mpg vs. `displ`/`cyl` ≈ -0.8, `year` ≈ 0.

HW2 — abalone, linear regression vs. KNN (HW2; submission)

Q1 `age = rings + 1.5` (`mutate()`); mean 11.4, median 10.5, SD 3.22, range 2.5–30.5: unimodal, right-skewed, ≈6.7% boxplot outliers. **Q2** `initial_split()` (`prop = 0.85`, `strata = age`) → 3548 / 629. **Q3** recipe: `step_dummy()` → `step_interact()` (type×shucked, longest×diameter, shucked×shell) → `step_center()` → `step_scale()`; exclude `rings`: age is an exact function of it (target leakage). **Q4–6** `linear_reg()` `"lm"`, `nearest_neighbor()` (`neighbors = 7`) `"kknn"`, two workflows, fit on training. **Q7** hypothetical female (`predict()`): $\hat{y} = 21.8$ years. **Q8** `metric_set(rsq, rmse, mae)` + `augment()` on test: LM R^2 0.585, RMSE 1.95, MAE 1.44; KNN 0.483 / 2.21 / 1.62. $R^2 = 0.585$: 58.5% of the variance in test-set age explained. **Q9** LM wins on test; KNN training R^2 0.811 → test 0.483 = overfitting (flexible, high variance); LM 0.552 → 0.585 (rigid, stable). **Q10–12 (231)** reducible error $= \mathrm{Var} + \mathrm{Bias}^2$, irreducible error $= \mathrm{Var}(\epsilon)$; both nonnegative ⇒ test error $\geq \mathrm{Var}(\epsilon)$.

HW3 — Titanic classifiers (MY OWN answers; not submitted/graded) (HW3; raw HW section)

Self-worked solutions (R, seed 3435); exact numbers depend on the seed/split — know the reasoning. **Q1** 70/30, `strata = survived`: 623 / 268. missing data: `age` 128 (impute), `cabin` 481 (drop), `embarked` 2; IDs (`passenger_id`, `name`, `ticket`) not predictors. Stratify because the outcome is imbalanced (38.4% Yes) — both sets keep the same proportion. **Q2** sex is a strong predictor (survival 72.9% female vs. 20.3% male); class too (63.0 / 47.7 / 23.8% for 1st/2nd/3rd). Percent stacked bars (`geom_bar()`, `position = "fill"`) compare *proportions* when the bar totals differ. **Q3** `sib_sp–parch` $+0.43$, `age–sib_sp` -0.33, `fare` weakly $+$ with the family counts; all $|r| < 0.45$, no collinearity problem (needs `use = "pairwise.complete.obs"`). **Q4** `step_impute_linear(age, impute_with = imp_vars(...))` → `step_dummy(all_nominal_predictors())` → `step_interact(~ starts_with("sex"):fare + age:fare)`. **Q5–8** `logistic_reg()` `"glm"`, `discrim_linear()` / `discrim_quad()` `"MASS"` (`discrim`), `nearest_neighbor(neighbors = 10)` `"kknn"`; same recipe, four workflows. **Q9** `predict()` `type = "prob"` + `bind_cols()` → `roc_auc()`; training AUC: logistic 0.845, LDA 0.844, QDA 0.852, KNN 0.964 (inflated:

own neighbor). **Q10** testing AUC: logistic 0.880, **LDA 0.881**, QDA 0.859, KNN 0.879 $\Rightarrow$ LDA (near-tie). LDA test confusion matrix (`conf_mat()`) TP 73, FP 22, FN 30, TN 143: accuracy 0.806, sensitivity 0.709, specificity 0.867; `roc_curve()` + `autoplot()` for the plot. Train 0.844 vs. test 0.881: rigid low-variance model, the difference is split-to-split sampling noise (small test set); KNN's 0.964 $\rightarrow$ 0.879 is the overfitting pattern. **Q11–12 (231)** logit $=$ inverse logistic function; $x_1 + 2$ multiplies odds by $e^{2\beta_1}$ (odds ratio); $\beta_1 < 0$: $p \rightarrow 0$ as $x_1 \rightarrow \infty$, $p \rightarrow 1$ as $x_1 \rightarrow -\infty$.

PART II RAW TRANSCRIBED MATERIAL (only what Part I does not cover)

7 Slide Set 1 — Intro to ML, bias–variance tradeoff, KNN (pp. 1–58)

[In Part I: S1.1–5, 7, 10, 14, 25, 29, 33, 36–37, 39–44, 46–47, 49–50, 52–53, 55–58.]

7.1 Course overview and introduction

S1.6 **Required reading**

Undergrads: ch. 1 of ISLR; grads: ch. 1 of ESL.

S1.8 **A silly introduction**

[Figure: decision tree for a cat: "Am I supposed to scratch on it?" Yes → don't scratch; No → "Will my owners be mad?" Yes → scratch!, No → don't. Internal nodes = questions (splits), leaves = decisions (classes).]

S1.9 **[Speed vs. yams consumed]**

[Figure: toy scatterplot, Speed vs. Amount of Yams Consumed (me = low/slow, Usain Bolt = high/fast) with a fitted dashed regression line; a new x is read up to the line and across to give the predicted speed. Link: "Jamaica's sprinting glory: Is it the yams or the genes?"]

S1.11–13 **[Comics]**

[Figure: xkcd 1838: ML = pour data into a pile of linear algebra, collect answers, "stir the pile" until they look right (opaque trial-and-error). xkcd 1425: checking if a photo is in a national park = easy GIS lookup; checking if it is a bird = research team + 5 years. Meme (S1.13): crack labeled "statistics" → framed as "Machine Learning" → "Artificial intelligence" (ML/AI = rebranded statistics).]

S1.15–17 **[AI ⊃ ML ⊃ deep learning]**

[Figure: Nvidia timeline: AI (1950s, checkers) ⊃ ML (1980s, spam filtering) ⊃ deep learning (2010s, image recognition). Venn: AI (rule-based, expert systems) ⊃ ML (CART, SVM) ⊃ DL (neural networks).] [Figure: AI popularity vs. time: Birth 1956 → Inflated Hype 1974 → AI winter I 1980 → New Hopes 1987 → AI winter II 1993 → Explosive Growth.]

- 1950s–60s: first AI boom (reasoning, prototype AI); 1970s: AI winter I
- 1980s–90s: second boom (knowledge representation, expert systems reproducing human decisions); 1990s: AI winter II
- 1997 Deep Blue beats Kasparov; 2006 U. Toronto develops deep learning; 2011 IBM Watson wins Jeopardy; 2016 deep-learning Go software beats world champions

S1.18 **[How a child vs. a program learns "dog" (New Yorker)]**

Old program: thousands of if-then-else rules (ears, snout, hair, not a rat …). Child: sees labeled dogs, errs (wolf = dog), is corrected. ML, like the child, uses a classified training set (dog / not dog), extracts features, tests itself on thousands of labeled images, and builds its own recognizer.

S1.19–23 **[ML applications]**

Generative AI: Lensa, DALL·E, ChatGPT, Jukebox (music as raw audio). Lee Sedol vs. AlphaGo (2016). Google AI car first non-human to qualify as a driver. **Netflix Challenge**: 100M+ ratings, 400k+ users, 17k+ movies; predict user's star rating (1–5); prize $1,000,000. Kaggle competitions.

S1.24 **ML in research**

High-energy physics; gene-expression microarrays (20,000+ features); GWAS (millions of features). ⇒ unprecedented data scale + highly complicated structure.

S1.26–28 **[Data science workflow; tidymodels]**

[Figure: R4DS: Import → Tidy → [Transform → Visualize → Model loop = Understand] → Communicate. Package view: tidyverse (Import, Tidy, Transform, Visualize) → tidymodels (Model, loops back to Transform) → rmarkdown/Shiny (Share).] `tidymodels` = collection of packages for modeling/ML using `tidyverse` principles.

```
install.packages("tidymodels")
```

7.2 The learning set-up: response, predictors, f and error

S1.30–32 **Example data (observation level)**

$$\underset{\text{wage}}{\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_9 \end{pmatrix}} \quad \underset{\text{age, education, year}}{\begin{pmatrix} x_{11}, & x_{12}, & x_{13} \\ x_{21}, & x_{22}, & x_{23} \\ x_{31}, & x_{32}, & x_{33} \\ & \vdots & \\ x_{91}, & x_{92}, & x_{93} \end{pmatrix}}$$

Row 3 ("John") = one observation (one employee's wage, age, education, year). Column 2 = one predictor (all 9 employees' education). x_{ij} = obs. i, predictor j.

S1.34 **[Regression vs. ML cartoon]**

[Figure: same scatterplot+line, one plain ("REGRESSION"), one in top hat and monocle ("MACHINE LEARNING"): ML is often regression dressed up.]

S1.35 **Evaluate: How well did the machine learn?**

Need to give the model observed output and input. Learner/supervisor: college student/answer key; machine/raw observed output.

S1.38 **[Error of one test prediction]**

[Figure: yam/speed line with 4 test points: for test $\tilde{x}$, prediction $\hat{f}(\tilde{x})$ on the line vs. observed $\tilde{y}$; vertical gap = error $\tilde{y} - \hat{f}(\tilde{x})$ (test residual).]

S1.45 **[Map of methods]**

[Figure: SUPERVISED: continuous outcome (linear regression); categorical outcome (logistic regression, decision tree, naive Bayes, KNN); regularization (ridge, LASSO). UNSUPERVISED: clustering (DBSCAN, hierarchical, K-means, Gaussian mixtures/EM). Dimension reduction straddles both (LASSO; PCA).]

S1.48 **[ML mind map]**

[Figure: Unsupervised → dimensionality reduction (compression, structure discovery, big-data visualization, feature elicitation), clustering (recommenders, targeted marketing, customer segmentation). Supervised → classification (image classification, customer retention, identity fraud, diagnostics), regression (ad popularity, weather/market forecasting, life expectancy, population growth). Reinforcement → real-time decisions, game AI, robot navigation, skill acquisition, learning tasks.]

S1.51 **General comments**

Important: understand ideas behind methods; learn simpler methods first; accurately evaluate performance.

7.3 Bias–variance tradeoff: flexibility

S1.54 **[Model flexibility/complexity]**

[Figure: ISLR `Income` vs. Years of Education, Seniority (3-D). True f = smooth surface. Fits in order of increasing flexibility: flat plane (misses curvature, large errors); smooth surface $\approx f$; wiggly surface through every point (zero training error, far from f = overfitting).]

8 Slide Set 1 (cont.) — pp. 59–116

[In Part I: S1.59–60, 66, 68–80, 82, 85–96, 99–102.]

8.1 Training MSE vs. test MSE; overfitting

S1.61–62 **Lowest training MSE $\neq$ lowest test MSE**

*[Figure: see **F2**, p. 32]*

S1.63 **[Ex. 2: true f nearly linear]**

*[Figure: see **F3**, p. 32]*

S1.64 **[Ex. 3: true f highly nonlinear]**

[Figure: f: ≈27 at $X = 0$, ≈11 at 30, bump ≈13.8 at 65, plunges to ≈ -11 at 100. Linear line fits badly; both splines follow the data. Test/training MSE: 18.8/14.8 at flexibility 2, fall steeply between 3 and 6; 1.2/0.8 at ≈10; 1.5/0.4 at ≈25; test →2.5 at the end. Flexible method needed: linear fit has huge bias.]

S1.65 **Overfitting = small train MSE, large test MSE**

[Figure: Under / optimal / over-fitting. Regression (U-shaped data): straight line / parabola / zig-zag through every point. Classification: straight boundary (many errors) / smooth curve around red cluster / snaking boundary looping individual points. Deep learning, error vs. epochs: training and validation both level off high / both low, validation slightly above / training →0 while validation stays high (big gap).]

S1.67 **[Overfitting memes]**

[Figure: mattress bent to the shape of a body ("best way to explain overfitting"); "Not sure if 99% accuracy … or overfitting".]

8.2 Bias-variance decomposition

S1.81 **Bias-variance tradeoff (Ex. 1)**

*[Figure: see **F5**, p. 32]*

S1.83–84 **[Ex. 2 and 3 decompositions]**

[Figure: Ex. 2 (nearly linear): bias^2 tiny (≈0.05 at 2, then ≈0); variance ≈0.04→1.33; test MSE min ≈1.06 at ≈3 ⇒ optimum at low flexibility, variance dominates. Ex. 3 (highly nonlinear): MSE 18.8→1.2 between 2 and 6, →2.5 at end; bias^2 17.7→0 by ≈8; variance ≈0 until rising to ≈1.5 at right; min at ≈10 ⇒ bias dominates, optimum at higher flexibility.]

8.3 Classification: the Bayes classifier

S1.97–98 **[Bayes classifier, two classes]**

*[Figure: see **F7**, p. 32]* Each circle = an observation; location = predictor X; color = class label $Y \in \{1, 2\}$. Areas: $\Pr(Y = 1 \mid X = \mathbf{x}_0) > 50\%$ vs. $\Pr(Y = 2 \mid X = \mathbf{x}_0) > 50\%$.

9 Slide Set 2 — Linear models, feature engineering, EDA

[In Part I: S2.1, 3–5, 7–10, 12–14, 16–17, 20–24, 26–31, 33–36, 38–44, 46–49, 51–52, 54, 56–78.]

S2.2 **Required reading**

Undergrads: ch. 2 of ISLR; grads: ch. 2 of ESL.

S2.6 **[Missing-data patterns and mechanisms]**

[Figure: Patterns over $Y_1..Y_4$: (a) univariate*: only Y_4 has missing rows; (b)* monotone*: Y_3 missing bottom third, Y_4 bottom half (each later variable missing on a superset of rows); (c)* general*: scattered gaps everywhere; (d)* planned*: Y_2, Y_3, Y_4 missing in staggered blocks by design.] [Figure: Causal DAGs (McElreath), all with A→H, H→H*, D→H* (D = dog = missingness mechanism): MCAR: nothing → D; MAR: A→D; MNAR: H→D.]*

9.1 Linear regression

S2.11 **Linear regression**

PSTAT 126. Gauss, 1795. Simple, interpretable, often very useful; many nonlinear methods generalize linear regression. (1809): method of least squares, MLE, Gaussian distribution.

S2.15–18 **Simple linear regression; which line?**

[Figure: 9 counties, confirmed (50k–200k) vs. population (1M–3.3M): ≈(0.89M,73k), (0.98M,70k), (1.52M,70k), (1.66M,56k), (1.93M,75k), (2.15M,212k), (2.41M,200k), (3.17M,182k), (3.31M,171k); line $y = \beta_0 + \beta_1 x$ from ≈71k to ≈210k. Riverside (x ≈2.41M): observed ≈200k vs. predicted ≈157k; gap = residual.]

S2.19 **[Workflow]**

Data (training set $\{(x_i, y_i)\}_{i=1}^n$) → Learn $(\hat{\beta}_0, \hat{\beta}_1)$ → Evaluate (1. accuracy of $\hat{\beta}_0, \hat{\beta}_1$; 2. accuracy of the model).

S2.25 **Accuracy of the coefficient estimates**

*[Figure: see **F10**, p. 33]*

9.2 Multiple linear regression

S2.32 **Coefficient estimates**

ith residual: $y_i - \hat{y}_i = y_i - \hat{\beta}_0 - \sum_{j=1}^p \hat{\beta}_j x_{ij}$. *[Figure: ISLR 3.4: least-squares plane in (X_1, X_2, Y); vertical segments = residuals (points above and below).]*

S2.37 **[Stepwise selection, 5 variables]**

Forward: null → add most significant (X_2) → add X_5 → … until stopping rule or out of variables. Backward: full $\{X_1..X_5\}$ → remove least significant (X_4) → remove X_1 → $\{X_2, X_3, X_5\}$.

S2.45 **tidymodels: traditional dummy variables**

`recipes`: "`step_dummy()` creates a *specification* of a recipe step that will convert nominal data (e.g. character or factors) into one or more numeric binary model terms for the levels of the original data."

9.3 Interactions, polynomial regression

S2.50 **Interactions vs. main effects**

*[Figure: see **F11**, p. 33]*

S2.53 **Interaction: Credit example**

*[Figure: see **F12**, p. 33]*

S2.55 **Non-linear relationships**

*[Figure: see **F13**, p. 33]*

10 Slide Set 3 — Logistic regression & model evaluation (pp. 1–42, the tested part)

[Omitted (in Part I): S3.1–4, 6–7, 9–25, 28–29, 32–34, 37–38, 41–42.]

10.1 Classification: why logistic; Default data

S3.5 **Why not Linear Regression?**

*[Figure: see **F20**, p. 34]*

S3.8 **A data example**

*[Figure: see **F21**, p. 34]*

S3.26 **Estimating coefficients (MLE)**

```
glm(formula = default ~ student + balance + income,
    family = binomial, data = Default)
Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.4691  -0.1418  -0.0557  -0.0203   3.7383
Coefficients: ...  (table, deviances, AIC: see Part I)
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Dispersion parameter for binomial family taken to be 1)
Number of Fisher Scoring iterations: 8
```

10.2 Confounding

S3.27 **What is wrong?**

Simple logistic regression, **default** ∼ **student**:

	Coef.	Std. err.	Z-stat	P-value
`Intercept`	-3.5041	0.0707	-49.55	< 0.0001
`student[Yes]`	0.4049	0.1150	3.52	0.0004

Multiple logistic regression, **default** ∼ **balance** + **income** + **student**:

	Coef.	Std. err.	Z-stat	P-value
`Intercept`	-10.8690	0.4923	-22.08	< 0.0001
`balance`	0.0057	0.0002	24.74	< 0.0001
`income`	0.0030	0.0082	0.37	0.7115
`student[Yes]`	-0.6468	0.2362	-2.74	0.0062

S3.30–31 **[Simpson's paradox]**

[Figure: S3.30: blue points $(1, 6)$-$(4, 9)$ and red points $(8, 1)$-$(11, 4)$, each group on its own line of slope $+1$; a dashed line fitted to all 8 points slopes down *(slope ≈ -0.56, $y \approx 9.2 \to 0.8$ over $x \approx -1.5 \to 13.5$). S3.31 ("Korrelation:", animation frame): scatterplot of ∼500 points, Y 0–15 vs. X 0–10, in ∼5 clusters along a diagonal from upper left (X 1–5, Y 7–13) to lower right (X 9–13, Y 1–6): overall correlation negative, within each cluster positive.]*

10.3 Confusion matrix, ROC

S3.35 **Confusion matrices (multiclass)**

		True Class: Apple	Orange	Mango
Predicted	Apple	7	8	9
	Orange	1	2	3
	Mango	3	2	1

S3.36 **Many definitions and synonyms**

(Wikipedia "Receiver operating characteristic" table.) Row ratios: **FDR** = ΣFP/ΣPred. pos.; **FOR** = ΣFN/ΣPred. neg. Ratios of ratios: **LR+** = TPR/FPR, **LR−** = FNR/TNR; **DOR** = LR+/LR−.

S3.39–40 **ROC [curve examples]**

*[Figure: see **F23**, p. 34]*

11 Slide Set 3 (cont.) — Discriminant analysis: LDA & QDA (pp. 43–93; past the Slide-42 cutoff)

[Omitted (in Part I): S3.43–44, 46, 48–51, 53, 56, 58–60, 62, 65–74, 76–79, 81–84, 86–93. Past the Slide-42 cutoff; kept because PE Q20, Lab 3, HW 3 test LDA/QDA.]

11.1 Why LDA?

S3.45 **Why bother considering LDA?**

```
> x <- seq(100)
> y <- c(rep(0, 50), rep(1, 50))
> fit <- glm(y~x, family = "binomial")
```

S3.47 **[feature extraction vs feature selection]**

[Figure: Left, "feature extraction": original features transformed/combined into a few new *features (rows of 5 → 8 → 6 → 3 shapes). Right, "feature selection": row of 7 circles, 3 crossed out — a subset of the* original *features kept.][note: LDA = feature extraction — new axes LD1, LD2, … from combinations of predictors; S3.64–67, 80–85.]*

11.2 Geometric picture: gene-expression example

S3.52–57 **LDA: A motivating example**

[Figure: Number line "Transcription for Gene X" (fewer ←→ more transcripts), 12 dots left to right: G G G G R G R G R R R R (green = drug works, red = does not). `https://youtu.be/azXCzI57Yfc`*]*

- S3.54: Mostly, drug works with low Gene X transcription (4 leftmost green circled), not with high (4 rightmost red).
- S3.55: But overlap (middle R G R G) and no obvious "cutoff" for who gets the drug.
- S3.57: Gene X does an OK job. Can we do better with more than one gene?

S3.61–64 **LDA: A motivating example**

- S3.61: 2D scatterplot Gene X vs. Gene Y: 6 green mostly lower-left, 6 red along top/upper-right (+1 red lower right); a diagonal dashed line (linear decision boundary) separates them. Two genes separate better. Three?
- S3.63: With 3 genes (X, Y, Z) we use a plane to separate the categories.
- S3.64: Reducing 2-D graph → 1-D graph (number line): G G G G G R R G R R R R.

S3.75 **[distance between means alone vs. means and scatter]**

[Figure: 2D: 6 red in an upper band, 6 green in a lower band, both rising gently right. Projected onto a line along the bands: means far apart but classes interleave in the middle. Projected onto a near-vertical line: red tight at top, green tight at bottom, no overlap.] Maximizing only the distance between means → poor separation; optimizing distance between means *and* scatter → nice separation.

S3.80 **[LDA with three genes]**

[Figure: 3D Gene X/Y/Z cloud with a diagonal line = the new axis LDA created; data projected onto it: 6 green lower-left, 6 red upper-right, completely separated.]

S3.85 **[LDA plot: LD1 vs LD2]**

[Figure: R plot "LDA", LD1 (-4 to 2) vs. LD2 (-2 to 2), three classes: red at left (LD1 -4.2 to -0.8, LD2 -1 to 1); black upper right (LD1 0–2, LD2 -0.7 to 2.6); blue lower right (LD1 -0.2 to 2.2, LD2 -2.7 to 0), a few blue among black. LD1 separates red from the rest; LD2 separates black from blue.]

12 Reading — ROC Curves Deep Dive

[HTML title "ROC Curves: An Overview", PSTAT 131/231. Omitted (in Part I): data loading, factor, `coin_flip`*, 70–30 stratified split, Recipe B, workflows, training accuracy (0.807 / 0.616), test re-fit, confusion-matrix heatmaps (A at 0.5 and 0.00131; B at 0.5 and 0.4076525) with TPR/FPR arithmetic and points, TPR/FPR definitions, Model B AUC (0.496) and accuracy,* `roc_curve()` *needs probabilities, AUC = discrimination, two uses of ROC, AUC* $= 1 \Rightarrow$ *leakage.]*

12.1 Recipe A

```
titanic_recipe_a <- recipe(survived ~ pclass + sex + age +
                           sib_sp + parch + fare, titanic_train)
    ↪ %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp)) %>%
  # choice of predictors to impute with is up to you
  step_dummy(all_nominal_predictors()) %>%
  step_interact(~ starts_with("sex"):age + age:fare)
```

12.2 Model A: ROC curve

[Figure: see ***F25****, p. 35]*

```
# A tibble: 254 x 3
   .threshold specificity sensitivity
 1 -Inf           0               1
 2    0.00131     0               1
 3    0.00474     0.0121          1
 4    0.00563     0.0182          1
 5    0.00635     0.0242          1
 6    0.0116      0.0303          1
 7    0.0117      0.0364          1
 8    0.0122      0.0424          1
 9    0.0147      0.0485          1
10    0.0186      0.0545          1
# ... with 244 more rows
```

By default `roc_curve()` tries `threshold` values in a grid from `-Inf` (≈ 0) to `Inf` (≈ 1). First predicted class probabilities:

```
log_a_results %>% select(.pred_Yes, survived) %>% head()
```

```
  .pred_Yes survived
1   0.0672  No
2   0.268   No
3   0.792   Yes
4   0.00474 No
5   0.247   Yes
6   0.693   Yes
```

Default threshold for binary classification is generally 0.50: No if $p < .50$, else Yes:

```
augment(log_test_a, new_data = titanic_test) %>%
  mutate(.pred_class = factor(if_else(.pred_Yes < 0.5, "No", "Yes"
    ↪ ), levels = c("Yes", "No"))) %>%
  conf_mat(truth = survived, estimate = .pred_class) %>%
  autoplot(type = "heatmap")
# that point on the ROC curve:
log_a_results %>% roc_curve(survived, .pred_Yes) %>% autoplot() +
  geom_point(aes(x = 0.1212121, y = 0.776699), colour = spec_
    ↪ colors[3], size = 2.5)
```

Same for 0.25, 0.75, … any value in $[0, 1]$. Goal: maximize sensitivity (TPR) while minimizing 1 − specificity (FPR).

```
log_a_results %>%
  ggplot(aes(x = .pred_Yes, fill = survived)) +
  geom_histogram(position = "dodge") +
  theme_bw() + xlab("Probability of Yes") +
  scale_fill_manual(values = c("blue", "orange"))
# same plot with geom_boxplot() in place of geom_histogram(
    ↪ position = "dodge")
```

[Figure: see ***F27****, p. 35] [Figure: Model A horizontal boxplots (y-axis -0.2–0.2 meaningless: no y aesthetic). No: box 0.08–0.25, median 0.12, whiskers 0–0.49, outliers 0.51–0.81. Yes: box 0.54–0.93, median 0.79, whiskers 0.04–1.0. Boxes do not overlap.]*

12.3 Model B: ROC curve

For poorly predicting models the threshold choice barely matters — they NEVER reach TPR 1 with FPR 0 (a "perfect" classifier can). *[Figure: see* ***F26****, p. 35]*

```
# A tibble: 9 x 3
  .threshold specificity sensitivity
1   -Inf           0             1
2      0.113       0             1
3      0.193       0.0182        1
4      0.228       0.0364        1
5      0.267       0.0545        1
6      0.311       0.0788        1
7      0.358       0.0909        0.951
8      0.408       0.285         0.689
9    Inf           1             0
```

[Figure: Model B histogram of `.pred_Yes` *(x 0.1–0.42, y 0–120): few distinct values (one per* `sib_sp`*): 0.113, 0.193, 0.228: No 3 each; 0.267: No 4; 0.311: Yes 5, No 2; 0.358: Yes 27, No 32; 0.408: Yes 71, No 118. Classes at the same values — no separation; nothing above 0.41.]* Thresholds = grid over the observed predicted probabilities. No predicted P(Yes) > 0.4076525 = the largest threshold in its ROC curve; larger thresholds would give the same results as `Inf`. boxplot shows the overlap: *[Figure: Model B boxplots: No and Yes essentially identical: box 0.358–0.408, median at upper edge (0.408), left whisker 0.31; No also has outliers at 0.113, 0.193, 0.228, 0.267. Complete overlap.]*

12.4 Conclusions

ROC curve can also pick the optimal threshold(s) for a chosen model — less common, since defaults usually balance sensitivity and specificity well; but to be extra careful to predict **every** 0 (or every 1) correctly, shift the threshold that way. Multiclass outcome (> 2 values): one curve per class for how well the model distinguishes **each** class (examples in homework/labs).

13 Lab 1 — R basics, dplyr, EDA (diamonds), data splitting

[Omitted (in Part I): package loading, help, pipe/operators, `select()`*/*`mutate()`*/*`transmute()`*/*`ungroup()` *code+output, corrplot/boxplot/histogram code, seed + split code.]*

13.1 Basics of Data Processing

Tools: *tidyverse*; more in *R for Data Science* (`https://r4ds.had.co.nz/`). Packages: `tidyverse`, `tidymodels`, `ISLR` (`install.packages()`, then `library()`). Some packages ship datasets: `library(ISLR)`, then `mpg` (chunk `eval = FALSE`, no output); `?mpg` for info.

`filter()` — rows by value. Audi compact cars:

```
mpg %>%
  filter(class == "compact" & manufacturer == "audi")
```

```
# A tibble: 15 x 11  (all: manufacturer audi, fl p,
#   class compact; trans auto/manual truncated)
   model       displ year cyl trans drv cty hwy
 1 a4            1.8 1999   4 auto  f    18  29
 2 a4            1.8 1999   4 manu  f    21  29
 3 a4            2   2008   4 manu  f    20  31
 4 a4            2   2008   4 auto  f    21  30
 5 a4            2.8 1999   6 auto  f    16  26
 6 a4            2.8 1999   6 manu  f    18  26
 7 a4            3.1 2008   6 auto  f    18  27
 8 a4 quattro    1.8 1999   4 manu  4    18  26
 9 a4 quattro    1.8 1999   4 auto  4    16  25
10 a4 quattro    2   2008   4 manu  4    20  28
11 a4 quattro    2   2008   4 auto  4    19  27
12 a4 quattro    2.8 1999   6 auto  4    15  25
13 a4 quattro    2.8 1999   6 manu  4    17  25
14 a4 quattro    3.1 2008   6 auto  4    17  25
15 a4 quattro    3.1 2008   6 manu  4    15  25
```

Activities (`flights`, `nycflights13`) — filter flights that: arrival delay ≥ 2 h; flew to Houston (IAH or HOU); operated by United, American, or Delta; departed in summer (Jul–Sep); arrived >2 h late but didn't leave late; delayed ≥ 1 h but made up >30 min in flight; departed midnight–6am (inclusive).

`select()` / `mutate()` — activities. On `flights`: find many ways to select `dep_time`, `dep_delay`, `arr_time`, `arr_delay`; what if a variable is named multiple times in `select()`? `mutate()` adds columns that are functions of existing ones (useful functions: `https://r4ds.had.co.nz/transform.html#mutate-funs`). Convert `dep_time`, `sched_dep_time` (not really continuous) to minutes since midnight. What does `1:3 + 1:10` return, and why?

`summarise()` — grouped summaries. `summarise()` is most useful with `group_by()` (one summary per group). Without grouping:

```
mpg %>%
  summarise(avg_hwy = mean(hwy))
```

```
  avg_hwy
1    23.4
```

= average over *all cars*; not very useful. By class / manufacturer, with `arrange()`:

```
mpg %>%
  group_by(class) %>%
  summarise(avg_hwy = mean(hwy)) %>%
  arrange(avg_hwy)
# same with group_by(manufacturer)
```

```
  class      avg_hwy   manufacturer avg_hwy
1 pickup        16.9   land rover      16.5
2 suv           18.1   lincoln         17
3 minivan       22.4   jeep            17.6
4 2seater       24.8   dodge           17.9
5 midsize       27.3   mercury         18
6 subcompact    28.1   ford            19.4
7 compact       28.3   chevrolet       21.9
                       nissan          24.6
                       toyota          24.9
                       subaru          25.6
                       pontiac         26.4
                       audi            26.4
                       hyundai         26.9
                       volkswagen      29.2
                       honda           32.6
```

Avg hwy + count by manufacturer, the 10 with most cars (≥ 9), by mileage:

```
mpg %>%
  group_by(manufacturer) %>%
  summarise(avg_hwy = mean(hwy),
            count = n()) %>%
  filter(count >= 9) %>%
  arrange(avg_hwy)
```

```
  manufacturer avg_hwy count
1 dodge            17.9    37
2 ford             19.4    25
3 chevrolet        21.9    19
4 nissan           24.6    13
5 toyota           24.9    34
6 subaru           25.6    14
7 audi             26.4    18
8 hyundai          26.9    14
9 volkswagen       29.2    27
10 honda           32.6     9
```

13.2 Exploratory Data Analysis

exploratory data analysis on `diamonds` (from `ggplot2`):

```
diamonds %>%
  head()
```

```
# A tibble: 6 x 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

Predict `price`: not directly measurable, and we care what a diamond is worth. **Activities:** how many observations / variables in `diamonds`? How many are features usable to predict `price`? (`?diamonds`.)

correlation matrix of continuous variables vs `price`: *[Figure: `corrplot`, lower triangle, -1 dark red to $+1$ navy. Besides the strong $+$ carat/price/x/y/z block: depth–table ≈ -0.3 (only clearly negative); depth with carat, price, x, y ≈ 0; table with carat, price, x, y, z and depth with z weak $+$ ($\approx$0.1–0.2).]* Look at the `corrplot()` arguments; what if `diag = TRUE`, `method = 'square'`? **Activities:** which features are positively correlated with `price` (sensible?); any negative? which correlate with *each other*, and why?

boxplot of `price` per `cut` and `color`:*[Figure: Horizontal boxplots of price by cut (`reorder(cut, price)`, bottom to top: Ideal, Good, Very Good, Fair, Premium), 7 boxes per cut filled by color (D … J). All right-skewed (Q1 $\approx$800–2,500, outliers to $\approx$18,800; Fair fewest outliers). In every cut I, J have higher medians/wider boxes than D/E: Premium J median $\approx$5,000 (Q3 $\approx$9,000); Ideal D/E medians $\approx$1,500–1,800 (Q3 $\approx$3,000). Ideal (best cut) cheapest overall; Premium priciest.]* **Activities:** what does this say about `price`, `cut`, `color`? Anything surprising given `?diamonds`? Why do `J` (worst color) diamonds cost more? Check `color` vs `carat` (`carat` highly $+$ correlated with `price`):

```
diamonds %>%
  ggplot(aes(x = carat, y = reorder(color, carat))) +
  geom_boxplot() +
  theme_bw() +
  labs(x = "Carat", y = "Color")
```

[Figure: Carat (0–5) by color, ordered by median: D $\approx$0.53, E $\approx$0.53, F $\approx$0.70, G $\approx$0.70, H $\approx$0.90, I $\approx$1.0, J $\approx$1.1; Q3 $\approx$0.9 (D, E) to $\approx$1.5 (I, J). Many right outliers (D–G up to $\approx$3); largest J (5.01), I ($\approx$4.0), H one at $\approx$4.1. Worse colors = bigger diamonds.] **Activity:** explain why lower-quality colors cost more.

histogram of outcome `price`:*[Figure: 60 bins (more bins = finer picture). Peak $\approx$8,200 diamonds at price $\approx$700, next bin $\approx$7,000; drops to $\approx$3,200 and $\approx$2,300–2,600 around 1,500–2,000; small bump $\approx$1,600 near 4,500; long right tail, $\approx$100–300 per bin from 10,000 to max $\approx$18,800.]* **Activity:** one plot showing `cut`, `carat`, and `price`.

13.3 Data Splitting

Split `diamonds` into training and test sets (later: resampling / cross-validation; for now use the whole training set). `tidymodels` is easier than the textbooks' base-R way. Seed: any number. **Activities:** exact number of observations in training and testing sets? What does `strata = price` do? Guess, then check `?initial_split`.

14 Lab 2 — Model fitting with tidymodels: linear regression, KNN, missing data

[Omitted (in Part I): packages, split; recipe/`step_dummy()`, `prep()`/`bake()`; `lm` spec/workflow/fit; `bind_cols()`, pred-vs-actual code; RMSE values, test metrics (1146, 0.918, 751); KNN engine/mode + spec; `extract_fit_parsnip()` (best k 5); `vis_miss()` %s, NA/`drop_na()` (153 → 111), imputation choices, linear-imputation recipe, missing-data rules.]

14.1 Recipe

First step of any `tidymodels` fit: a recipe; `lm()`-like formula, outcome first, then features (`.` = all other variables):

```
simple_diamonds_recipe <-
  recipe(price ~ ., data = diamonds_train)
simple_diamonds_recipe
```

```
##       role #variables
##    outcome          1
##  predictor          9
```

(9 predictors.) Display baked data neatly with `kbl()`/`kable_styling()` (`kableExtra`, optional), `head()` for first rows, `scroll_box()` to scroll:

```
prep(diamonds_recipe) %>%
```

```
  bake(new_data = diamonds_train) %>%
  head() %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

[24 columns, split in 3 blocks of the same 6 rows; rounded to 4 places; `clar_k = clarity_k.` *`cut`, `color`, `clarity` are* ordered *factors* ⇒ `step_dummy()` *gives orthogonal-polynomial contrasts (*`cut_1..._4`, `color_1..._6`, `clarity_1..._7`*), not 0/1.]*

carat	depth	table	x	y	z	price	cut_1	cut_2
0.23	61.5	55	3.95	3.98	2.43	326	0.6325	0.5345
0.21	59.8	61	3.89	3.84	2.31	326	0.3162	-0.2673
0.23	56.9	65	4.05	4.07	2.31	327	-0.3162	-0.2673
0.29	62.4	58	4.20	4.23	2.63	334	0.3162	-0.2673
0.31	63.3	58	4.34	4.35	2.75	335	-0.3162	-0.2673
0.24	62.8	57	3.94	3.96	2.48	336	0.0000	-0.5345

cut_3	cut_4	color_1	color_2	color_3	color_4	color_5
0.3162	0.1195	-0.3780	0.0000	0.4082	-0.5641	0.4364
-0.6325	-0.4781	-0.3780	0.0000	0.4082	-0.5641	0.4364
0.6325	-0.4781	-0.3780	0.0000	0.4082	-0.5641	0.4364
-0.6325	-0.4781	0.3780	0.0000	-0.4082	-0.5641	-0.4364
0.6325	-0.4781	0.5669	0.5455	0.4082	0.2417	0.1091
0.0000	0.7171	0.5669	0.5455	0.4082	0.2417	0.1091

color_6	clar_1	clar_2	clar_3	clar_4	clar_5	clar_6	clar_7
-0.1974	-.3858	.0772	.3077	-.5238	.4922	-.3077	.1195
-0.1974	-.2315	-.2315	.4308	-.1209	-.3638	.5539	-.3585
-0.1974	.0772	-.3858	-.1846	.3626	.3210	-.3077	-.5974
-0.1974	-.0772	-.3858	.1846	.3626	-.3210	-.3077	.5974
0.0329	-.3858	.0772	.3077	-.5238	.4922	-.3077	.1195
0.0329	.2315	-.2315	-.4308	-.1209	.3638	.5539	.3585

Activity: name and describe three `step` functions not used here.

14.2 Linear Regression

```
lm_fit %>%
  extract_fit_parsnip() %>%  # parsnip object
  tidy()                     # tidy lm object
```

```
## # A tibble: 24 x 5
##    term         estimate std.error statistic   p.value
##  3 depth          -58.4       6.25     -9.33  1.07e- 20
##  4 table          -24.1       3.23     -7.45  9.40e- 14
##  7 z             -127.       73.2      -1.74  8.23e-  2
##  9 cut_2         -300.       19.9     -15.0   4.77e- 51
## 10 cut_3          153.       17.2       8.90  5.99e- 19
## # ... with 14 more rows (rows 1-2, 5-6, 8 in Part I)
```

Activities: what does the intercept represent? Effect of `carat`: significant? Effect on `price` of +1 `carat`, all else fixed?

Training-set predictions:

```
diamond_train_res <- predict(lm_fit, new_data = diamonds_train %>%
    ↪  select(-price))
diamond_train_res %>% head()
```

```
##    .pred
## 1 -1363.
## 2  -667.
## 3   199.
## 4  -815.
## 5 -3472.
## 6 -1373.
```

Predicted vs. actual (training): *[Figure: see **F19**, p. 34]* Poor fit: perfect predictions would lie on the line; negative price predictions; poor once actual price $>\approx \$5{,}000$. Metric set (RMSE, R^2, MAE); same call for each model, train and test:

```
diamond_metrics <- metric_set(rmse, rsq, mae)
diamond_metrics(diamond_train_res, truth = price,
                estimate = .pred)
```

```
## model       rmse   rsq    mae
## LR train   1126.  0.920  736.
## LR test    1146.  0.918  751.  (Part I)
## KNN5 train  386.  0.991  201.
## KNN5 test   711.  0.968  373.
## KNN2 train  152.  0.999   74.6
## KNN2 test   851.  0.955  421.
```

[one 3-row tibble (`.metric .estimator=standard .estimate`) per row, combined here; data: `diamond_train_res`, `diamond_{train,test}_knn`, _knn2.*]***Activities:** do train and test metrics differ? Explain why (or why not).

14.3 *k*-Nearest Neighbors

Same recipe, KNN model: `nearest_neighbor()` instead of `linear_reg()` (https://parsnip.tidymodels.org/reference/nearest_neighbor.html); only engine: kknn. *[Figure: Allison Horst cartoon "parsnip::set_engine()!": race cars STAN, keras, GLMNET, lm — parsnip = "a standardized interface for fitting models and making predictions."]*Unlike LR, needs a *hyperparameter* k (number of neighbors). Optimal k unknown; later resampling, cross-validation, tuning; for now default k. **Activity:** what is the default k, and how did you find out?

```
knn_wflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(simple_diamonds_recipe)
knn_fit <- fit(knn_wflow, diamonds_train)
```

Train/test metrics: KNN5 columns above. Different *k*: set it in `nearest_neighbor()` (later the tuned parameter goes here too); any *k* from 1 to *n*:

```
knn_model2 <- nearest_neighbor(neighbors = 2) %>%
  set_engine("kknn") %>%
  set_mode("regression")
knn_wflow2 <- workflow() %>%
  add_model(knn_model2) %>%
  add_recipe(diamonds_recipe)
knn_fit2 <- fit(knn_wflow2, diamonds_train)
# predict()/bind_cols() as in Part I; metrics: KNN2 columns above
```

Activities: which model (LR, KNN $k = 5$, KNN $k = 2$) is best on test data, and why?What explains the KNN train–test gap? How many predictors are in the KNN model(s)? Would changing that number help?

14.4 Missing Data

missing data with `airquality` (base R; 153 obs, 6 variables; `?airquality`) and `naniar` (https://cran.r-project.org/web/packages/naniar/vignettes/getting-started-w-naniar.html; loaded here as an exception to load-all-at-top; uncomment `install.packages()` if needed). *[Figure: see **F18**, p. 34]*Also visible in the data:

```
airquality %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6
23	299	8.6	65	5	7
19	99	13.8	59	5	8
8	19	20.1	61	5	9
NA	194	8.6	69	5	10
7	NA	6.9	74	5	11
16	256	9.7	69	5	12

```
# ... (141 more rows; e.g. rows 25-27, 32-37 Ozone NA)
```

Predict ozone (air quality); histogram:

```
ggplot(aes(x = Ozone), data = airquality) +
  geom_histogram()
```

[Figure: 30 bins, x 0–170, count 0–17. Strongly right-skewed: peak ≈17 at Ozone ≈10–15, 13 at ≈15–25, 10 at ≈35, 8 at ≈30 and ≈45, then 1–6 per bin from 50–125, isolated bars (1) at ≈130–135 and ≈165–170.] One `NA` ⇒ `mean()` returns `NA`; drop them:

```
airquality %>%
  summarise(mean(Ozone, na.rm = T))
```

```
## 1               42.12931
```

Only two variables missing ⇒ linear imputation in the recipe:

```
airquality_split <- initial_split(airquality, prop = 0.80,
                                  strata = Ozone)
airquality_train <- training(airquality_split)
airquality_test <- testing(airquality_split)
# air_recipe (2 x step_impute_linear()) in Part I
prep(air_recipe) %>%
  bake(new_data = airquality_train) %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

`prep()`/`bake()` verify: missing values imputed (predicted) from the complete variables.

Solar.R	Wind	Temp	Month	Day	Ozone	
149	12.6	74	5	3	12	
313	11.5	62	5	4	18	
19	20.1	61	5	9	8	
206	6.9	74	5	11	7	# Solar.R imputed
256	9.7	69	5	12	16	
290	9.2	66	5	13	11	

```
# ... (121 rows; Ozone now last column) ...
```

162	14.3	56	5	5	-11	# both imputed
194	8.6	69	5	10	37	# Ozone imputed

```
     66 16.6    57      5   25    -12   # Ozone imputed
    286  8.6    78      6    1     47   # Ozone imputed
    266 14.9    58      5   26     -3   # Ozone imputed
    128  8.0    57      5   27     22   # both imputed
    269  4.1    84      7    1    135
```

*["imputed" comments added by comparison with raw data. Linear imputation gave impossible negative Ozone ($-11, -12, -3$).]***Activities:** fit a linear regression on the imputed data; compare with one using mean imputation for both variables.

Resources: *Tidy Modeling with R* (`https://www.tmwr.org/`); ISLR labs in `tidymodels` (`https://emilhvitfeldt.github.io/ISLR-tidymodels-labs/index.html`).

15 Lab 3 — Intro to binary classification (logistic, LDA, QDA, naive Bayes)

[In Part I: package loading (tidymodels, discrim, corrr, corrplot, klaR); correlation code; Direction/Purchase bar plots + counts (602/648, 653/417); 70/30 stratified splits; `oj_recipe`, model specs, workflows; `conf_mat()`/`accuracy()` code, 8 training accuracies; comparison tables; training-accuracy pitfall; Smarket QDA + OJ logistic test results.]

Packages. Also loaded: `ISLR`, `ISLR2` (data), `discrim` (LDA, QDA, naive Bayes), `poissonreg` (Poisson regression), `corrr` (correlation matrices).

```
library(ISLR) # Smarket
library(ISLR2) # Bikeshare
library(poissonreg)
```

Data. `Smarket`: daily % returns of S&P 500, 2001–2005; 1,250 obs, 8 numeric vars + factor `Direction` ("Up"/"Down") = outcome; predictors `Volume` (shares traded) + `Lag1`–`Lag5` (previous days' % returns). `OJ` (Orange Juice): 1,070 purchases of Citrus Hill or Minute Maid; outcome `Purchase` = brand; predictors: prices of each brand, discounts, customer-loyalty metric.

Activities: Help page of `Smarket`: what does `Direction` represent, which predictors might correlate with it? Read help page of `OJ`.

`correlate()` (`corrr`) computes the correlation matrix (drop non-numeric `Direction`); `rplot()` plots it. *[Figure: `rplot()`, Smarket (Year, Lag1–5, Volume, Today; red -1, blue $+1$): all $\approx$0 except Year–Volume $r \approx 0.54$.]*

Activities: What do you notice? Need to worry about highly linearly correlated predictors? Does it say anything about predictor–outcome relationships?

Heatmap alternative with `ggplot` + `geom_tile()`:

```
cor_Smarket %>%
  stretch() %>%
  ggplot(aes(x, y, fill = r)) +
  geom_tile() +
  geom_text(aes(label = as.character(fashion(r))))
```

[Figure: Tiles labeled with r; diagonal NA (gray); all in $[-.05, .04]$ (e.g. Lag1–Lag2 $-.03$, Volume–Lag4 $-.05$, Year–Lag4 $.04$, Volume–Today $.01$) except Year–Volume $.54$.] Upper and lower triangles identical (always true of a correlation matrix); only `Year`–`Volume` have notable correlation (≈ 0.54). boxplot of `Volume` by `Year`:

```
ggplot(Smarket, aes(factor(Year), Volume)) +
  geom_boxplot() +
  geom_jitter(alpha = 0.1) +
  xlab("Year")
```

[Figure: Volume (0.4–3.2) by Year: medians 2001 $\approx$1.2, 2002 $\approx$1.38, 2003 $\approx$1.40, 2004 $\approx$1.42, 2005 $\approx$1.90 (box 1.65–2.15, max $\approx$3.15); 2001–02 many high outliers (1.8–2.8). Volume rises with Year, esp. 2005.] *[Figure: OJ `corrplot` (`diag = F, type = 'lower'`) of PriceCH, PriceMM, DiscCH, DiscMM, LoyalCH, PctDiscMM, PctDiscCH: PctDiscMM–DiscMM and PctDiscCH–DiscCH $r \approx 1$; PriceMM–PriceCH $\approx$0.6–0.7; $\approx$0.1–0.2 for PriceCH with DiscCH, DiscMM, LoyalCH, PctDiscMM, PctDiscCH, for LoyalCH with PriceMM, DiscCH, PctDiscCH; rest $\approx$0 (a few faintly negative).]* `PriceMM`, `PriceCH` positively correlated: when OJ is cheaper to make, both brands lower prices.

Activities: Describe the relationship between `Year` and `Volume`.

OJ is unbalanced: 653 Citrus Hill vs. 417 Minute Maid; not adjusted here (**upsampling**/**downsampling** next week). Far from extreme — some outcomes have very rare levels.

15.1 Logistic Regression

Recipe as in the linear regression lab; no categorical predictors ⇒ no `step_dummy`.

```
smarket_recipe <- recipe(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 +
    ↪
                           Lag5 + Volume, data = smarket_train)
smarketlog_fit %>% tidy()
```

```
  term         estimate std.error statistic p.value
1 (Intercept) -0.197      0.293    -0.672    0.502
2 Lag1        -0.00479    0.0601   -0.0797   0.937
3 Lag2        -0.0537     0.0619   -0.868    0.386
4 Lag3         0.00265    0.0611    0.0435   0.965
5 Lag4        -0.0440     0.0594   -0.740    0.459
6 Lag5         0.0381     0.0605    0.630    0.529
7 Volume       0.184      0.193     0.952    0.341
```

```
ojlog_fit %>% tidy()
```

```
  term         estimate std.error statistic  p.value
2 PriceCH         3.68      1.49      2.48  1.31e- 2
3 PriceMM        -4.17      0.980    -4.26  2.03e- 5
4 DiscCH        -10.0      23.2      -0.431 6.66e- 1
5 DiscMM         27.1      10.1       2.69  7.20e- 3
7 PctDiscMM     -51.8      21.0      -2.47  1.36e- 2
8 PctDiscCH       8.43     43.5       0.194 8.46e- 1
```

[rows 1, 6 (Intercept 4.47, LoyalCH -6.07) in Part I.]

Assessing Model Performance. Training-set probability predictions:

```
predict(smarketlog_fit, new_data = smarket_train, type = "prob")
```

```
# A tibble: 874 x 2
   .pred_Down .pred_Up
 1      0.521    0.479
 2      0.470    0.530
 3      0.498    0.502
 4      0.498    0.502
 5      0.467    0.533
 6      0.518    0.482
 7      0.468    0.532
 8      0.478    0.522
 9      0.517    0.483
10      0.521    0.479
# ... with 864 more rows
```

Each row = predicted probability of each class; redundant for 2 classes (one = 1 − other) but useful for multiclass. Text: row 1 P(Down) ≈0.56, P(Up) ≈0.44 *[earlier knit; output shows 0.521/0.479]*. None of first 10 below 0.40 or above 0.59 → model cannot discriminate the classes well. Compare `OJ`:

```
predict(ojlog_fit, new_data = oj_train, type = "prob")
```

```
# A tibble: 748 x 2
   .pred_CH .pred_MM
 3    0.970  0.0300
 4    0.958  0.0424
 5    0.951  0.0494
 6    0.952  0.0484
 7    0.994  0.00625
 8    0.840  0.160
 9    0.846  0.154
10    0.795  0.205
# ... with 738 more rows
```

Summarize: `augment()` attaches predictions → confusion matrix. Smarket (left), OJ (right; $623/748 = 0.833$ correct):

```
          Truth                       Truth
Prediction Down  Up         Prediction  CH  MM
      Down  104  98                  CH 403  71
      Up    317 355                  MM  54 220
```

Activities: What do you notice from each confusion matrix? For each outcome, do the models overpredict one class; which? Are the models doing well? Why/why not?

15.2 LDA, QDA, Naive Bayes — training confusion matrices

Same procedure: confusion matrix (lab says "confidence matrix", sic) and accuracy on **training data**. Rows = Prediction, cols = Truth.

```
              Smarket          OJ
            Down  Up        CH  MM
LDA   Down   103  98    CH 398  70
      Up     318 355    MM  59 221
QDA   Down   125  99    CH 370  65
      Up     296 354    MM  87 226
NB    Down   107  98    CH 338  47
      Up     314 355    MM 119 244
```

Activities: Compare LDA with logistic regression. Which model performed best on training data for each data set? Which would you choose, why?

15.3 Fitting to Testing Data

*[Figure: see **F29, p. 35**]* Logistic regression fit to `OJ` test set (best on training, same caveats); code (`roc_curve(Purchase, .pred_CH)`) and metrics in Part I.*[Figure: see **F28, p. 35**]*

Activities: Interpret sensitivity and specificity in terms of the data. How well did the models do on **testing** data, and why? Which was better?

Resources: *Tidy Modeling with R* (free). Source: "ISLR Tidymodels Labs" by Emil Hvitfeldt.

16 Homework 1–3: prompts, my submissions, solutions

[Omitted (in Part I: d5 "HW answers at a glance", d2/d2b pipelines, d4 templates): HW1 setup, Q1–Q6, most plot code; HW2 setup, Q5, Q7, Q10–Q12, most recipe/work-

flow/metric code + tables; HW3 setup, Q7, Q11–Q12, model specs/workflows, AUC tables, confusion matrix + metrics.]

*Key: **Prompt.** = assignment (condensed); **My answer.** = my graded submission (HW1, HW2); **Answer.** = correction/note (HW1, HW2) or my own solution, run in R 4.5.3 (HW3, unsubmitted).*

16.1 Homework 1 – ML main ideas; EDA on mpg

HW1 Ex1

Prompt. histogram of `hwy` (highway mpg); describe.
My answer. *[Figure: binwidth 2, x = Highway MPG, y = Count. Counts (centers 12, 14, …, 44): 5, 12, 38, 23, 13, 14, 28, 46, 29, 11, 6, 3, 3, 0, 1, 0, 2.]* Two peaks, ≈16–18 and 24–28 mpg; median 24, mean ≈23.4. Most cars 15–30 mpg; a few outliers > 40.

HW1 Ex2

Prompt. scatterplot, `hwy` on x, `cty` on y. Relationship? Meaning?
My answer. *[Figure: tight straight upward band from (12, 9) to (44, 35); integer grid, so only 78 distinct points for 234 cars.]* City MPG consistently $<$ highway; fitted slope ≈0.7 (+1 hwy mpg → +0.7 cty mpg). Fewer than 234 dots: rounding to whole numbers → overlap.

HW1 Ex3

Prompt. Horizontal bar plot of `manufacturer`, ordered by height. Most / fewest cars?
My answer.

```
mpg %>% count(manufacturer, sort = TRUE)
```

```
dodge 37, toyota 34, volkswagen 27, ford 25, chevrolet 19,
audi 18, hyundai 14, subaru 14, nissan 13, honda 9, jeep 8,
pontiac 5, land rover 4, mercury 4, lincoln 3   # 15 rows
```

HW1 Ex4

Prompt. boxplot of `hwy` by `cyl`, add points with `geom_jitter()` + `alpha`. Relationship?
My answer.

```
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(alpha = 0.3, width = 0.2) +
  labs(x = "Number of Cylinders", y = "Highway MPG")
```

[Figure: 4 cyl: box 26–31, points 20–44 (three > 40); 5 cyl: 4 cars, box 28.75–29; 6 cyl: box 19–26, points 17–29; 8 cyl: box 16–19, points 12–26.] More cylinders → less chance of full utilization → lower efficiency. 4cyl widest spread, has the > 40 outliers. 5cyl too few cars for a reliable box. Groups overlap (some 4cyl worse than some 8cyl), but on average cyl negatively correlated with mpg.
Answer. (note) 4-cyl widest *range* (20–44), but widest *box* (IQR) is 6-cyl (7 vs. 5). `outlier.shape = NA` stops outliers being drawn twice (box + jitter).

HW1 Ex5

Prompt. Lower-triangle correlation matrix of `mpg` with `corrplot`. Which pairs ± correlated? Sensible / surprising?
My answer. *[Figure: numeric vars, blue = +, red = −. year–displ 0.15; cyl–displ 0.93, cyl–year 0.12; cty–displ −0.80, cty–year −0.04, cty–cyl −0.81; hwy–displ −0.77, hwy–year 0.00, hwy–cyl −0.76, hwy–cty 0.96.]* `displ`–`cyl` strongly + (more cylinders = more volume displaced per cycle). Both mpg vars strongly − with both engine-size vars (more volume/mass to heat → higher fuel burn). Year uncorrelated with mpg — surprising (EPA tightening), but data only 1999 and 2008.

[Ex6–8: 231 only, from `homework-1-1.html`; not in my submission. Answers = my recreations, checked in R.]

HW1 Ex6 (231 only)

Prompt. Recreate with `ggplot()` (hint: `ggthemes`): *[Figure: horizontal boxplots, x = Highway MPG, y = Vehicle Class (2seater … suv), unfilled boxes, black outlier dots, gray jittered points, Google-Docs-style theme. Pickup, suv lowest (≈17); compact/midsize/subcompact highest (medians ≈26–27); subcompact widest.]*
Answer.

```
ggplot(mpg, aes(x = hwy, y = class)) +
  geom_boxplot() + geom_jitter(alpha = 0.3) +
  theme_gdocs() +
  labs(x = "Highway MPG", y = "Vehicle Class")
```

HW1 Ex7 (231 only)

Prompt. Recreate: *[Figure: default theme; boxplots of `hwy` by `class`, fill = `drv` (4 red, f green, r blue); medians: 2seater r ≈25; compact 4 ≈25, f ≈29 (outliers 37, 44); midsize 4 ≈24, f ≈27; minivan f ≈23; pickup 4 ≈17; subcompact 4 26 (flat), f ≈29 (to 44), r ≈23; suv 4 ≈18 (many outliers), r ≈17.]*
Answer.

```
ggplot(mpg, aes(x = class, y = hwy, fill = drv)) + geom_boxplot
    ↪ ()
```

HW1 Ex8 (231 only)

Prompt. Recreate: *[Figure: `hwy` vs. `displ`, points colored by drv; one blue loess curve per drv, no ribbon, linetype by drv, one merged legend: 4 solid, ≈27→16 then flat; f short dash, ≈33→25 then flat; r long dash, ≈25, dip to ≈18 at displ 5.3, back to ≈25.]*
Answer.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(aes(linetype = drv), se = FALSE)
```

16.2 Homework 2 – linear regression and KNN (abalone)

Prompt. UCI abalone data: 4,177 abalone (Tasmania). Age (= **rings + 1.5**) normally needs cutting the shell and counting rings under a microscope; goal: predict age from easier measurements. Codebook:

Sex	nominal	–	M, F, I (infant)
Length	cont.	mm	longest shell measurement
Diameter	cont.	mm	perpendicular to length
Height	cont.	mm	with meat in shell
Whole weight	cont.	g	whole abalone
Shucked weight	cont.	g	weight of meat
Viscera weight	cont.	g	gut weight (after bleeding)
Shell weight	cont.	g	after being dried
Rings	integer	–	+1.5 = age in years

[CSV names: `type, longest_shell, diameter, height, whole_weight, shucked_weight, viscera_weight, shell_weight, rings`.]

HW2 Q1

Prompt. Add age (= rings + 1.5); describe its distribution.
My answer.

```
head(abalone)
```

```
  type  longest_shell diameter height whole_weight shucked_weight
    ↪ viscera_weight
1 M             0.455    0.365  0.095        0.514          0.224
    ↪     0.101
2 M             0.35     0.265  0.09         0.226          0.0995
    ↪     0.0485
3 F             0.53     0.42   0.135        0.677          0.256
    ↪     0.142
4 M             0.44     0.365  0.125        0.516          0.216
    ↪     0.114
5 I             0.33     0.255  0.08         0.205          0.0895
    ↪     0.0395
6 I             0.425    0.3    0.095        0.352          0.141
    ↪     0.0775
# i 3 more variables: shell_weight <dbl>, rings <dbl>, age <dbl>
```

```
abalone %>% summarise(n = n(), mean = mean(age), median = median
    ↪ (age),
  sd = sd(age), min = min(age), q1 = quantile(age, 0.25),
  q3 = quantile(age, 0.75), max = max(age))
```

```
     n  mean median    sd  min   q1   q3  max
1 4177 11.4   10.5  3.22  2.5  9.5 12.5  30.5
```

[Figure: histogram (binwidth 1): unimodal, peak ≈690 at 10.5 (11.5 ≈630, 9.5 ≈570, 12.5 ≈490), long thin right tail to 30.5 (bars barely visible > 24). Boxplot: box 9.5–12.5, median 10.5, whiskers 5.5–16.5, outliers 2.5–4.5 and 17.5–30.5.] Max ≈ 3× median, but IQR only 3 years → tightly bound around median.
Answer. (check) Correct: `boxplot.stats()` flags 278/4177 = 6.66% as outliers (outside $9.5 - 1.5 \cdot 3 = 5, 12.5 + 1.5 \cdot 3 = 17$).

HW2 Q2

Prompt. Train/test split with stratified sampling; choose proportions.
My answer. 85/15 (Andrew Ng's split, I think): 3548 train rows, 629 test rows — both adequate.

HW2 Q3

Prompt. Recipe on **training** data: age ~ all predictors **except** `rings`. *Why not use `rings`?*
My answer.

```
abalone_recipe %>% prep() %>% bake(new_data = NULL) %>% head()
```

```
  longest_shell diameter height whole_weight shucked_weight
    ↪ viscera_weight
1         -1.62    -1.54  -1.41        -1.27          -1.22
    ↪ -1.29
2         -0.828   -1.09  -1.05        -0.975         -0.985
    ↪ -0.943
3         -1.41    -1.29  -1.29        -1.10          -1.19
    ↪ -1.29
4         -0.496   -0.537 -0.817       -0.715         -0.599
    ↪ -0.519
5         -0.621   -0.537 -0.817       -0.627         -0.554
    ↪ -0.587
6         -2.66    -2.60  -1.99        -1.61          -1.50
    ↪ -1.51
# i 8 more variables: shell_weight <dbl>, age <dbl>, type_I <dbl>,
```

```
#   type_M <dbl>, type_I_x_shucked_weight <dbl>,
      ↪ type_M_x_shucked_weight <dbl>,
#   longest_shell_x_diameter <dbl>, shucked_weight_x_shell_weight <dbl>
```

Answer. (note) target leakage: age = rings $+1.5$ exactly → perfect fit ($R^2 = 1$), uninformative; and rings is the hard-to-obtain measurement we want to avoid.

HW2 Q4

Prompt. Linear regression object, "lm" engine.
My answer. Linear Regression Model Specification (regression) / Computational engine: lm

HW2 Q6

Prompt. For LM and KNN: two separate workflows (empty workflow → add model → add Q3 recipe); fit both to training.
My answer. (`lm_fit %>% extract_fit_parsnip() %>% tidy()`, then knn_fit):

```
   term                          estimate std.error statistic  p
     ↪ .value
 1 (Intercept)                    11.5      0.0367    312.     0
 2 longest_shell                   0.466    0.281       1.66
     ↪ 9.67e- 2
 3 diameter                        2.20     0.308       7.16
     ↪ 1.00e-12
 4 height                          0.262    0.0689      3.81
     ↪ 1.43e- 4
 5 whole_weight                    4.98     0.390      12.8
     ↪ 1.71e-36
 6 shucked_weight                 -4.26     0.247     -17.3
     ↪ 3.07e-64
 7 viscera_weight                 -1.02     0.155      -6.54
     ↪ 7.22e-11
 8 shell_weight                    1.61     0.209       7.72
     ↪ 1.52e-14
 9 type_I                         -0.919    0.111      -8.26
     ↪ 1.99e-16
10 type_M                         -0.223    0.101      -2.21
     ↪ 2.71e- 2
11 type_I_x_shucked_weight         0.468    0.0843      5.54
     ↪ 3.16e- 8
12 type_M_x_shucked_weight         0.237    0.106       2.23
     ↪ 2.59e- 2
13 longest_shell_x_diameter       -2.79     0.394      -7.06
     ↪ 1.94e-12
14 shucked_weight_x_shell_weight -0.149     0.201      -0.742
     ↪ 4.58e- 1
```

```
kknn::train.kknn(formula = ..y ~ ., data = data,
  ks = min_rows(7, data, 5))
Type of response variable: continuous
minimal mean absolute error: 1.654218
Minimal mean squared error: 5.598762
Best kernel: optimal
Best k: 7
```

HW2 Q8

Prompt. With yardstick, for LM and KNN: metric set of R^2, RMSE, MAE; `augment()` on **testing** data (predictions + observed age); apply metrics, report, interpret R^2.
My answer. (`head()` of `augment(...) %>% select(age, .pred)`, LM and KNN side by side):

```
    age .pred(LM) .pred(KNN)
1   8.5  9.51       9.31
2  11.5 12.5       12.0
3  13.5 12.9       13.4
4   8.5 10.5       10.9
5  12.5  9.91       8.68
6  13.5 12.0       11.7
```

[Figure: Predicted vs. observed test age (4–23), facets KNN (k = 7) and LM, red dashed $y = x$: both follow the line for young abalone, fall below it (under-predict) above age ≈15; KNN more spread.]

HW2 Q9

Prompt. Which model did better on testing? Why? Surprised?
My answer.

```
bind_rows(
  abalone_metrics(lm_test_results, truth = age, estimate = .pred
    ↪ ) %>%
    mutate(model = "Linear Regression", data = "Testing"),
  abalone_metrics(knn_test_results, truth = age, estimate = .
    ↪ pred) %>%
    mutate(model = "KNN (k = 7)", data = "Testing"),
  augment(lm_fit, new_data = abalone_train) %>%
    abalone_metrics(truth = age, estimate = .pred) %>%
    mutate(model = "Linear Regression", data = "Training"),
  augment(knn_fit, new_data = abalone_train) %>% ...  # same, "
    ↪ KNN (k = 7)"
) %>%
  select(model, data, .metric, .estimate) %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
    ↪
  arrange(data, model)
```

```
  model             data       rsq  rmse   mae
1 KNN (k = 7)       Testing   0.483  2.21  1.62
2 Linear Regression Testing   0.585  1.95  1.44
3 KNN (k = 7)       Training  0.811  1.46  1.02
4 Linear Regression Training  0.552  2.18  1.57
```

16.3 Homework 3 – binary classification (Titanic)

Prompt. Part of the Kaggle Titanic data; predict which passengers survived. Codebook:

survival / sex / Age:	Survival / Sex / Age in years
pclass:	Ticket class: 1 = 1st, 2 = 2nd, 3 = 3rd
sibsp / parch:	# siblings+spouses / parents+children aboard
ticket / fare / cabin:	Ticket number / Passenger fare / Cabin number
embarked:	Port: C = Cherbourg, Q = Queenstown, S = Southampton

[CSV: passenger_id, survived (Yes/No), pclass, name, sex, age, sib_sp, parch, ticket, fare, cabin, embarked; 891 rows. set.seed(3435).]

HW3 Q1

Prompt. Split, stratified on survived; verify row counts; note issues (e.g. missing data) in training. Why stratify here?

```
colSums(is.na(titanic_train))
```

Answer. missing data: age 128/623 (20.5%; `step_impute_linear()` in Q4), cabin 481 (77%, too sparse), embarked 2. fare right-skewed (max 512).

HW3 Q2

Prompt. Describe distribution of survived (training). Percent stacked bar charts of survived (x) with `fill = sex`, then `fill = pclass`: good predictors? Why percent stacked vs. traditional stacked?

```
titanic_train %>% count(survived) %>% mutate(prop = n / sum(n))
titanic_test %>% count(survived) %>% mutate(prop = n / sum(n))
```

```
  survived   n  prop       survived   n  prop    # train | test
1 Yes      239 0.384     1 Yes      103 0.384
2 No       384 0.616     2 No       165 0.616
```

Chart numbers (% within each bar) + survival rate per group (same code with pclass):

```
titanic_train %>% count(survived, sex) %>%
  group_by(survived) %>% mutate(prop = n / sum(n))
titanic_train %>% group_by(sex) %>%
  summarise(n = n(), surv_rate = mean(survived == "Yes"))
```

```
  survived sex        n  prop       sex        n surv_rate
1 Yes      female   156 0.653     1 female   214      0.729
2 Yes      male      83 0.347     2 male     409      0.203
3 No       female    58 0.151
4 No       male     326 0.849
  survived pclass     n  prop       pclass     n surv_rate
1 Yes      1         97 0.406     1 1        154      0.630
2 Yes      2         61 0.255     2 2        128      0.477
3 Yes      3         81 0.339     3 3        341      0.238
4 No       1         57 0.148
5 No       2         67 0.174
6 No       3        260 0.677
```

HW3 Q3

Prompt. correlation matrix of all continuous variables (training); visualize; which predictors correlated, which direction?

```
cor_titanic <- titanic_train %>%
  select(age, sib_sp, parch, fare) %>%
  cor(use = "pairwise.complete.obs")
round(cor_titanic, 3)
corrplot(cor_titanic, method = "number", type = "lower")
```

```
          age sib_sp  parch  fare
age     1.000 -0.327 -0.173 0.113
sib_sp -0.327  1.000  0.433 0.145
parch  -0.173  0.433  1.000 0.254
fare    0.113  0.145  0.254 1.000
```

Answer. (passenger_id = ID, pclass = factor, so excluded.) Pearson correlation: sib_sp–parch $+$ (0.43, families travel together); age–sib_sp $-$ (-0.33, children travel with siblings); age–parch weak $-$; fare weak $+$ with parch, sib_sp (fare is per ticket, families share one) and age.

HW3 Q4

Prompt. Recipe (training) for `survived` from pclass, sex, age, sib_sp, parch, fare; impute age with `step_impute_linear()`; **dummy** encode categoricals with `step_dummy()`; interactions sex:fare and age:fare (find the step in tidymodels docs).

```
titanic_recipe <- recipe(survived ~ pclass + sex + age + sib_sp
    ↪ +
                           parch + fare, data = titanic_train)
    ↪ %>%
  step_impute_linear(age,
                     impute_with = imp_vars(pclass, sex, sib_sp,
    ↪
                                            parch, fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
baked <- titanic_recipe %>% prep() %>% bake(new_data = NULL)
baked; sum(is.na(baked$age))
```

```
# A tibble: 623 x 10
     age sib_sp parch   fare survived pclass_X2 pclass_X3
    ↪ sex_male
 1    22      1     0   7.25 No                0          1
    ↪     1
 2    35      0     0   8.05 No                0          1
    ↪     1
 3    54      0     0  51.9  No                0          0
    ↪     1
 4     2      3     1  21.1  No                0          1
    ↪     1
 5    39      1     5  31.3  No                0          1
    ↪     1
 6    14      0     0   7.85 No                0          1
    ↪     0
# ... (4 more rows shown in R)
# i 2 more variables: sex_male_x_fare <dbl>, fare_x_age <dbl>
[1] 0
```

Answer. Imputation = linear model of age on the other 5 predictors (training rows with age observed). `step_dummy()`: sex → sex_male (female = reference level); pclass → pclass_X2, pclass_X3 (class 1 = reference). 1 outcome, 6 inputs → 9 predictors, no missing age.

HW3 Q5

Prompt. logistic regression, `"glm"` engine; workflow + recipe; `fit()` on training.
Answer. (`log_fit %>% extract_fit_parsnip() %>% tidy()`):

	term	estimate	std.error	statistic	p.value
1	(Intercept)	-4.40	0.683	-6.45	1.13e-10
2	age	0.0629	0.0136	4.62	3.77e- 6
3	sib_sp	0.437	0.132	3.30	9.52e- 4
4	parch	0.151	0.153	0.989	3.23e- 1
5	fare	-0.00116	0.0107	-0.108	9.14e- 1
6	pclass_X2	1.25	0.363	3.46	5.48e- 4
7	pclass_X3	2.44	0.382	6.39	1.62e-10
8	sex_male	2.15	0.303	7.09	1.32e-12
9	sex_male_x_fare	0.0139	0.00836	1.66	9.65e- 2
10	fare_x_age	-0.000360	0.000206	-1.75	8.03e- 2

"Yes" is the first level → `glm` models $P(\text{"No"})$: log odds of *dying*. sex_male $+2.15$: male odds of dying $e^{2.15} \approx 8.6\times$ female's (at fare = 0, others fixed); pclass_X3 $+2.44$: $e^{2.44} \approx 11.5\times$ vs. 1st class; age $+0.063$/yr. Significant (p-value < 0.05): age, sib_sp, pclass, sex; interactions borderline ($p \approx 0.08$–0.10).

HW3 Q6

Prompt. Repeat Q5 with linear discriminant analysis (LDA), `"MASS"` engine.
Answer. (`lda_fit`; group means re-wrapped):

```
lda(..y ~ ., data = data)
Prior probabilities of groups:
      Yes        No
0.3836276 0.6163724
Group means:
          age    sib_sp     parch     fare pclass_X2 pclass_X3
Yes 28.74378 0.4393305 0.4267782 48.81424 0.2552301 0.3389121
No  29.64519 0.5807292 0.3463542 22.99299 0.1744792 0.6770833
     sex_male sex_male_x_fare fare_x_age
Yes 0.3472803         12.91229    1568.232
No  0.8489583         19.63018     703.509
Coefficients of linear discriminants (LD1):
age 0.0358259965     sib_sp 0.2231961671   parch 0.0876726369
fare 0.0032525962    pclass_X2 0.8724616082
pclass_X3 1.6983328837   sex_male 1.9274805785
sex_male_x_fare 0.0020072010   fare_x_age -0.0001487374
```

Survivors paid more (48.8 vs. 23.0), less often male (0.35 vs. 0.85), less often 3rd class (0.34 vs. 0.68). LD1: positive weights on sex_male, pclass_X3 point toward "No".

HW3 Q8

Prompt. Repeat Q5 with *k*-nearest neighbors, `"kknn"` engine; choose *k*.
Answer. $k = 10$; `knn_fit %>% extract_fit_parsnip()`:

```
Type of response variable: nominal
Minimal misclassification: 0.1990369
Best kernel: optimal
Best k: 10
```

(≈80% leave-one-out training accuracy.) KNN is distance-based, needs comparable scales; recipe has no `step_normalize()`, but kknn standardizes internally (`scale = TRUE` default), so fare does not dominate the Euclidean distance.

HW3 Q9

Prompt. With `predict()` + `bind_cols()`, predict all 4 models on **training** data; assess by **area under the ROC curve**.
Answer. `head(log_train)` (`predict(..., type = "prob")` + `bind_cols(survived)`):

	.pred_Yes	.pred_No	survived
1	0.114	0.886	No
2	0.0835	0.916	No
3	0.310	0.690	No
4	0.115	0.885	No
5	0.0219	0.978	No
6	0.755	0.245	No

Extra: training accuracy (default `type = "class"` → `.pred_class`, `accuracy()`): Logistic 0.814, LDA 0.793, QDA 0.778, KNN 0.891.

HW3 Q10

Prompt. Report each model's **testing** AUC; which is highest? For the best model: confusion matrix (+ visualize) and ROC curve plot. Compare its training vs. testing AUC; why different?

```
test_pred <- function(f) predict(f, titanic_test, type = "prob")
    ↪  %>%
  bind_cols(predict(f, titanic_test)) %>%
  bind_cols(titanic_test %>% select(survived))
log_test <- test_pred(log_fit); lda_test <- test_pred(lda_fit)
qda_test <- test_pred(qda_fit); knn_test <- test_pred(knn_fit)
```

Extra: testing accuracy Logistic 0.832, LDA 0.806, QDA 0.810, KNN 0.843. *[Figure: LDA test ROC (x = 1 − specificity, y = sensitivity): sens ≈0.70 at 0.05, ≈0.85 at 0.18, ≈0.90 from 0.25 to 0.6, then (1, 1); AUC 0.881.]*

Prompt (231 only). Binary problem: p = P(class 1), $1 - p$ = P(class 0). The *logistic function* ("inverse logit") is the CDF of the logistic distribution, mapping real *z* to $(0, 1)$. *[Q11–Q12 + answers: Part I.]*

17 Practice exam (Fall 2026) and Sai's graded midterm (last quarter)

[In Part I: PE Q1–6, 10–12, 15–19 and all PE solution texts; Sai Q1, Q4, Q8a–d, Sai's answers, Q3/Q6 figures/tables, rubric lines restated there; cover sheet (mostly), scratch pages.]

17.1 Practice exam with official solutions

*[Correct choices in **green bold** (red bold in the key); prompts identical in exam and key.]*

PE Q7

7. Researcher uses KNN to predict whether a bank customer defaults on a loan:

```
loan_recipe <-
  recipe(defaulted ~ ., data = loans) |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE)
loan_model <-
  nearest_neighbor(neighbors = 5) |>
  set_engine("kknn") |>
  set_mode("classification")
loan_fit <-
  loan_model |>
  fit(defaulted ~ ., data = loans)
```

Variables: `application_id`, `age`, `income`, `employment_type`, `credit_score`, `default_date`, `defaulted`. Identify at least four problems; for each, explain why it matters and how to correct it.

PE Q8

8. scatterplot of X, Y: very strong U-shape; Pearson correlation 0.03. Best statement?
a. X and Y are essentially unrelated
b. The scatterplot is likely wrong
c. **Little linear correlation, despite a strong nonlinear relationship**
d. X will not be useful for predicting Y
e. Both C and D

PE Q9

9. Which EDA finding is evidence of an **interaction**?
a. X1, X2 Pearson correlation 0.91
b. **X1–Y relationship positive for one level of X2, negative for the other**
c. X1 strongly right-skewed, X2 strongly left-skewed
d. X1, Y Pearson correlation 0.80
e. X1–Y relationship the same across both levels of X2

Q10–12 stem: university predicts graduating within four years from information available **at the end of first year**.Variables: StudentID (unique ID), HS_GPA, FirstYearGPA, Major (at end of 1st yr), CreditsAttempted, CreditsCompleted (1st yr), Dorm (1st-yr hall name), GraduationDate (if graduated), FinalGPA (on leaving), Graduated4Years (outcome).

PE Q13

13. Predict "salary" from "experience" and "department" (levels Sales, IT, HR):

```
salary_recipe <- recipe(salary ~ experience + department,
                        data = employees) %>%
  step_dummy(department, one_hot = TRUE)
salary_model <- linear_reg() %>%
  set_engine("lm")
salary_workflow <- workflow() %>%
  add_recipe(salary_recipe) %>%
  add_model(salary_model)
salary_fit <- fit(salary_workflow, data = employees)
```

R reports the model matrix is rank-deficient; one or more coefficients cannot be uniquely identified. (a) What caused it? (b) Why does "one_hot = TRUE" cause this for ordinary linear regression with an intercept? (c) How to change the code? (d) Then how many indicator variables for "department"?

PE Q14

14. logistic regression predicts pneumonia from symptoms/clinical measurements; one patient's predicted probability = 0.78. Correct interpretation?
a. Model is 78% accurate for this patient
b. Patient definitely has pneumonia
c. **Fitted model assigns this patient probability 0.78 of belonging to the pneumonia class**
d. 78% of the patient's symptoms indicate pneumonia

PE Q20

20. 50,000 obs, two predictors, clear evidence covariance structures differ substantially across outcome levels. Which has greater potential to capture the true decision boundary? (a) LDA (b) **QDA** (c) both equally (d) neither, both are parametric

17.2 Sai's midterm (graded, 20/26)

[Gradescope export: PDF pp. 1–3 rubric summary, 4–8 scanned booklet, 9–10 blank. Labels "Gradescope Q12 / booklet Q9" (Gradescope 1–12, booklet 1–9 with Q8a–d). ✓ = rubric item applied; green −0, red = deduction.]

Cover sheet excerpt (Summer 2026): (3) Mark answers to **all questions** in the booklet. (4) Open book/notes: any **physical materials**; **absolutely no** electronic/digital tools. (8) Round to 3–4 decimals when appropriate; must be legible; if multiple answers, the first is graded, except multiple choice: multiple marked answers score zero unless otherwise specified.

Multiple choice (PDF pp. 5–6)

Sai Q2 (GS Q2) — 2 / 2 pts

2. Why can KNN represent nonlinear decision boundaries without explicit polynomial or interaction terms? (A) assumes predictors normal; (B) does not assume equal variances across outcome levels; (C) classification determined locally by neighboring observations; (D) estimates a different regression coefficient for every observation; (E) automatically creates squared terms; (F) both A and B.
Grader / rubric.
✓ **−0** C. KNN makes no normality/equal-variance assumptions, creates no squared terms.
−1 B (partial): that assumption lets QDA represent nonlinear boundaries.
−2 A, D, E, or F.

Sai Q3 (GS Q3) — 2 / 2 pts

3. Moving along the ROC curve from Point A toward Point C generally corresponds to: (A) increasing the classification threshold; (B) decreasing the threshold; (C) increasing the AUC; (D) decreasing the AUC.
Grader / rubric. **−1** A (wrong direction, but recognizes threshold changes). **−2** C or D.

Sai Q5 (GS Q5) — 2 / 2 pts

5. If population covariance matrices really are identical across classes, what advantage does QDA gain from a separate covariance matrix per class? (A) closer fit to the population decision boundary; (B) no inherent advantage; may simply increase estimation variance; (C) perfect classification; (D) QDA, unlike LDA, is nonparametric.

Sai Q6 (GS Q6) — 0 / 2 pts

6. Which feature of these results most strongly suggests an interaction? (A) Treatment B always has the larger fitted value; (B) increasing dosage changes the fitted value by 10 under Treatment A but by 25 under B; (C) all four fitted values differ; (D) increasing dosage increases the fitted value under both treatments; (E) dosage has two levels; (F) both B and D.

Sai Q7 (GS Q7) — 0 / 2 pts

7. In logistic regression, the age coefficient is positive. Raising one customer's age 40 → 41 does not change their predicted class. Why possible? (A) a positive coefficient does not affect predicted probabilities; (B) predicted probability can increase without crossing the classification threshold; (C) coefficients affect only training observations; (D) age must interact with another predictor to affect classification.

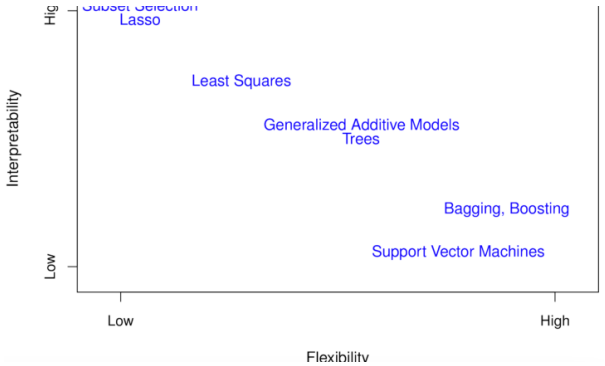
Free response (PDF pp. 7–8)

Sai Q9 (GS Q12) — 3 / 4 pts

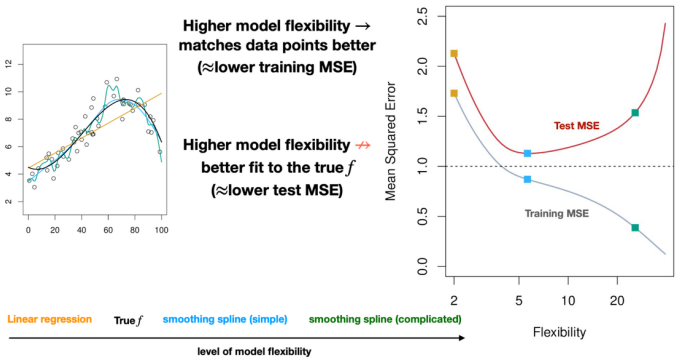
9. Predict home sale price; `square_feet` = size in feet, `square_meters` = size in meters:

```
house_recipe <-
  recipe(sale_price ~ ., data = houses) %>%
  step_mutate(
    square_meters = square_feet * 0.092903
  ) %>%
  step_normalize(all_numeric_predictors())
```

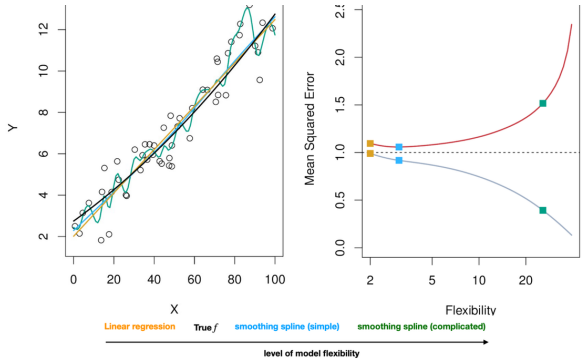
A student argues that since both are normalized, keeping both in ordinary linear regression is appropriate. Correct or incorrect? Explain.
Grader / rubric.
−2 Logic partially correct. **−4** Logic completely incorrect.
✓ **−1** Correct (reasoning is wrong) but doesn't specifically mention collinearity.



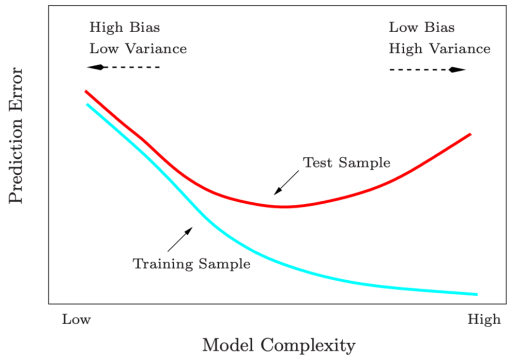
F1. Flexibility vs. interpretability (S1.55): subset selection/lasso (high interp., low flex.) → least squares → GAMs, trees → bagging/boosting, SVM (high flexibility, low interp.).



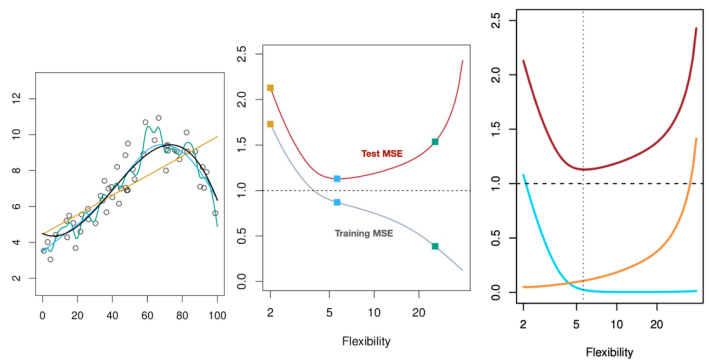
F2. Ex. 1, curved true f (S1.62). Left: linear (orange), simple and complex smoothing splines (blue, green). Right: training error (gray) keeps falling; test MSE (red) U-shaped, min at medium flexibility; dashed = Var(ε).



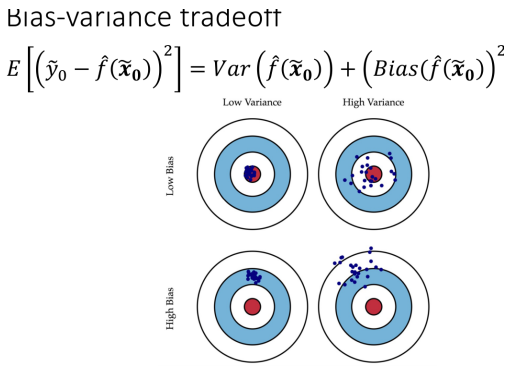
F3. Ex. 2, true f nearly linear (S1.63): the linear fit is already close; test MSE is lowest at low flexibility and rises for the wiggly spline (overfitting).



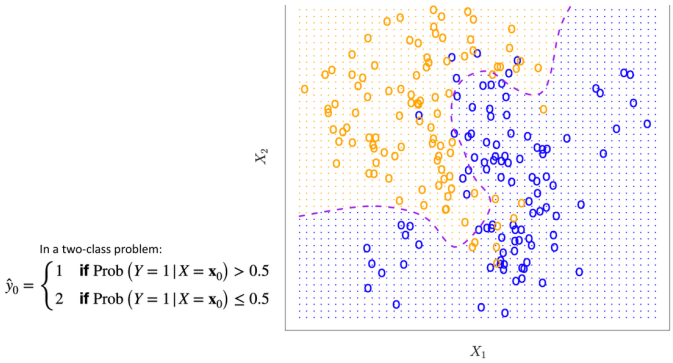
F4. Prediction error vs. model complexity (S1.66): training sample error falls monotonically; test sample error U-shaped. Left = high bias/low variance, right = low bias/high variance.



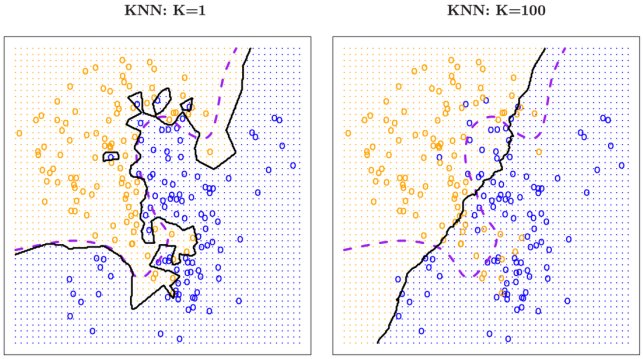
F5. bias-variance tradeoff for Ex. 1 (S1.82). Right panel: test MSE (red) = squared bias (blue, falls) + variance (orange, rises) + Var(ε) (dashed); dotted vertical line = flexibility with the lowest test MSE.



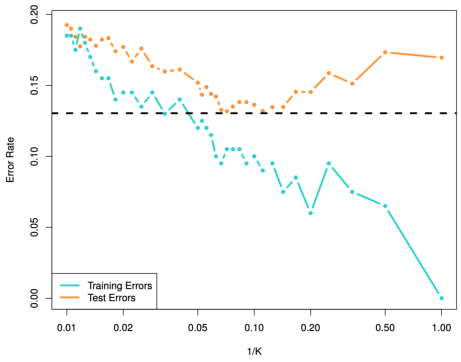
F6. Dartboards (S1.80): rows = low/high bias (off-center), columns = low/high variance (spread). Ideal = top-left (tight on the bullseye).



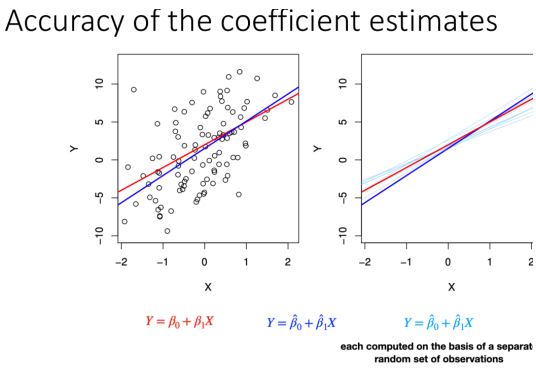
F7. Bayes classifier (S1.97): orange vs. blue simulated classes in (X_1, X_2); purple dashed = Bayes decision boundary where Prob$(Y = 1 \mid X) = 0.5$; shaded grid = predicted class.



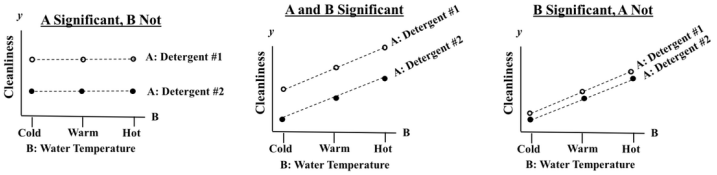
F8. KNN (S1.113): $K = 1$ (left) gives a jagged, island-filled boundary (overfits, training error 0); $K = 100$ (right) is too smooth, nearly linear (underfits). Purple dashed = Bayes boundary.



F9. KNN error rate vs. $1/K$ (log scale) (S1.115): training (blue) falls to 0 at $K = 1$; test (orange) U-shaped, min near $K \approx 10$; dashed ≈ 0.13 = Bayes error rate.



F10. Accuracy of least squares estimates (S2.25). Left: red = true line, blue = LS fit to one sample. Right: light-blue LS lines from many random samples scatter around the true line (unbiased, but variable).



F11. main effect plots (S2.50): Cleanliness vs. water temperature for detergents A/B. Parallel lines in every panel ⇒ *no* interaction term (only which factor shifts the lines differs).

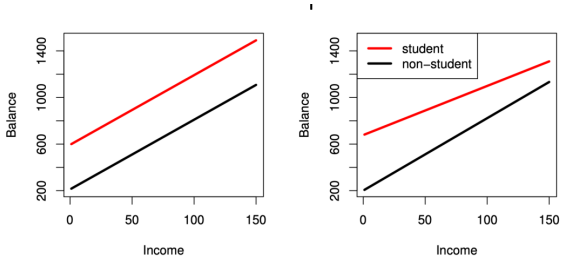


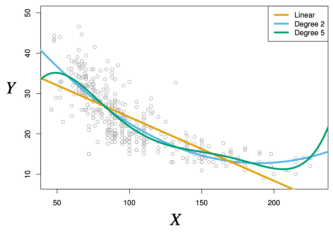
FIGURE 3.7. *For the* `Credit` *data, the least squares lines are shown for prediction of* `balance` *from* `income` *for students and non-students. Left: The model (3.34) was fit. There is no interaction between* `income` *and* `student`*. Right: The model (3.35) was fit. There is an interaction term between* `income` *and* `student`*.*

F12. Credit data (S2.53): balance vs. income, student (red) vs. non-student (black). Left: no interaction ⇒ parallel lines. Right: with interaction ⇒ different slopes (student line flatter).

Polynomial regression of Y on X

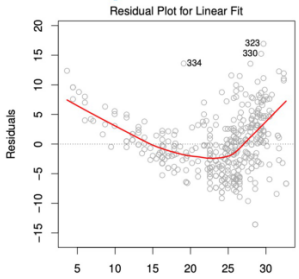
$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \ldots + \beta_d X^d + \varepsilon$$

degree



F13. polynomial regression (S2.55): Auto mpg vs. horsepower with linear, degree-2 and degree-5 fits. The curved (degree 2) fit follows the data; still a linear model in X, X^2.

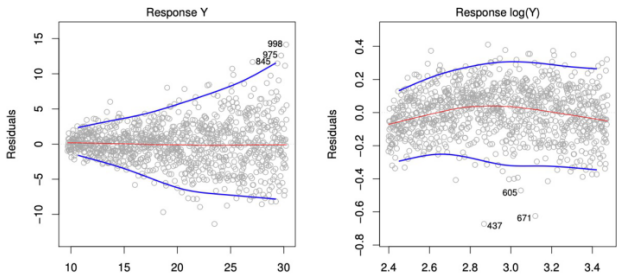
Residual plot plot the residual $y_i - \hat{y}_i$ v.s the fitted value (prediction) $\hat{y}_i$



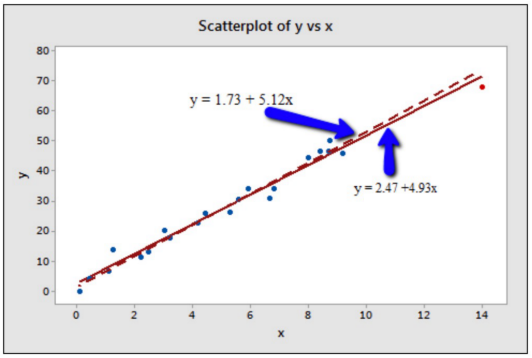
If our linear assumption is correct, then the residual plot will NOT show discernible pattern

F14. residual plot (S2.59): residuals vs. fitted values. A U-shaped pattern ⇒ nonlinear relationship missed; a correct linear model shows no discernible pattern.

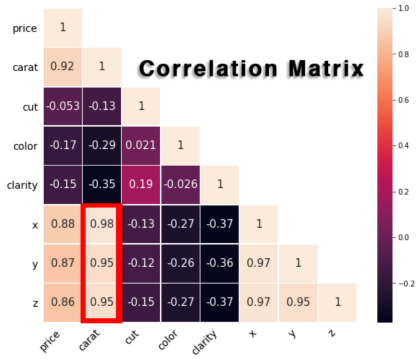
Inaccurate confidence interval or hypothesis testing results



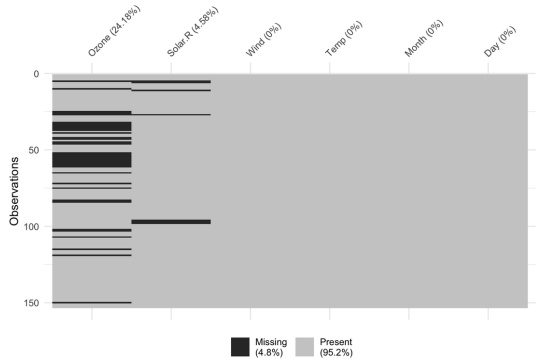
F15. heteroscedasticity (S2.61): residual spread grows with the fitted value (funnel), left; after transforming Y (e.g. log) the spread is roughly constant, right.



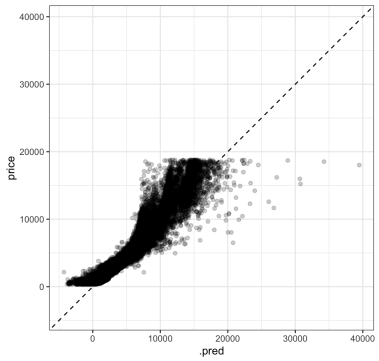
F16. influential observation (S2.70): fit without the flagged point $y = 1.73 + 5.12x$ vs. with a high-leverage point on the trend $y = 2.47 + 4.93x$ — almost the same line, so *not* influential.



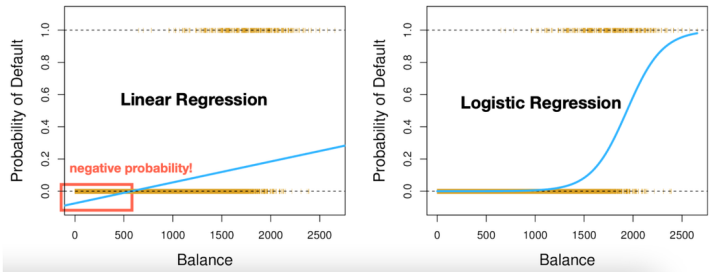
F17. correlation matrix of `diamonds` (S2.74): price–carat $r = 0.92$; strong positive correlations between size variables ⇒ collinearity.



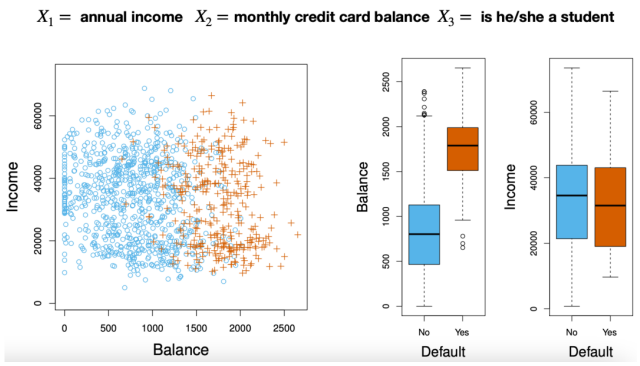
F18. `vis_miss()` on `airquality` (Lab 2): black = missing. Ozone ≈24% missing, Solar.R ≈5%; other columns complete (≈4.8% of all cells).



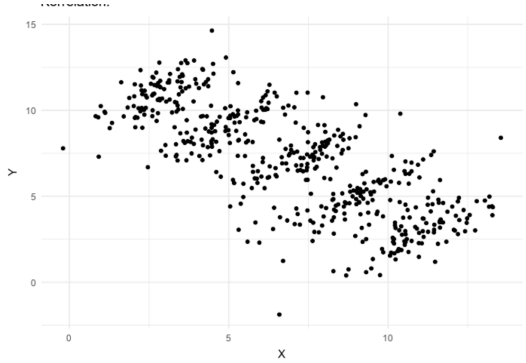
F19. Predicted vs. actual `price`, linear model on `diamonds` (Lab 2): curved band, not along $y = x$; cheap diamonds get negative predictions ⇒ nonlinear relationship the linear model misses.



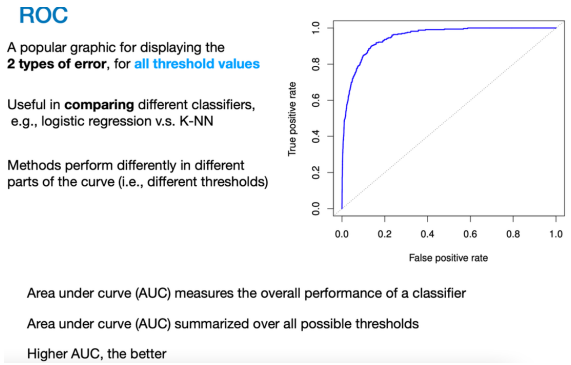
F20. Why not linear regression? (S3.5): on Default vs. balance, the linear fit gives *negative* probabilities at low balance; the logistic regression S-curve stays in $[0, 1]$.



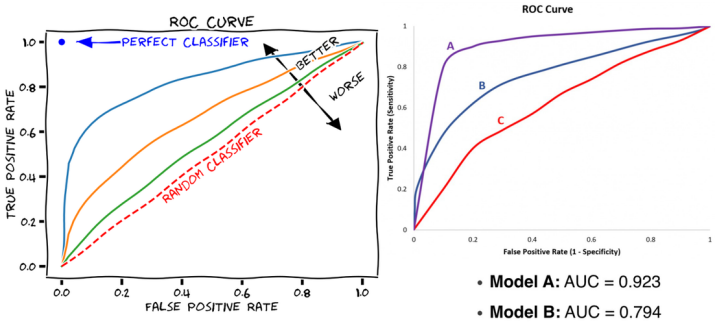
F21. Default data (S3.8): balance vs. income, orange = defaulted. Boxplots: balance much higher for default = Yes; income barely differs ⇒ balance is the useful predictor.



F22. Simpson's paradox (S3.31): pooled points trend downward, but each visible cluster trends upward — a confounding variable reverses the sign (like the student coefficient in Default).

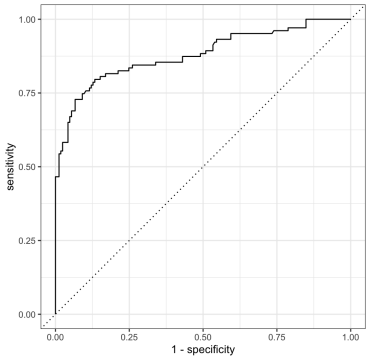


F23. ROC curve (S3.39): TPR (sensitivity) vs. FPR (1 − specificity) over all thresholds. AUC summarizes performance over all thresholds; higher AUC is better.

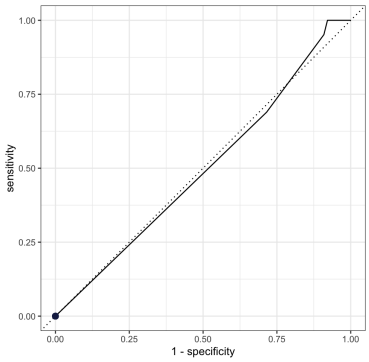


- **Model A:** AUC = 0.923
- **Model B:** AUC = 0.794
- **Model C:** AUC = 0.588

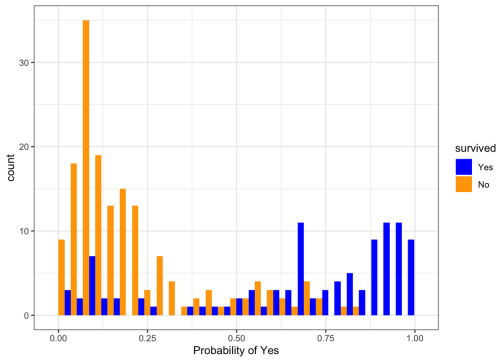
F24. ROC examples (S3.40). Left: perfect classifier at the top-left corner $(0, 1)$, random guessing = diagonal; curves toward the top-left are better. Right: AUC Model A 0.923 > B 0.794 > C 0.588.



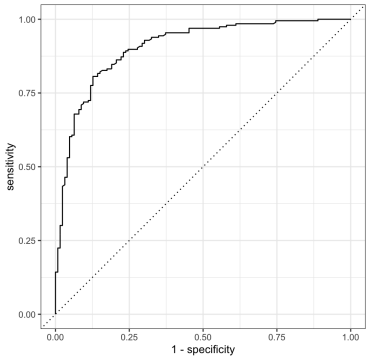
F25. ROC reading, Titanic Model A, test set (ROC): curve bows well toward the top-left (good discrimination). Axes: sensitivity vs. $1 -$ specificity; dotted diagonal = coin flip.



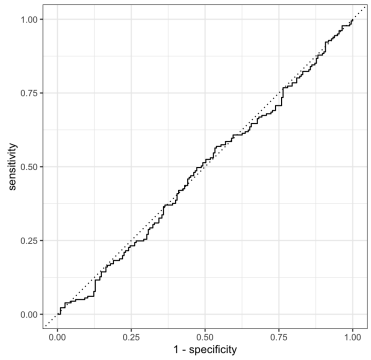
F26. ROC reading, Titanic Model B (ROC): curve sits on the diagonal, AUC $= 0.496$ — no better than a coin flip; it never reaches TPR 1 with FPR 0.



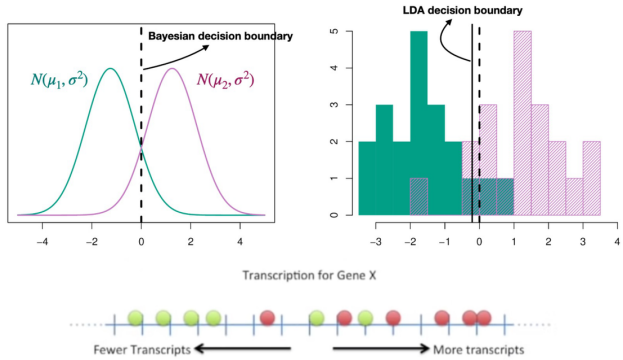
F27. Model A predicted P(Yes) by true class (ROC): survivors (blue) mostly near 1, non-survivors (orange) mostly near 0 — good separation, which is why its ROC bows toward the top-left.



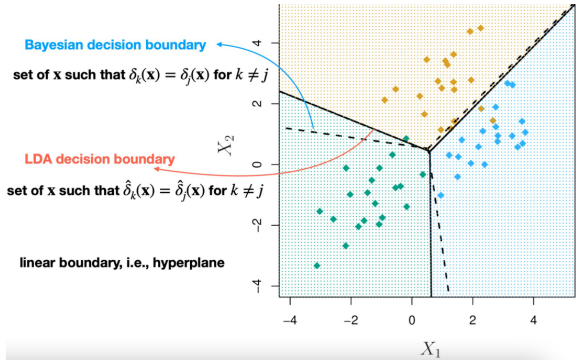
F28. Lab 3 OJ logistic regression, test ROC (Lab 3): sensitivity ≈0.68 at FPR 0.06, 0.90 at FPR 0.25 — good discrimination.



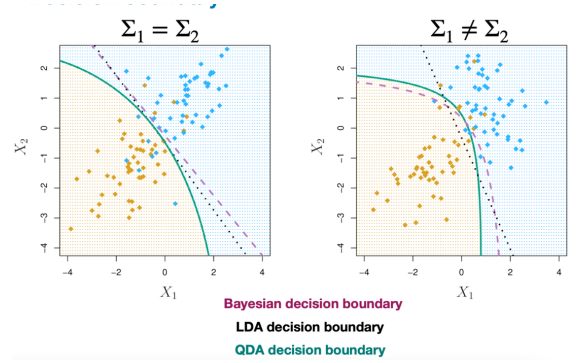
F29. Lab 3 Smarket QDA, test ROC (Lab 3): jagged and hugging the diagonal $\Rightarrow$ AUC ≈ 0.5; stock direction is essentially unpredictable from the lags.



F30. LDA, $p = 1$ (S3.77). Left: two normal class densities with equal variance; dashed = Bayes boundary (midpoint if $\pi_1 = \pi_2$). Right: histograms of training data with the LDA boundary (solid) next to the Bayes one.



F31. LDA, $p > 1$, three classes (S3.81): dashed = Bayes boundaries, solid = linear discriminant analysis (LDA) boundaries; both are straight lines (hyperplanes).



F32. LDA vs. QDA (S3.88). Purple dashed = Bayes, black dotted = LDA, green solid = QDA. Left, $\Sigma_1 = \Sigma_2$: LDA is closer to Bayes. Right, $\Sigma_1 \neq \Sigma_2$: quadratic discriminant analysis (QDA) is closer.

INDEX – page : grid cell

How to read: 12:C4,D7 = distilled notes, page 12, grid cells C4 and D7; 45:B2 = raw transcription, page 45, column B, row 2; pages are separated by ';'. Columns A–H run left→right across the sheet (A–D ≈ left text column, E–H ≈ right text column); rows 1–11 run top→bottom, one inch each. The small grey letters/numbers in the page margins mark the grid.

Symbols & operators

%>% 15:A6,B8,C8,D10; 25:A2; 28:C4; 31:D3, H4
%in% 15:G6
.estimate 13:C3,C4; 17:F2; 20:D3; 29:G2
.estimator 13:C3,C4; 17:H2
.metric 13:C3; 17:G2
.pred 13:B2; 15:H2,B3; 17:B3; 26:B7
.pred_CH 27:G5,H10
.pred_class 4:H11; 10:D8,C9; 14:H4; 15:A1,G4; 17:B4; 19:F9; 20:C3; 24:G4; 30:G4
.pred_Down 17:D5; 27:G3
.pred_MM 27:G5
.pred_No 14:E11; 15:G4; 17:B4
.pred_Up 27:G3
.pred_Yes 10:F2; 12:D9; 14:H4; 15:D1, G4; 17:D4; 19:E9; 20:A3; 24:H3
.threshold 14:G8; 24:G7,C11
== 25:D2
|> 15:D6; 30:H8

A

abalone 5:C10; 7:D4; 8:D10; 15:C9; 20:E6; 28:E3
accuracy 2:B5; 3:E6; 12:E2; 13:A10; 18:F3; 19:A3; 20:C2,E3; 23:B1; 27:H8
accuracy() 5:C1; 12:D2; 14:H5; 15:D1, H5; 17:E2; 30:G4
add_formula() 15:H5
add_model() 4:G10; 8:G2; 10:D8; 15:G5, B8,D11; 19:G9; 26:G2; 31:D4
add_recipe() 4:G10; 8:G2; 10:D8; 15:C8, D11; 18:C9; 19:F9; 26:G2; 31:D4
adjusted R-squared 6:G3
aes() 5:D11; 15:F11; 24:F5,G5; 27:C6
age 28:E5; 29:G4
AIC 9:E6
airquality 5:G8; 26:E4,G5
all_nominal_predictors() 4:G9; 7:B6; 8:C11; 12:D7; 15:D10; 16:C7,A10; 20:G10; 24:C10; 30:C3,H8
all_numeric_predictors() 4:G9; 15:D10; 16:E2,C10; 31:H4
all_outcomes() 16:D10
all_predictors() 8:G1; 16:G2,A10
alpha 28:D5,C6
AlphaGo 22:A8
arrange() 15:E7,H9; 25:B7,D7; 29:F2
artificial intelligence (AI) 22:B6
AUC *see* AUC (area under the ROC curve)
AUC (area under the ROC curve) 2:A5; 10:C4; 12:A10; 13:F9; 14:A5; 18:G4; 19:A5,A6; 20:A10,E11; 23:E11; 24:A8; 30:F3; 31:A10,B10; 34:G8
augment() 4:G11; 8:H3; 10:D8,B10; 14:G4; 15:H2,A3,A9; 16:F4; 17:A4,E6; 20:E1,C3,E7; 24:G4; 27:E7; 29:A7
autoplot() 12:C9; 14:F5; 15:H5; 21:B2

B

backward selection 6:H6; 23:A9
bake() 8:B9; 16:H3,F4; 26:C1,G9,E10; 28:H9; 30:D3
bar plot 5:B8; 20:H5; 28:B4; 29:F5
Bayes classifier 3:E8; 10:E3; 20:A6; 23:A3; 32:E8
Bayes decision boundary 3:E9; 10:E10; 32:H8
Bayes error rate 3:E8,E9; 20:B7; 33:B3
Bayes' theorem 10:E2,F4
best subset selection 6:F5
$\hat{\beta}$ 23:C8
bias 3:A7; 6:E1; 11:G3; 12:F5; 19:B7; 20:G4; 22:G10; 23:A2; 32:B10
bias-variance decomposition 23:A2
bias-variance tradeoff 1:E7; 3:A6,A8; 8:H8; 11:A10; 12:G4; 19:A7; 20:B5; 22:E7,G8; 26:F3; 32:E3
Bikeshare 27:B4
bind_cols() 5:A2; 12:D9; 13:B2; 15:B3, E8; 17:D5,C6; 20:G11; 30:F3
bind_rows() 15:E9
bins 25:F7
boxplot 5:B8; 20:G5; 24:G6,G9; 25:E5,G5; 27:C7; 28:A5

C

cabin 29:E4
CART 22:B6
categorical predictor 6:E11; 7:A1; 18:F8, D11; 23:A10; 27:D10
causation 9:C10
centering 8:A8
class imbalance 5:A8; 6:C5; 12:A5; 13:E8; 20:A2,F9; 27:B9
class label 23:A4
class overlap 24:B5
classical programming 22:B6
classification 1:F8; 2:A4; 8:E9,G10; 20:G3, B5; 22:H6; 23:A3,E3; 24:F4; 27:A4; 29:E3
clustering 22:G5,H6
coefficient 6:F8; 18:H11; 23:G6; 26:A6,B7; 27:C10; 29:A6; 30:D8; 31:E2,D4
coefficient interpretation 9:A10
coin_flip 14:B9
collinearity 5:F2,E7; 7:E10; 8:A1; 17:H9; 18:G1,E7; 19:F5; 20:A2,E10; 25:F4; 27:A6; 31:E3; 34:A3
color 28:F2
colSums() 29:F5
conf_mat() 10:D8; 14:H5; 15:A1,B2,H4; 17:E3; 19:C2; 21:D1; 24:H4
confidence interval 6:G2
confounding 5:E3,F7; 9:E8,E10,G10; 20:A9; 23:E6; 25:F6; 34:E6
confusion matrix 10:C4; 13:A8,B8; 19:A1; 21:C1; 23:E9,E10; 27:G7,E8; 30:E5
coord_obs_pred() 16:C3
cor() 5:H4,C9,D11; 16:D3; 29:G9
correlate() 16:D3; 27:A5
correlation 5:E3; 23:E8; 25:F4,G4,E6; 27:B7,B9; 28:B4
correlation matrix 5:B9; 8:C2; 20:G5; 25:E4; 27:D4,C5,D7; 28:B7; 29:E9; 34:A3
corrplot 15:D5; 28:C7
corrplot() 5:D9,D11; 16:D3; 20:F5; 25:H4; 29:H10
corrr 15:D5; 27:B4,B5
count() 5:D8; 15:E8; 28:C4; 29:E6
covariance matrix 11:A2; 18:F4; 20:B8; 31:A6,D6,A10,C11
Credit (dataset) 23:E2
cross-validation 2:E11; 4:F2; 11:F4; 12:D4, G5; 25:G8; 26:F1,F2
cty 28:B3
curse of dimensionality 4:E6,G6; 8:F6; 20:A6; 26:G4
cyl 28:B5

D

data processing 25:A1
data science workflow 22:A9
data splitting 6:A2; 16:A5; 24:F11; 25:E8; 28:E9
decision boundary 4:A9; 10:E9; 11:C7,A9; 20:B8; 24:D6; 31:A6,A8,C8
decision tree 22:A3,G5
deep learning 22:B6
Default 8:F10; 9:H4; 23:H4
Default data set 23:E4
density function 10:G4
density plot 5:C8
descriptive model 2:A7,B7; 20:G3
deviance 9:E6
diagnostic odds ratio 23:F11
diameter 28:E4
diamonds 3:A3; 5:H2,A10; 6:E6; 12:E7; 15:B9; 24:F11; 25:E2,F2,F9
dim() 15:E8
dimension reduction 22:G5,H6; 24:D3,C6
Direction (Smarket) 27:B5
discrim 10:D6; 11:E10; 15:D5; 16:H8; 20:H10; 27:C4
discrim_linear() 11:G10; 12:C2; 16:E8; 20:E10
discrim_quad() 11:G10; 12:A2; 16:E9; 20:F10
discriminant analysis 8:G10; 10:E2; 24:A1
discriminant function 10:F7; 30:A11
discriminative model 11:G2,C9
distance between means 24:A6
distance metric 4:A5; 18:D8
downsampling 27:B9
dplyr 15:C4,E5; 24:F11
drop_na() 5:F9; 15:E8
dummy variable 6:E11; 7:C1; 9:F6; 19:E2; 20:C7; 23:A10; 24:C10; 26:D2; 27:D10; 30:A2; 31:A3

E

EDA *see* exploratory data analysis (EDA)
embarked 29:F4
empirically-driven model 2:G1; 20:E4
engine 16:E7,F9; 26:D11
error rate 2:B5; 3:E5,F6; 12:F2; 13:C10; 23:A3
ESL 22:C3; 23:C5
estimate 5:C1; 8:G3; 13:C2; 15:D1,G2
Euclidean distance 4:A5; 18:C8; 19:A9; 30:G2
eval 25:D2
event level 10:A11; 14:E10; 20:B10
event_level 17:F4
everything() 15:F6
exam patterns 18:A4; 19:A1
exploratory data analysis 25:E2
exploratory data analysis (EDA) 5:A4,A5; 15:E10; 23:A4; 24:F11; 25:E2; 28:A2; 31:A1
extract_fit_parsnip() 5:A2; 6:G7; 15:H1; 17:C7; 26:C5

F

F-statistic 6:G4
F1 score 13:E2
facet_wrap() 16:A4
factor level 10:A11
factor() 2:A6; 12:D5; 14:C9; 15:H2,E8; 20:B10; 24:G4; 27:C7; 28:C5
false discovery rate (FDR) 23:H10
false negative 13:B9
false omission rate (FOR) 23:F10
false positive 10:A4; 13:C9; 19:A1
false positive rate (FPR) 13:F10,A11; 18:F3; 19:B3; 20:B9; 24:F5,F7
family = "binomial" 24:C3
family = binomial 9:F4,H4; 17:D10; 23:G5
fare 29:E4
fashion() 27:D7
feature 1:F10; 8:D4; 22:C7,D9; 25:H3,E10
feature engineering 5:A4,B5; 8:A4; 16:A6; 19:E6; 23:A4; 31:C2,E3
feature extraction 24:B3
feature selection 24:C3
fill 28:B11
filter() 15:E6,G9; 25:A2,D2,B10
fit() 4:G10; 8:H2; 10:D8; 14:F4; 15:G1, A8; 16:F4; 17:F6; 18:A9; 20:G1; 26:G2; 30:H9; 31:D4
flexibility 2:E4,G5; 4:A9; 11:A10; 12:E3; 19:A7; 20:B6; 22:E7,E8; 32:B3
flights 25:A4
formula (R) 25:G10
forward selection 6:E6; 23:A9
FPR *see* false positive rate (FPR)

G

Gauss, Carl Friedrich 23:B6
Gauss-Markov theorem 8:E8
gene-expression example 24:A4
generative AI 22:A8
generative model 11:G2,C9
geom_abline() 16:C3
geom_bar() 5:C8; 16:D2; 20:E9
geom_boxplot() 5:C8; 16:C2; 24:F6; 27:C8; 28:C6
geom_histogram() 5:C8; 16:D1; 24:G5; 25:F7; 26:G7
geom_jitter() 16:C2; 27:C8; 28:C5,C6
geom_point() 5:C9; 16:D1; 24:F5
geom_smooth() 5:C9; 16:D2; 28:G2
geom_text() 27:D7
geom_tile() 27:B6,C7
ggplot() 5:D11; 15:E11; 24:G5; 26:G7; 27:C6,C7; 28:B9
ggplot2 15:C4,E10; 25:G2; 27:B6
ggthemes 15:D5; 28:C9
glimpse() 15:E8
glm 10:D7,A11; 15:G4; 16:H8; 20:A10,E10
glm() 9:E4,H4; 17:D10; 23:H4; 24:C3
group_by() 15:E7,G9; 25:A6,B7; 29:G7

H

head() 15:E8; 24:H3; 25:E2,F11; 26:C1; 28:E5
height 28:F4
heteroscedasticity 7:F8; 33:E8
hierarchical clustering 22:G5
hierarchical principle 7:G3; 20:D7
high leverage point 7:E9
high-dimensional data 22:D9
histogram 5:B8; 20:E5; 24:E8; 25:E7; 26:F7; 28:A2
homework 20:E2; 27:E11
hwy 20:E5; 28:B2,B3,B5
hyperparameter 4:E2,A9; 26:C11
hyperplane 24:C6
hypothesis test 2:B7

I

identifier variable 5:B2; 8:A7; 17:E7; 18:B8, H8; 19:E5,F5; 20:E9; 30:E8
if_else() 15:H7; 24:G4
imp_vars() 6:D1; 12:B7,B10; 15:F3; 16:C8; 19:G7; 20:H10; 24:D10; 30:B3
imputation 5:E8,F10; 12:B10; 30:A2
impute_with 24:D10; 30:B3
Income data 22:E7
indicator function 3:E5
inference 1:F9; 18:B6; 20:E5
inferential model 2:A7,B7; 20:F3
influential observation 7:E10; 33:E10
initial_split() 6:D3; 10:D6; 13:F9; 15:B7,D9; 16:A5; 17:F5; 19:G10; 20:C4, E6; 25:H8; 26:H8
install.packages() 22:C10; 26:E4
interaction term 3:A5; 5:F7; 7:A8,B10; 12:C10; 18:F5,B7; 19:E4; 20:B7,G10; 23:E1; 24:D10; 30:A2; 31:A1,C1,E1,H2; 33:A7
intercept 6:D9; 7:B7; 18:F2; 26:A6,B6; 27:E1; 31:C4
interpretability 1:E9; 2:E4; 11:B10; 18:D6; 23:C6; 32:A3
irreducible error 3:E2,E3; 20:D5,F8; 23:A2
is.na() 29:F5
ISLR 22:B3; 23:B5; 25:B1,C2; 27:B4
ISLR2 27:B4

K

k-means clustering 22:G5
k-nearest neighbors *see* k-nearest neighbors (KNN)
K-nearest neighbors *see* k-nearest neighbors (KNN)
k-nearest neighbors *see* k-nearest neighbors (KNN)
k-nearest neighbors (KNN) 1:E7; 4:A3; 8:E5, F10; 11:A9; 12:E8; 16:E8; 18:A5; 19:A9; 20:A5; 22:G5,G8; 25:F9; 26:A10; 28:E3; 29:B3; 30:G1,F2,E7,E8; 31:A8
k-NN *see* k-nearest neighbors (KNN)
k-NN *see* k-nearest neighbors (KNN)
kable() 26:C1,G5
kable_styling() 25:G11; 26:C2
kableExtra 25:E11
Kaggle 22:B8
kbl() 25:G11
kknn 4:E8,F9; 8:G2; 12:D8; 15:D5,D11; 20:E7; 26:A11; 30:H9
klaR 10:D6; 11:F10,H11; 15:D5; 16:G9
KNN *see* k-nearest neighbors (KNN)

L

labs() 15:E11; 16:D2

lasso 22:G5
LD1 24:A7,B8
LD2 24:C8
LDA *see* linear discriminant analysis (LDA)
least squares **6:D10**; 23:D7; **33:A5**
`levels()` **15:E8**
`library()` **15:C5**; 25:D1
`library(discrim)` **11:E10**
likelihood ratio 23:E11
linear discriminant (LD1, LD2) 24:A7,A8
linear discriminant analysis (LDA) **8:E10**; **10:E4,E6**; **11:A2**; **16:E8**; **18:H4**; **19:E10**; **20:A8**; 24:A1,A2; 27:A4,C4,E8,E9; 29:E3; 30:B8; 31:A6,A10; **35:G8**
linear imputation **5:E10**; **20:F10**; 24:D10; 26:F8; 27:C2; 30:A2
linear model 23:A4
linear probability model 23:E4
linear regression **2:E3**; **3:A3**; **5:A4**; **6:A6**; **9:B1**; **16:E8**; **19:E2**; 22:G5; 23:A4,E4,B7; 25:F9; 26:A5; 27:D2,B10; 28:E3; 31:A3,E3, H5
`linear_reg()` **6:E7**; **7:C6**; **8:G2**; **15:D10**; **16:E8**; **20:E7**; 26:A5,C10; 31:D3
`linetype` 28:G2
`lm` **8:G2**; **15:D10**; **16:H8**; 31:D4
`lm()` **17:D10**
log odds **9:C5**,**C11**; **17:C9**; **18:E3**,**G10**; **19:A11**,**B11**; 30:C8; 31:E2
log transformation **5:C10**; **7:G6**
logistic function **9:C4**; **21:B2**; 30:H6
logistic regression **2:A3**; **8:F10**; **9:A3**; **10:E3**; **13:A4**,**B4**; **16:E8**; **18:B11**; **19:A11**; **20:A8**; 22:G5; 23:E3; 27:A4,G9,A10,F10; 29:E3; 30:A6; 31:E2,A5; **34:A10**
`logistic_reg()` **10:D7**,**A11**; **15:G4**; **16:E8**; **20:G10**
logit **9:D5**,**C7**; **20:B8**; **21:A2**
`longest_shell` 28:E4

M

machine learning **1:E7**,**E8**; 22:C5,H6
MAE **2:B4**; **8:H3**; **12:E9**,**H10**; **20:H3**,**D7**; 26:B8; 29:D7
`mae()` **8:G3**; **13:C2**; **15:G2**; **17:E2**; 26:C9
main effect **18:F9**; 23:E1; 31:E1; **33:A7**
Manhattan distance **4:A6**
`manufacturer` 28:B4
`MASS` **11:G10**; **16:G9**; **20:G10**
maximum likelihood **9:E3**,**F3**; **11:F1**; **20:B8**; 23:E4,D7
mean imputation **5:F10**; 27:D2
mean squared error (MSE) **2:A4**,**E8**; **12:F2**, **E9**; **20:B5**
`mean()` 25:C6; 26:G7
mechanistic model **2:E1**; **20:F4**
`metric_set()` **5:C2**; **8:G3**; **12:B4**; **13:C2**; **15:G2**,**B9**; **17:E3**; **20:F7**; 26:C9
Minkowski distance **4:A6**
missing at random (MAR) **5:F9**; 23:D6
missing completely at random (MCAR) **5:H8**; 23:D6
missing data **5:B7**,**E8**; **20:E9**; 25:F9; 26:E4; 29:E5
missing not at random (MNAR) **5:E9**; 23:A6
missingness pattern 23:C5
mixed selection **6:G6**
MLE 23:E4
`mode` **16:E7**,**E9**
model comparison **11:A8**; **12:E1**; **18:A4**; 26:E3; 27:G9; 29:D9; 30:E7
model complexity 22:E7
model evaluation **12:E1**; **17:E1**; **18:A4**; 22:F6; 23:E3,E9; 24:E10; 27:E3; 30:F5,E7; 31:A7
`mpg` **5:F2**,**A10**; **15:G9**; **20:E3**; 25:C2; 28:A2
MSE *see* mean squared error (MSE)
multi-class classification **10:G3**
multiclass classification 23:E9; 24:E10; 27:F5
multiclass confusion matrix **13:E4**
multicollinearity **8:B2**
multiple linear regression **6:C8**; 23:A9
multiple logistic regression **9:E8**; 23:F7
multivariate normal distribution **11:D2**
`mutate()` **7:E7**; **8:D10**; **15:H2**,**E7**; **20:F6**; 24:G4; 25:B5; 29:E6

N

`n()` **15:F7**,**G9**; 25:B10; 28:F7
`na.rm` **15:H10**; 26:G8
naive Bayes **8:F10**; **11:E8**; **16:E9**; 22:G5; 27:A4,D4,E8
`naive_Bayes()` **11:G11**; **12:B2**; **16:E9**
`naniar` **5:F8**; **15:D5**; **16:A4**; 26:E4
`nearest_neighbor()` **4:E8**,**G10**; **8:G2**; **12:D8**; **15:D10**; **16:E8**; **19:E9**; **20:F7**; 26:G2,A10,B10; 30:H9
negative predictive value (NPV) **13:F2**
`neighbors` **4:G8**,**G10**; **8:G2**,**G6**; **12:D8**; **15:D10**; **16:G8**; **20:G11**; 26:G2; 30:H9
Netflix Challenge 22:A8
neural network 22:B6
`new_data` 26:C1; 27:H3; 28:H9
non-parametric model **2:E1**,**F2**; **4:F7**; **8:G5**; **11:A9**,**A10**; **20:F4**,**D5**; 31:E1,A8
nonlinear relationship **5:E6**; **7:A8**; **18:A9**; 22:E9; 23:E2; 30:E10; 31:C8; **33:G5**; **34:B8**
normalization **4:E4**; **8:A8**; **20:B6**; 31:E3,G5
`nycflights13` 25:B4

O

observation 22:A10
odds **9:C4**,**D8**; **21:D2**
odds ratio **9:C8**; **17:B9**; **19:A11**; **20:C8**; **21:A2**; 30:D8
`OJ` **10:B6**; **12:B2**; **13:G9**; **14:G11**; **15:G2**; 27:F1,C5,G10
one-hot encoding **7:B6**; **16:C7**; **20:C7**; 31:A3
`one_hot` **5:A3**; **7:A5**,**A6**; **16:B7**; **18:C9**; 30:H8; 31:D3
orthogonal polynomial contrast 26:D2
outcome **1:F10**; **5:E9**; **18:B10**; **19:G5**; 25:E3,E7,G10; 27:B5
outlier **5:C10**; **7:E9**; **20:E6**; 28:D3
`outlier.shape` 28:C6
overfitting **2:F3**,**F6**,**E9**; **6:G3**; **11:E8**; **12:E4**, **D10**; **18:A6**; **19:C9**; **20:F4**,**G7**; 22:E7,E8; 23:E2; 26:E3,F3; 30:E7; **32:B8**

P

p-value **2:B7**; **6:E5**; **9:H5**; 23:G6; 26:A6,B7; 27:E1; 29:A6; 30:D8
parametric model **2:E1**; **6:C8**; **8:E5**; **11:A9**, **C10**; **20:H4**
`parch` 29:H4
`parsnip` **4:E8**; **15:B4**; **16:E7**; 22:A10; 26:C5,A11
`passenger_id` 29:E4
`pclass_X2` 30:A6
`pclass_X3` 30:B6
Pearson correlation **3:B5**; **5:E3**; **18:C7**; **20:A2**,**H10**; 28:D8; 29:G10; 30:E10
percent stacked bar chart **20:E9**; 29:F5
perfect separation **9:E7**; 24:A2
`pivot_wider()` 29:E2
plane 24:C6
`poissonreg` 27:A4,B4
polynomial contrasts **6:G10**
polynomial regression **7:E5**; 23:E2; **33:E3**
pooled variance **10:G8**
`position = "fill"` **16:D2**
posterior probability **10:F5**; **20:F2**; 24:F2; 27:G5; 31:A5
practice exam **18:A5**
precision **2:C5**; **13:E2**; **19:D3**; **20:B9**
`precision()` **17:F3**
`predict()` **4:H11**; **5:D1**; **8:H2**,**C9**; **10:D8**, **A9**; **13:B2**; **15:C3**,**A9**; **16:E4**; **17:D5**,**E6**; **19:F9**; **20:E1**,**A3**,**E7**,**E11**; 26:B7; 27:H3, H5; 30:E3
predicted probability **9:E11**; **10:A1**,**B3**
predicted vs. actual plot 26:B8
prediction **1:E8**; **17:A3**; **20:F5**; 22:B4,G4, C8; 26:B7
predictive model **2:A7**,**B8**; **20:F3**
predictor **1:E10**; 22:F2; 23:D4; 25:F5,E11; 26:G3; 31:C2; **34:F3**
`prep()` **6:B6**; **8:A9**; **16:G3**,**E4**,**B7**; 25:G11; 26:G9,E10; 28:H9; 30:D3
principal component analysis (PCA) 22:G5
prior probability **10:E4**; 30:A11
projection 24:A4
`prop` **6:A3**,**D3**; **13:F9**; **15:D9**; **16:A5**; **20:H6**; 26:H8
Purchase (OJ) 27:C5

Q

QDA *see* quadratic discriminant analysis (QDA)
quadratic discriminant analysis (QDA) **8:E10**; **11:A5**,**C6**; **16:E9**; **18:H4**; **19:E10**; **20:C8**; 24:A1; 27:A4,D4,E8; 29:E3; 31:A6,C9,A10,B11; **35:F10**
qualitative response **8:E10**
`quantile()` 28:H7

R

R code **15:A3**
R output **20:A3**
R-squared **2:B4**; **6:E3**,**F10**; **12:E10**; **20:H3**, **A7**; 26:B8; 29:D7
random seed **6:C4**
rank deficiency **7:C6**; **18:F1**; 31:A3,C4
`read_csv()` **8:D10**; **12:D5**; **15:G2**
`readr` **15:D4**
recall *see* true positive rate (TPR, sensitivity, recall)
`recipe()` **4:G9**; **7:D4**; **15:B8**,**D10**; 24:D10; 25:E10,F10,G10; 27:A10; 30:B2,H8; 31:D3,H4
`recipes` **8:A4**; **15:D4**; **16:A6**; **19:E6**; 22:A10; 23:A10; 30:E8
reducible error **2:B8**; **3:E2**,**F2**; **20:H7**
reference level **7:A2**,**A5**; **9:G6**; **16:B7**; **18:H2**; **19:F3**; 30:D6
regression **2:A4**; **12:E9**; **20:F3**,**D5**; 22:F3, H6
regression line 22:B4
regression plane 23:A9
regularization 22:G5
reinforcement learning 22:H6
`reorder()` 25:G6
reproducibility **6:C4**; **16:D5**; **17:E5**; **20:F1**
resampling 25:G8; 26:F1
residual **6:D9**; 22:G4; 23:C7,A9; **33:E5**
residual plot **7:E7**,**F8**
residual standard error (RSE) **6:G3**; **12:E10**
residual sum of squares (RSS) **6:D10**
ridge regression 22:G5
`rings` 28:F4,G9
`rmarkdown` 22:A9
RMSE *see* root mean squared error (RMSE)
`rmse()` **8:G3**; **13:C2**; **15:G2**; **17:E2**
ROC AUC *see* AUC (area under the ROC curve)
ROC curve **10:B4**; **13:E10**; **17:G3**; **18:F4**,**E7**; **19:A5**; **20:A10**; 23:E11; 24:A8,H9,E10; 30:F3; 31:A10,B10; **34:E8**
`roc_auc()` **10:G2**; **12:D9**; **14:E6**,**E10**; **15:B1**,**G5**; **17:E3**; **19:E9**; **20:H11**
`roc_curve()` **12:B4**,**C9**; **14:G6**,**E10**,**E11**; **15:H5**; **17:E3**; **21:A2**; 24:E2; 27:H10
root mean squared error (RMSE) **2:B4**; **6:E10**; **12:E9**,**E10**; **20:G3**,**D7**; 26:D10; 29:D7
`rplot()` **16:D3**; 27:A6
`rsample` **15:C4**; **16:A5**; 22:A10
`rsq()` **8:G3**,**E4**; **12:E11**; **13:C2**; **15:G2**; **17:E2**; 26:C9

S

Sai midterm **18:E5**
sampling variability 23:A8
`scale_fill_manual()` 24:G6
scaling **8:C8**
scatter 24:A6
scatterplot **5:B9**; **20:F5**; 22:A4; 23:E8; 24:A5; 28:A3; 30:E10
`scroll_box()` 25:G11; 26:C2
`se = FALSE` 28:G2
`select()` **5:D11**; **15:E6**; 24:H3; 25:A5; 26:B7; 29:G9
self-driving car 22:D8
sensitivity *see* true positive rate (TPR, sensitivity, recall)
`sensitivity()` **14:G5**,**F10**; **17:E2**
`set.seed()` **6:D3**,**A4**; **15:B7**,**D9**; **16:A5**; **17:E5**; **19:F10**; **20:E1**
`set_args()` **11:G11**; **16:G10**
`set_engine()` **4:F9**,**G10**; **6:E7**; **8:G2**; **10:D7**; **11:G10**; **15:C8**,**D10**; 26:A11; 30:H9; 31:D4
`set_mode()` **2:D6**; **4:F9**,**G10**; **8:G2**; **10:D7**; **15:C8**,**D11**; **19:F9**; 30:H9
`sex` 29:G4
`sex_male` 30:C6
`shell_weight` 28:E4
`Shiny` 22:A9
`shucked_weight` 28:G4
`sib_sp` 29:G4
sigmoid **9:C5**
simple linear regression **6:C7**; 23:A7
simple logistic regression 23:F6
Simpson's paradox **9:G10**; 23:E8
skewness **5:D10**; **20:E5**; 26:G7
slope **6:B9**
`Smarket` **10:B6**; **12:B2**; **13:E6**,**E9**; **14:G10**; **15:G2**; 27:B4,B5
specificity *see* specificity (TNR)
specificity (TNR) **10:A4**; **13:F9**,**A11**; **18:E6**; **19:D3**,**B4**; **20:D9**; **21:D1**; 24:F5,G10; 27:F10
`specificity()` **14:G5**,**G10**; **17:E3**
standard error **6:F1**; 23:G6; 26:A6; 27:E1; 29:A6
standardization **4:E4**; **8:B8**; **18:C6**; 30:H2
`starts_with()` **7:H4**,**E5**; **12:D7**,**A10**; **15:H4**,**G6**; **16:B10**; **20:G10**; 24:D10; 30:D3
statistical learning 22:C5
statistical machine learning **1:E8**,**F9**
`step_center()` **8:G1**,**B8**; **16:C8**; **19:G7**; **20:G6**
`step_dummy()` **4:E8**,**G9**; **5:B3**; **6:F6**,**F10**; **7:B4**,**G5**,**A6**; **12:D7**,**D10**; **15:D10**; **16:F1**, **A7**; **18:F1**; **19:F7**,**E10**; **20:F6**,**G10**; 23:A10; 24:C10; 27:D10; 30:A2,C3,B6,H8; 31:D3
`step_impute_bag()` **5:F10**; **16:A9**
`step_impute_knn()` **5:G10**; **16:A9**
`step_impute_linear()` **5:E10**; **6:D1**; **12:B7**; **15:F3**; **16:A8**; **19:F7**; **20:E10**; 24:D10; 29:F5; 30:B1,B2
`step_impute_mean()` **5:E10**; **16:A8**; **19:F7**; 27:D2
`step_impute_mode()` **5:G10**; **16:A8**
`step_interact()` **7:E4**,**H4**; **12:D7**; **15:H4**; **16:A8**; **19:F7**,**G10**; **20:G6**,**G10**; 24:D10; 30:D3
`step_log()` **7:G6**; **16:A9**; **19:G8**
`step_mutate()` **16:B9**; 31:H4
`step_normalize()` **4:G5**,**G9**; **5:A3**; **8:F2**, **A8**; **15:D10**; **16:A8**; **17:E7**; **18:C9**,**G10**; **19:F7**,**G9**; **20:E2**; 30:F2; 31:H4
`step_poly()` **7:E6**; **16:A9**; **19:F8**
step_rm() **16:A9**
`step_scale()` **8:F2**,**D8**; **16:B8**; **19:F7**; **20:E6**
stepwise selection 23:A9
`strata` **6:D3**,**A4**; **10:D6**; **12:D5**; **13:F9**; **14:B10**; **15:D9**; **16:A5**; **19:E10**; **20:A2**,**H6**, **G8**; 25:F8; 26:H9
stratified sampling **6:B4**; **12:B5**; **13:E9**; **16:B5**; **20:H6**,**H9**,**A11**; 25:H8; 28:F8; 29:F4
`stretch()` 27:C6
structural collinearity **8:B3**
studentized residual **7:G9**
`summarise()` **15:E7**,**G9**; 25:A6,C6; 28:F7; 29:H7
`summary()` **9:H4**; **15:E8**; **17:D10**
supervised learning **2:A1**,**A2**; **20:E3**,**A5**; 22:G5,H6
SVM 22:B6

T

t-statistic **6:E5**; **19:E3**; 26:A6; 29:A6
`table()` **15:E9**
target leakage **5:C2**; **8:A5**; **14:C7**; **17:E5**,**H7**; **18:D8**; **19:E5**,**H5**,**B6**; **20:H6**,**B10**; 29:B1; 30:E8
test MSE **2:E8**,**G9**; 22:E8; 23:A2
testing data **11:G8**; **20:B6**; **21:C1**; 25:F8; 27:E10; 30:F5
testing error **2:E7**; **3:E5**; **12:E1**,**F2**; **18:B6**; **19:A8**; 26:C10
`testing()` **6:C3**; **15:B7**,**D10**; **16:A6**; 26:H9
`theme_bw()` 24:G6
`theme_gdocs()` 28:B10
threshold **9:E11**; **10:B3**; **13:A4**; **18:F2**,**G6**; **19:A3**; **20:A9**; 24:F2,E4,G7,G9,G10; 31:E2, A10,B10
`tibble` **15:D4**
`tibble()` **17:C6**
tidy data 22:A9
`tidy()` **6:G7**; **10:D8**,**D10**; **15:H1**,**A4**; **17:A7**; **19:E3**; 26:C5; 27:F1,C10
`tidymodels` **4:E8**; **10:D6**; **15:A3**,**B4**,**C5**,**A7**; 22:A9,C10; 23:A10; 25:F9
`tidymodels_prefer()` **15:A5**,**D5**

`tidyverse` **15:A3**,**A4**,**C5**; 22:D9; 24:F11
tie-breaking **4:E4**
`titanic` **12:A4**,**A5**; **14:A9**; **15:F2**; **20:E8**; 24:A8; 29:E3
TPR *see* true positive rate (TPR, sensitivity, recall)
training data 22:A7; 23:B8; 25:F8; 27:E8
training error **2:E7**; **3:E5**; **11:E7**; **12:E1**,**F2**, **B4**; **18:D6**; **19:A8**; 26:C10; **32:A6**
training MSE **2:E8**,**E9**; 22:E8
training set 22:A7
`training()` **6:C3**; **15:A7**,**D10**; **16:A6**; 26:H9
training/testing split **6:A2**; 25:E8; 28:E9; 29:F4
transformation **7:H6**
`transmute()` **15:E7**
true negative **13:D9**
true positive **10:C3**; **13:A9**; **19:A1**
true positive rate (TPR, sensitivity, recall) **10:D4**; **13:E9**,**A10**,**B10**,**C10**,**E10**; **18:H3**,**F6**; **19:C3**,**A4**; **20:F3**,**A9**,**B9**; **21:C1**; 24:G5,H5,E7,F10; 27:E10
`truth` **5:C1**; **8:G3**; **13:C2**; **15:B1**,**G2**; **17:F2**; 26:D9
tuning parameter **4:F2**,**A9**; **8:F6**; 26:G1,C11
`type` 28:H4
`type = "prob"` **4:G11**; **10:D8**,**A9**; **12:D8**; **14:F4**; **17:B4**,**D5**; 27:H3,H5
Type I error **13:A9**
Type II error **13:D9**

U

underfitting **2:E10**; **12:E5**; **19:B9**; 22:E10
`ungroup()` **15:H7**
unsupervised learning **2:A1**,**C2**; **20:G3**,**B5**; 22:G5,H6
upsampling 27:A9
`use = "pairwise.complete.obs"` 29:G9
`usekernel` **11:E9**,**G11**

V

validation set 22:E10
variance **3:A7**; **11:E3**; **18:B6**,**G6**; **19:C7**; **20:H4**; 23:A2; 31:A10,B11; **32:B10**
variance inflation factor (VIF) **8:D2**
`vis_miss()` **5:F8**; **6:D2**; **16:B4**; 26:H4; **34:A5**
`viscera_weight` 28:E4

W

`where()` **15:G6**
`whole_weight` 28:G4
`workflow()` **4:G10**; **5:B3**; **8:G2**; **10:D8**; **15:B8**,**D11**; **18:C9**; **19:E8**,**F9**; **20:F1**,**F7**; 26:G1; 31:D4
`workflows` **15:C4**,**A7**

X

`xlab()` 24:G6; 27:C8

Y

`yardstick` **12:E11**; **14:E4**; **15:C4**,**B9**; **17:E1**; **20:A10**; 22:A10; 29:B7

Z

z-statistic **9:F5**; 23:G6